Chaser.

Diseño y validación de una herramienta para la evaluación de ataques físicos en sistemas embebidos

Iker Martín Canut

Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Universidad Nacional de Rosario

IKERLAN S. Coop · Igor Armendariz

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Motivación

Motivación

Seguridad Física de Sistemas Embebidos

Motivación

Seguridad Física de Sistemas Embebidos

Los mismos pueden operar

Motivación

Seguridad Física de Sistemas Embebidos

Los mismos pueden operar

- En entornos físicamente inseguros

Motivación

Seguridad Física de Sistemas Embebidos

Los mismos pueden operar

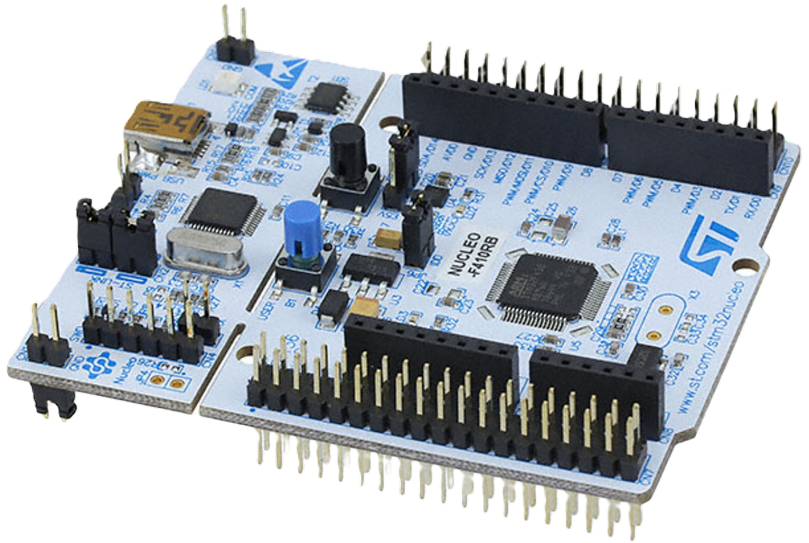
- En entornos físicamente inseguros
- Bajo restricciones de rendimiento, consumo, memoria y costos

Motivación

Seguridad Física de Sistemas Embebidos

Los mismos pueden operar

- En entornos físicamente inseguros
- Bajo restricciones de rendimiento, consumo, memoria y costos



Clasificación de Seguridad

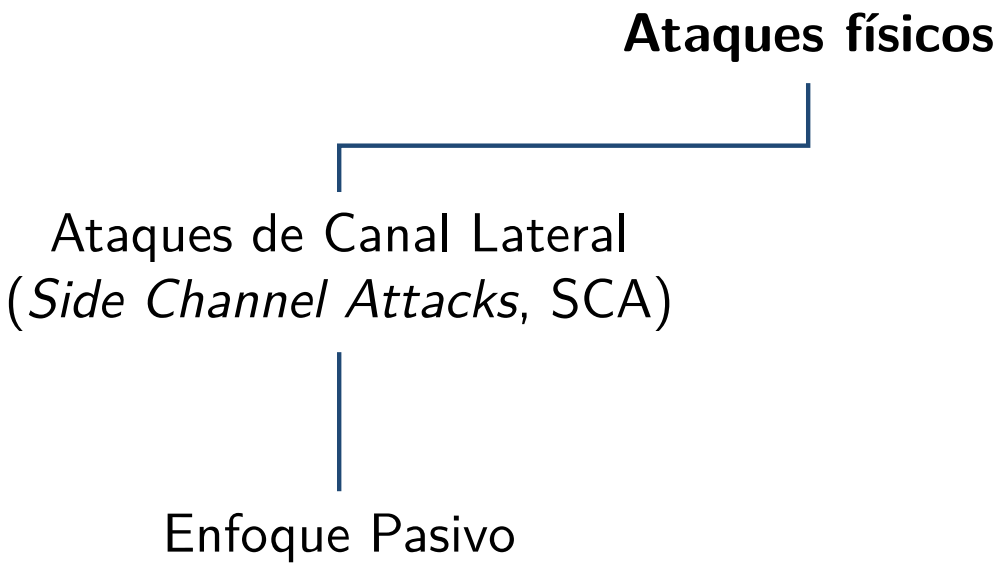
Ataques físicos

Clasificación de Seguridad

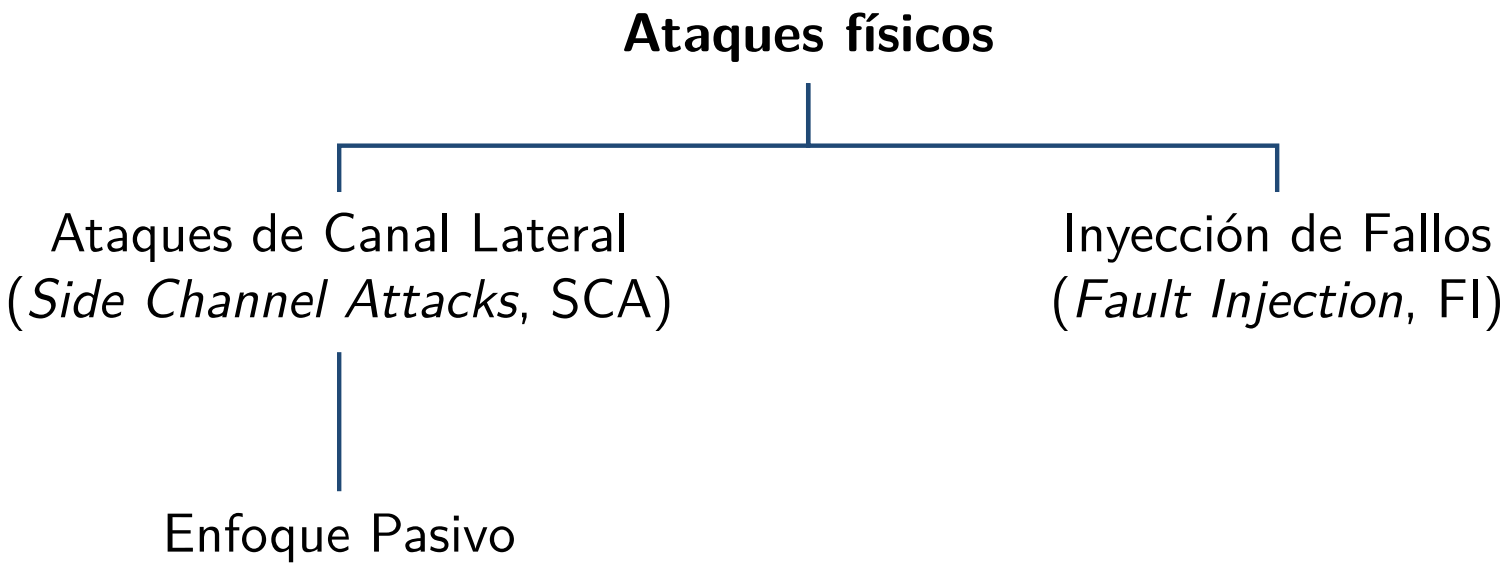
Ataques físicos

Ataques de Canal Lateral
(*Side Channel Attacks*, SCA)

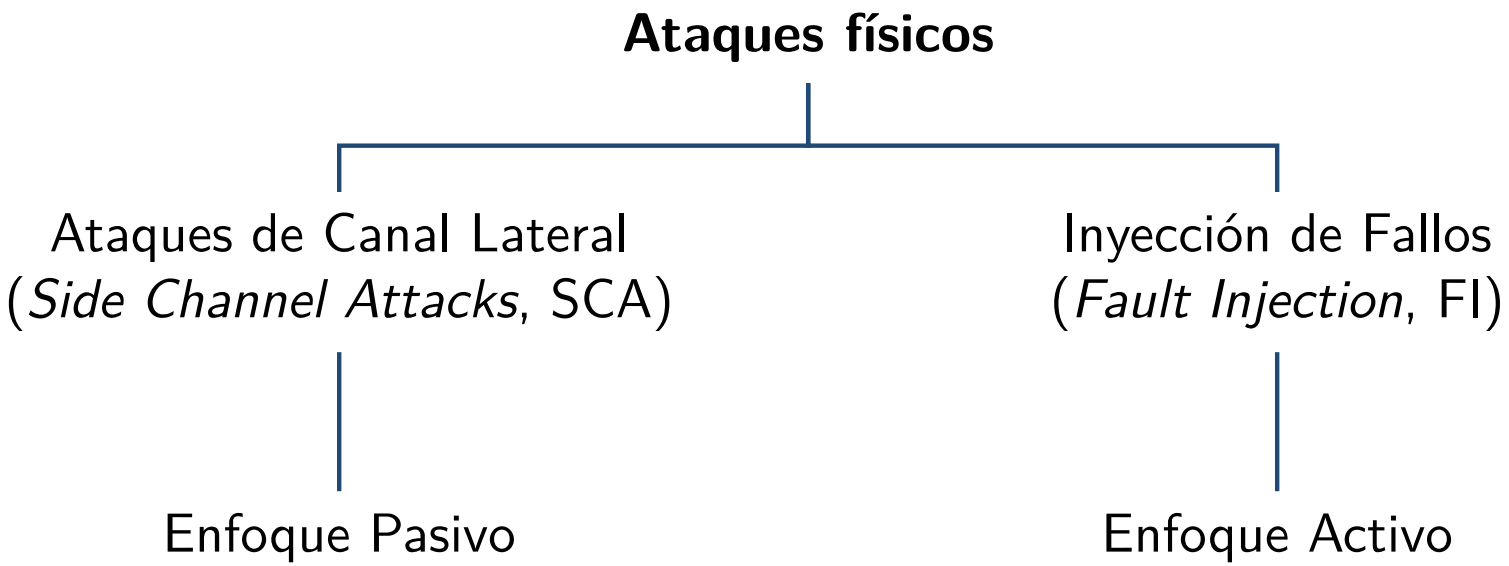
Clasificación de Seguridad



Clasificación de Seguridad



Clasificación de Seguridad



Problema

Problema

Existe una brecha instrumental y metodológica.

Problema

Existe una brecha instrumental y metodológica.

Suele haber código propietario cerrado, que genera dependencia tecnológica y no responde a todas las necesidades internas.

Problema

Existe una brecha instrumental y metodológica.

Suele haber código propietario cerrado, que genera dependencia tecnológica y no responde a todas las necesidades internas.

Las soluciones de código abierto suelen ser distintas bibliotecas incompletas que no manejan diferentes inyectores de fallos.

Problema

Existe una brecha instrumental y metodológica.

Suele haber código propietario cerrado, que genera dependencia tecnológica y no responde a todas las necesidades internas.

Las soluciones de código abierto suelen ser distintas bibliotecas incompletas que no manejan diferentes inyectores de fallos.

Cuando es frecuente modificar parámetros, repetir experimentos y comparar resultados, es crucial tener una plataforma común.

Objetivos

Objetivos

1. Desarrollo del módulo de FI.

Objetivos

1. Desarrollo del módulo de FI.
2. Reingeniería del módulo de SCA ya existente.

Objetivos

1. Desarrollo del módulo de FI.
2. Reingeniería del módulo de SCA ya existente.
3. Diseño de la interfaz gráfica general.

Objetivos

1. Desarrollo del módulo de FI.
2. Reingeniería del módulo de SCA ya existente.
3. Diseño de la interfaz gráfica general.
4. Validación Experimental.

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Definiciones

Definiciones

- **Fallo (*Fault*):** Representa un defecto físico, imperfección o anomalía (voluntaria o involuntaria).

Definiciones

- **Fallo (*Fault*):** Representa un defecto físico, imperfección o anomalía (voluntaria o involuntaria).
- **Error (*Error*):** Es la manifestación explícita de un fallo. Ocurre cuando un fallo altera un estado interno del sistema.

Definiciones

- **Fallo (*Fault*):** Representa un defecto físico, imperfección o anomalía (voluntaria o involuntaria).
- **Error (*Error*):** Es la manifestación explícita de un fallo. Ocurre cuando un fallo altera un estado interno del sistema.
- ***Glitch*:** Acción de inducir un *fallo transitorio de manera controlada*.

Modelos de Fallos (*Fault Models*)

Modelos de Fallos (*Fault Models*)

Efectos lógicos que un glitch puede producir:

Modelos de Fallos (*Fault Models*)

Efectos lógicos que un glitch puede producir:

- ***Bit Flip* (Inversión de bit):** Cambiar el valor lógico de un bit específico por su opuesto.

Modelos de Fallos (*Fault Models*)

Efectos lógicos que un glitch puede producir:

- ***Bit Flip* (Inversión de bit):** Cambiar el valor lógico de un bit específico por su opuesto.
- ***Bit Set / Bit Reset*:** Forzar un bit concreto a un 1 (*set*) o 0 (*reset*).

Modelos de Fallos (*Fault Models*)

Efectos lógicos que un glitch puede producir:

- ***Bit Flip* (Inversión de bit):** Cambiar el valor lógico de un bit específico por su opuesto.
- ***Bit Set / Bit Reset*:** Forzar un bit concreto a un 1 (*set*) o 0 (*reset*).
- ***Random Byte* (Byte aleatorio):** Transformar un byte completo a un valor aleatorio.

Modelos de Fallos (*Fault Models*)

Efectos lógicos que un glitch puede producir:

- ***Bit Flip* (Inversión de bit):** Cambiar el valor lógico de un bit específico por su opuesto.
- ***Bit Set / Bit Reset*:** Forzar un bit concreto a un 1 (*set*) o 0 (*reset*).
- ***Random Byte* (Byte aleatorio):** Transformar un byte completo a un valor aleatorio.
- ***Instruction Skip* (Salto de instrucción):** El procesador ignora la instrucción en curso.

Utilidad Práctica de los Fallos

Utilidad Práctica de los Fallos

- **Omisión de instrucciones:** En las instrucciones de control de flujo, puede forzar a tomar una ruta de código no deseada.

Utilidad Práctica de los Fallos

- **Omisión de instrucciones:** En las instrucciones de control de flujo, puede forzar a tomar una ruta de código no deseada.
- **Lectura/Escritura incorrecta de datos:** Se altera la transferencia de datos entre la memoria y los registros.

Utilidad Práctica de los Fallos

- **Omisión de instrucciones:** En las instrucciones de control de flujo, puede forzar a tomar una ruta de código no deseada.
- **Lectura/Escritura incorrecta de datos:** Se altera la transferencia de datos entre la memoria y los registros.

El efecto exacto de un fallo es inherentemente probabilístico.

Técnicas de Inyección de Fallos

Técnicas de Inyección de Fallos

- Inyecciones de **Reloj**.

Técnicas de Inyección de Fallos

- Inyecciones de **Reloj**.
- Inyecciones de **Voltaje**.

Técnicas de Inyección de Fallos

- Inyecciones de **Reloj**.
- Inyecciones de **Voltaje**.
- Inyecciones **Electromagnéticas**.

Inyecciones de Reloj

Inyecciones de Reloj

Manipular la señal de sincronismo del reloj.

Inyecciones de Reloj

Manipular la señal de sincronismo del reloj.

De manera simplificada, en arquitecturas síncronas, el sistema procesa una instrucción en dos etapas consecutivas.

Inyecciones de Reloj

Manipular la señal de sincronismo del reloj.

De manera simplificada, en arquitecturas síncronas, el sistema procesa una instrucción en dos etapas consecutivas.

- **Fase de Carga**: La instrucción se lee de la memoria.

Inyecciones de Reloj

Manipular la señal de sincronismo del reloj.

De manera simplificada, en arquitecturas síncronas, el sistema procesa una instrucción en dos etapas consecutivas.

- **Fase de Carga**: La instrucción se lee de la memoria.
- **Fase de Ejecución**: Se procesa la operación.

Inyecciones de Reloj: Cronograma de Ejecución Normal

Inyecciones de Reloj: Cronograma de Ejecución Normal

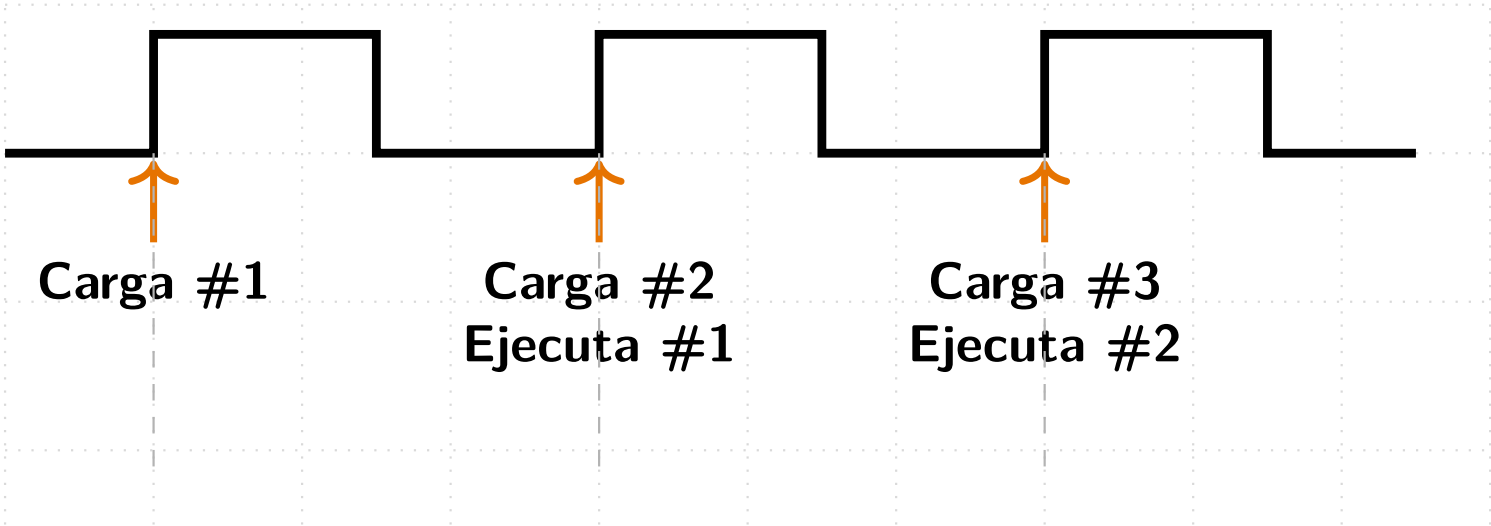


Figura: Flujo temporal de una ejecución sin inyección de fallos.

Inyecciones de Reloj: Cronograma de Ejecución Normal

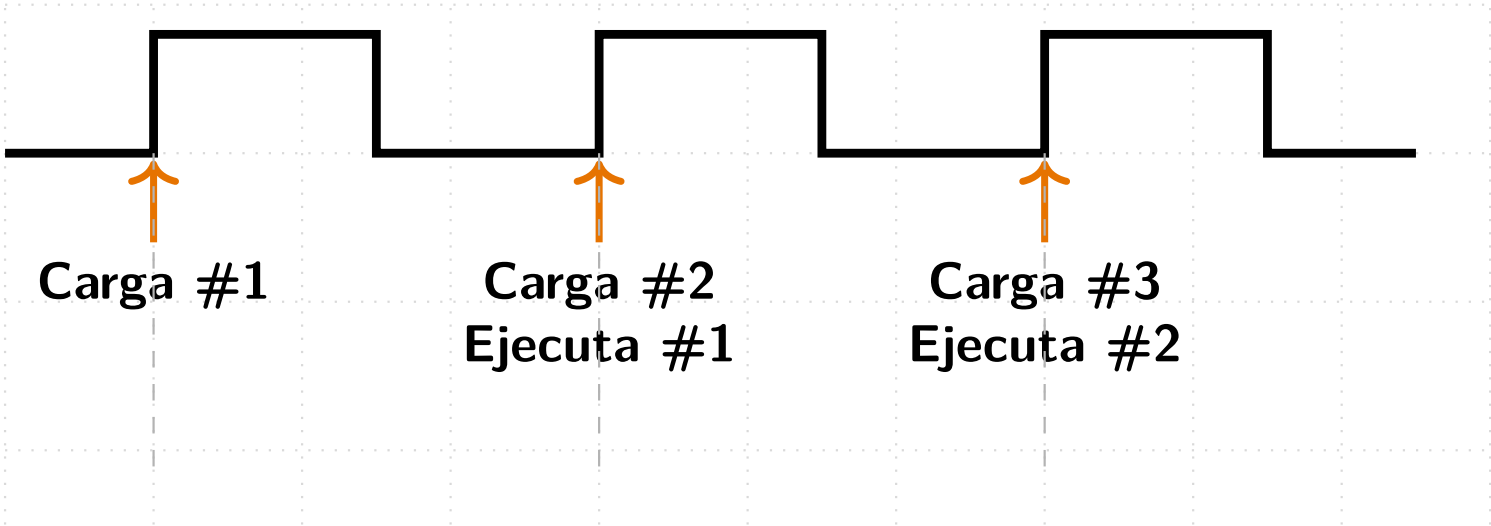


Figura: Flujo temporal de una ejecución sin inyección de fallos.

Se dispone del tiempo completo del ciclo.

Inyecciones de Reloj: Impacto del Glitch en el ciclo

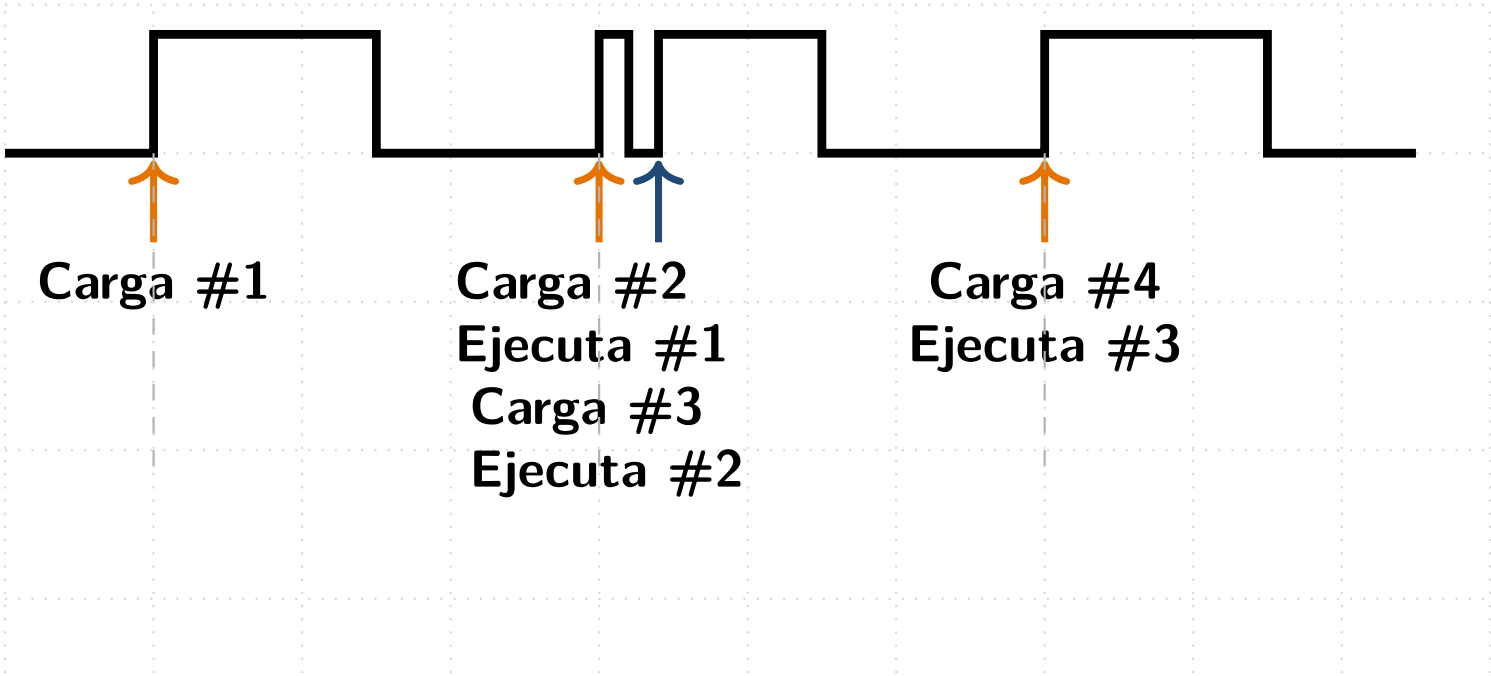


Figura: Impacto del glitch en las fases de carga y ejecución.

Inyecciones de Reloj: Impacto del Glitch en el ciclo

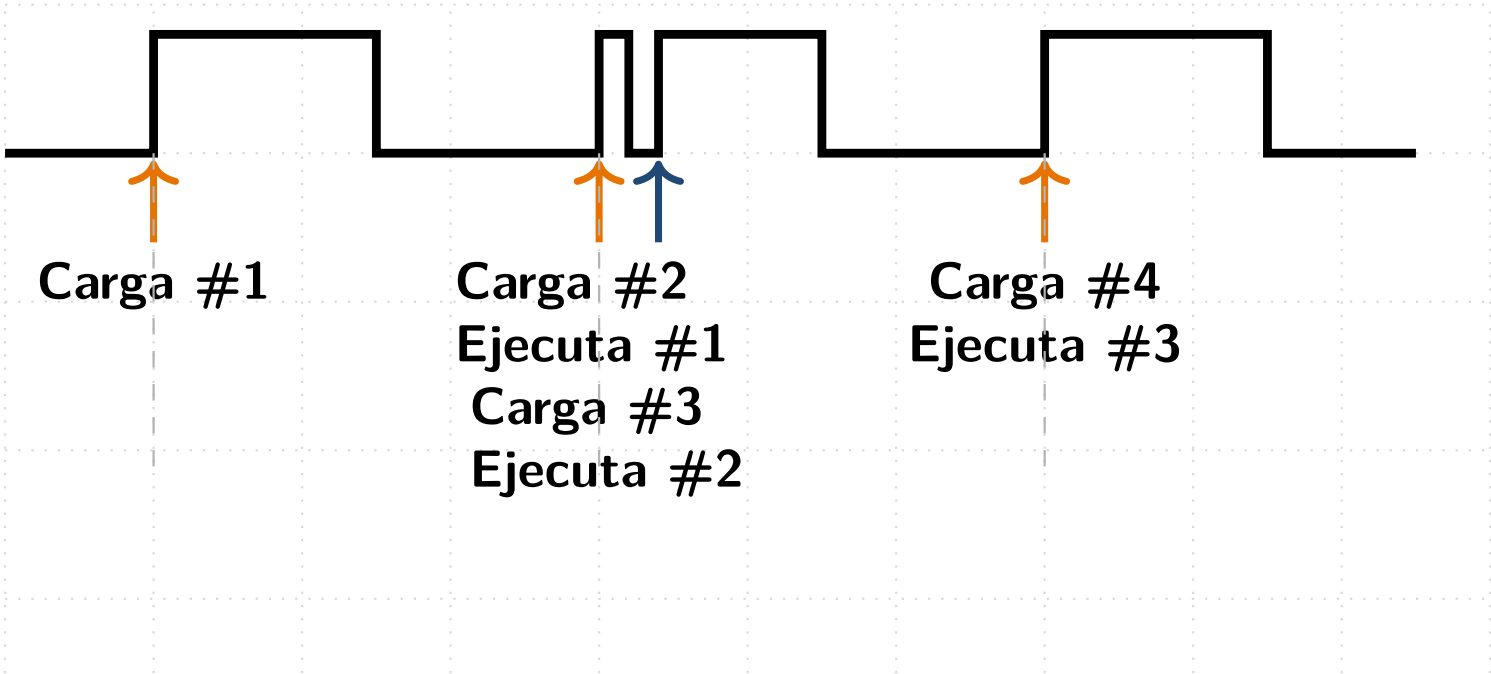


Figura: Impacto del glitch en las fases de carga y ejecución.

Resultado corrupto o el contador de programa podría avanzar.

Inyecciones de Reloj: Modelar con compuerta XOR

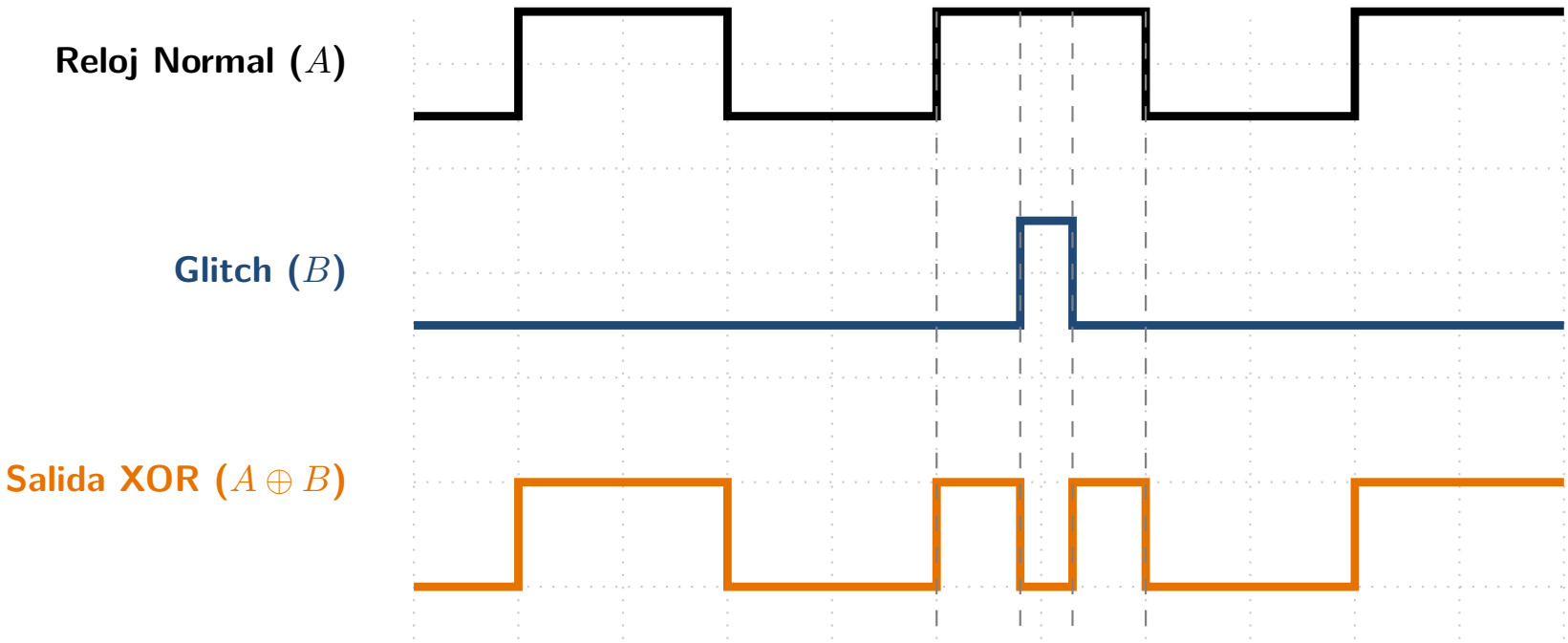


Figura: Generación de un fallo de reloj mediante un XOR.

Inyecciones de Voltaje

Inyecciones de Voltaje

Perturba la línea de alimentación eléctrica.

Inyecciones de Voltaje

Perturba la línea de alimentación eléctrica.

Inducir errores sin provocar un corte de energía prolongado.

Inyecciones de Voltaje

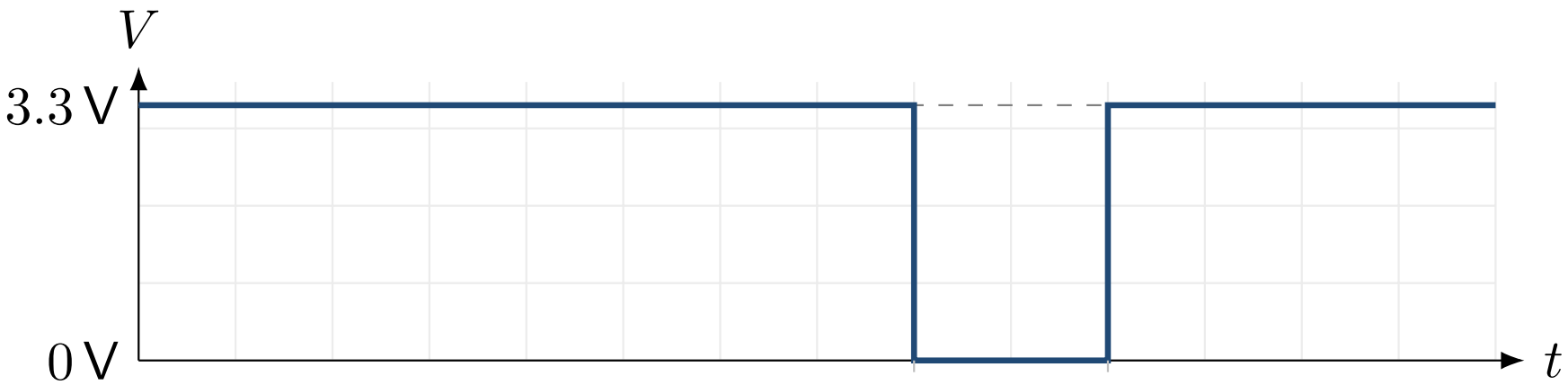


Figura: Caída de voltaje.

Inyecciones Electromagnéticas

Inyecciones Electromagnéticas

Utiliza una sonda que contiene una bobina en su extremo.

Inyecciones Electromagnéticas

Utiliza una sonda que contiene una bobina en su extremo.

Genera un campo magnético transitorio que induce corrientes.

Inyecciones Electromagnéticas

Utiliza una sonda que contiene una bobina en su extremo.

Genera un campo magnético transitorio que induce corrientes.

Al ser un ataque local, tiene la capacidad de eludir defensas:

Inyecciones Electromagnéticas

Utiliza una sonda que contiene una bobina en su extremo.

Genera un campo magnético transitorio que induce corrientes.

Al ser un ataque local, tiene la capacidad de eludir defensas:

- Capacitancia de los condensadores de desacoplo.

Inyecciones Electromagnéticas

Utiliza una sonda que contiene una bobina en su extremo.

Genera un campo magnético transitorio que induce corrientes.

Al ser un ataque local, tiene la capacidad de eludir defensas:

- Capacitancia de los condensadores de desacoplo.
- Filtrado de los circuitos de gestión de energía.

Inyecciones Electromagnéticas

Utiliza una sonda que contiene una bobina en su extremo.

Genera un campo magnético transitorio que induce corrientes.

Al ser un ataque local, tiene la capacidad de eludir defensas:

- Capacitancia de los condensadores de desacoplo.
- Filtrado de los circuitos de gestión de energía.

Deben sumarse las coordenadas espaciales de la sonda.

Sincronización y Mecanismos de Disparo (*Triggers*)

Sincronización y Mecanismos de Disparo (*Triggers*)

La efectividad depende fuertemente de su precisión temporal.

Sincronización y Mecanismos de Disparo (*Triggers*)

La efectividad depende fuertemente de su precisión temporal.

Requiere tareas de ingeniería inversa y/o la introducción de mecanismos de sincronización.

Sincronización y Mecanismos de Disparo (*Triggers*)

La efectividad depende fuertemente de su precisión temporal.

Requiere tareas de ingeniería inversa y/o la introducción de mecanismos de sincronización.

Trigger: señal que se activa antes de la operación sensible.

Sincronización y Mecanismos de Disparo (*Triggers*)

Sincronización y Mecanismos de Disparo (*Triggers*)

Al detectar esto, el ataque se parametriza con:

Sincronización y Mecanismos de Disparo (*Triggers*)

Al detectar esto, el ataque se parametriza con:

- **Offset**: Retardo hasta la activación del fallo.

Sincronización y Mecanismos de Disparo (*Triggers*)

Al detectar esto, el ataque se parametriza con:

- **Offset**: Retardo hasta la activación del fallo.
- **Duración**: Tiempo por el que está activa la perturbación.

Sincronización y Mecanismos de Disparo (*Triggers*)

Al detectar esto, el ataque se parametriza con:

- **Offset**: Retardo hasta la activación del fallo.
- **Duración**: Tiempo por el que está activa la perturbación.
- **Potencia**: Dependiendo del ataque, regular la intensidad.

Sincronización y Mecanismos de Disparo (*Triggers*)

Al detectar esto, el ataque se parametriza con:

- **Offset**: Retardo hasta la activación del fallo.
- **Duración**: Tiempo por el que está activa la perturbación.
- **Potencia**: Dependiendo del ataque, regular la intensidad.

Cuando no es posible insertar triggers, la precisión temporal podría recuperarse analizando las fugas físicas de información.

Inyectores de Glitches: ChipWhisperer

Inyectores de Glitches: ChipWhisperer

Bajo costo.

Inyectores de Glitches: ChipWhisperer

Bajo costo.

Permite hacer ataques de reloj y de voltaje.

Inyectores de Glitches: ChipWhisperer

Inyectores de Glitches: ChipWhisperer

- **ext_offset**: El ciclo de reloj donde se inyecta el glitch.

Inyectores de Glitches: ChipWhisperer

- **ext_offset**: El ciclo de reloj donde se inyecta el glitch.
- **offset**: Dónde comienza el glitch dentro del ciclo de reloj seleccionado.

Inyectores de Glitches: ChipWhisperer

- **ext_offset**: El ciclo de reloj donde se inyecta el glitch.
- **offset**: Dónde comienza el glitch dentro del ciclo de reloj seleccionado. La duración de cada paso es:

$$\frac{1}{56 \cdot f_{\mathrm{VCO}}},$$

donde f_{VCO} es la frecuencia del MMCM, configurable entre $600\,\mathrm{MHz}$ y $1\,200\,\mathrm{MHz}$. La resolución temporal teórica, `phase_shift_steps` (o `pss`), está entre $29.76\,\mathrm{ps}$ y $14.88\,\mathrm{ps}$.

Inyectores de Glitches: ChipWhisperer

Inyectores de Glitches: ChipWhisperer

Específico para **Reloj**:

Inyectores de Glitches: ChipWhisperer

Específico para **Reloj**:

- **width**: Controla la duración del glitch. Medido en `pss`.

Inyectores de Glitches: ChipWhisperer

Específico para **Reloj**:

- **width**: Controla la duración del glitch. Medido en `pss`.

Específico para **Voltaje**:

Inyectores de Glitches: ChipWhisperer

Específico para **Reloj**:

- **width**: Controla la duración del glitch. Medido en `pss`.

Específico para **Voltaje**:

- **repeat**: Durante cuántos ciclos de reloj se envía el glitch.

Inyectores de Glitches: Spider

Inyectores de Glitches: Spider

Costo más elevado.

Inyectores de Glitches: Spider

Costo más elevado.

Mucho más complejo.

Inyectores de Glitches: Spider

Costo más elevado.

Mucho más complejo.

Permite hacer ataques de voltaje y electromagnéticos.

Inyectores de Glitches: Spider

Costo más elevado.

Mucho más complejo.

Permite hacer ataques de voltaje y electromagnéticos.

Define todo en tiempos absolutos, limitado a 4 ns.

Inyectores de Glitches: Spider

Inyectores de Glitches: Spider

- **offset**: Determina el comienzo del glitch.

Inyectores de Glitches: Spider

- **offset**: Determina el comienzo del glitch.
- **duration**: Por cuánto tiempo debe mantenerse el glitch.

Inyectores de Glitches: Spider

- **offset**: Determina el comienzo del glitch.
- **duration**: Por cuánto tiempo debe mantenerse el glitch.
- **voltage**: Indica el nivel de tensión solicitado durante el glitch. Se pueden programar valores entre $[-4, 4]$ V.

Inyectores de Glitches: Spider

Inyectores de Glitches: Spider

No se limita a un único pulso, sino que se pueden definir segmentos, con distintos voltajes y distintas duraciones:

Inyectores de Glitches: Spider

No se limita a un único pulso, sino que se pueden definir segmentos, con distintos voltajes y distintas duraciones:

$$V_{\text{glitch}}(t) = \begin{cases} V_1, & 0 \leq t < d_1, \\ V_2, & d_1 \leq t < d_1 + d_2, \\ \vdots & \\ V_m, & \sum_{i=1}^{m-1} d_i \leq t < \sum_{i=1}^{m} d_i. \end{cases}$$

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Metodología de Diseño

Metodología de Diseño

El modelado de Chaser se realizó con Enterprise Architect mediante diagramas UML.

Metodología de Diseño

El modelado de Chaser se realizó con Enterprise Architect mediante diagramas UML.

Se organiza en dos fases:

Metodología de Diseño

El modelado de Chaser se realizó con Enterprise Architect mediante diagramas UML.

Se organiza en dos fases:

- **Análisis**: qué debe hacer el sistema.

Metodología de Diseño

El modelado de Chaser se realizó con Enterprise Architect mediante diagramas UML.

Se organiza en dos fases:

- **Análisis**: qué debe hacer el sistema.
- **Diseño**: cómo se construye.

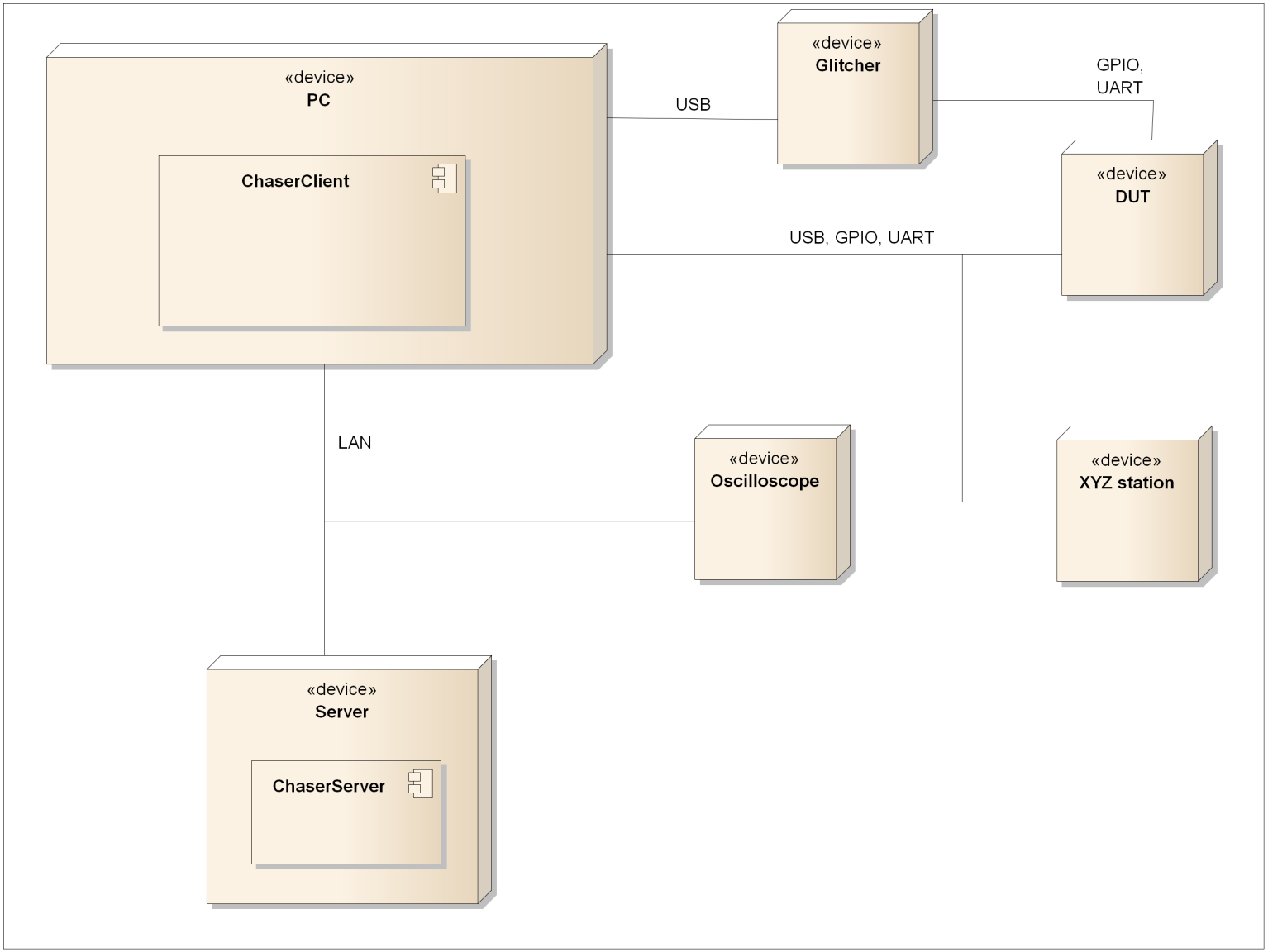
Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Fase de Análisis: Diagramas

- Modelo Arquitectónico.

Fase de Análisis: Diagramas



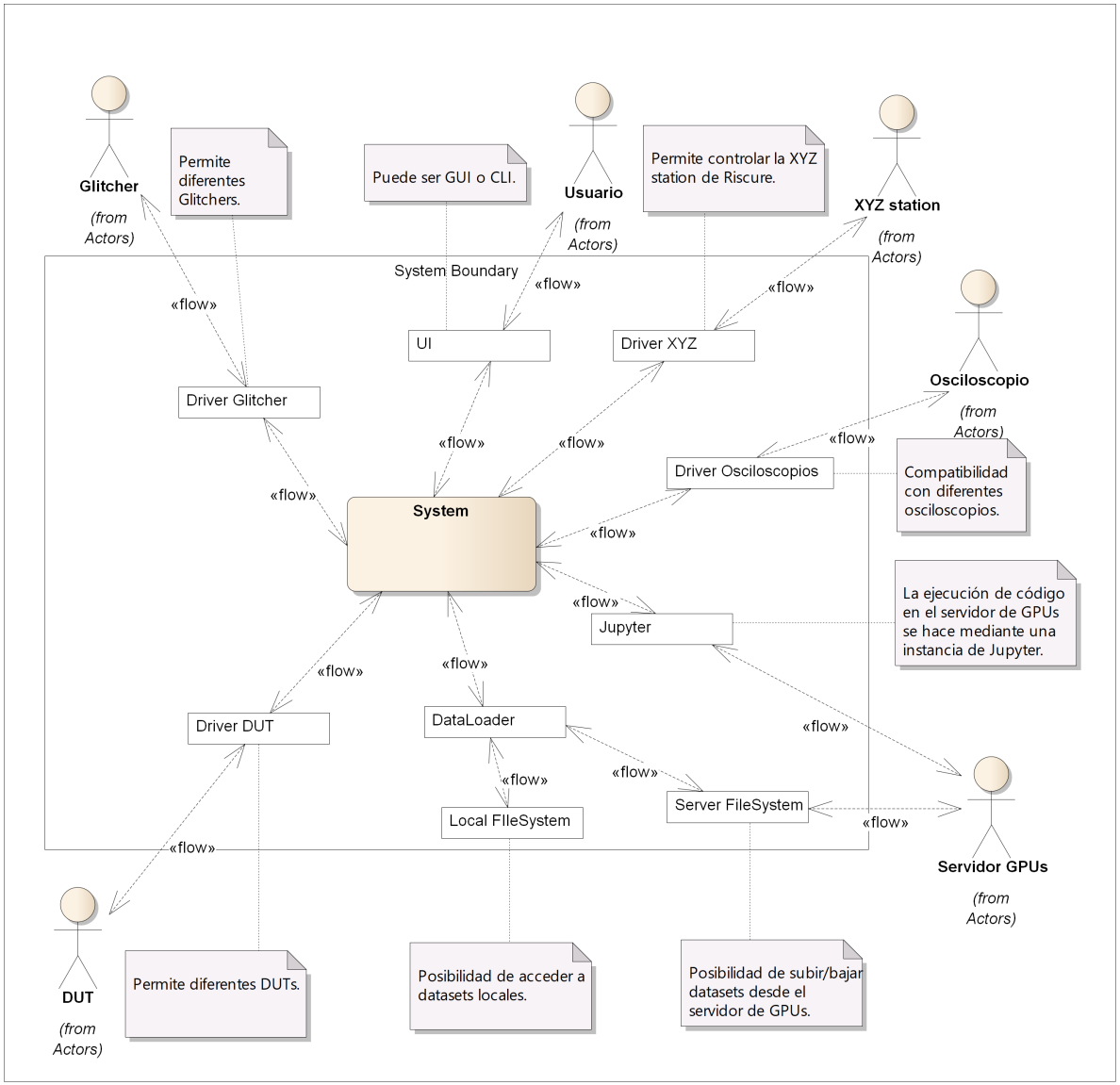
Fase de Análisis: Diagramas

- Modelo Arquitectónico.
- Requerimientos.

Fase de Análisis: Diagramas

- Modelo Arquitectónico.
- Requerimientos.
- Diagrama de contexto.

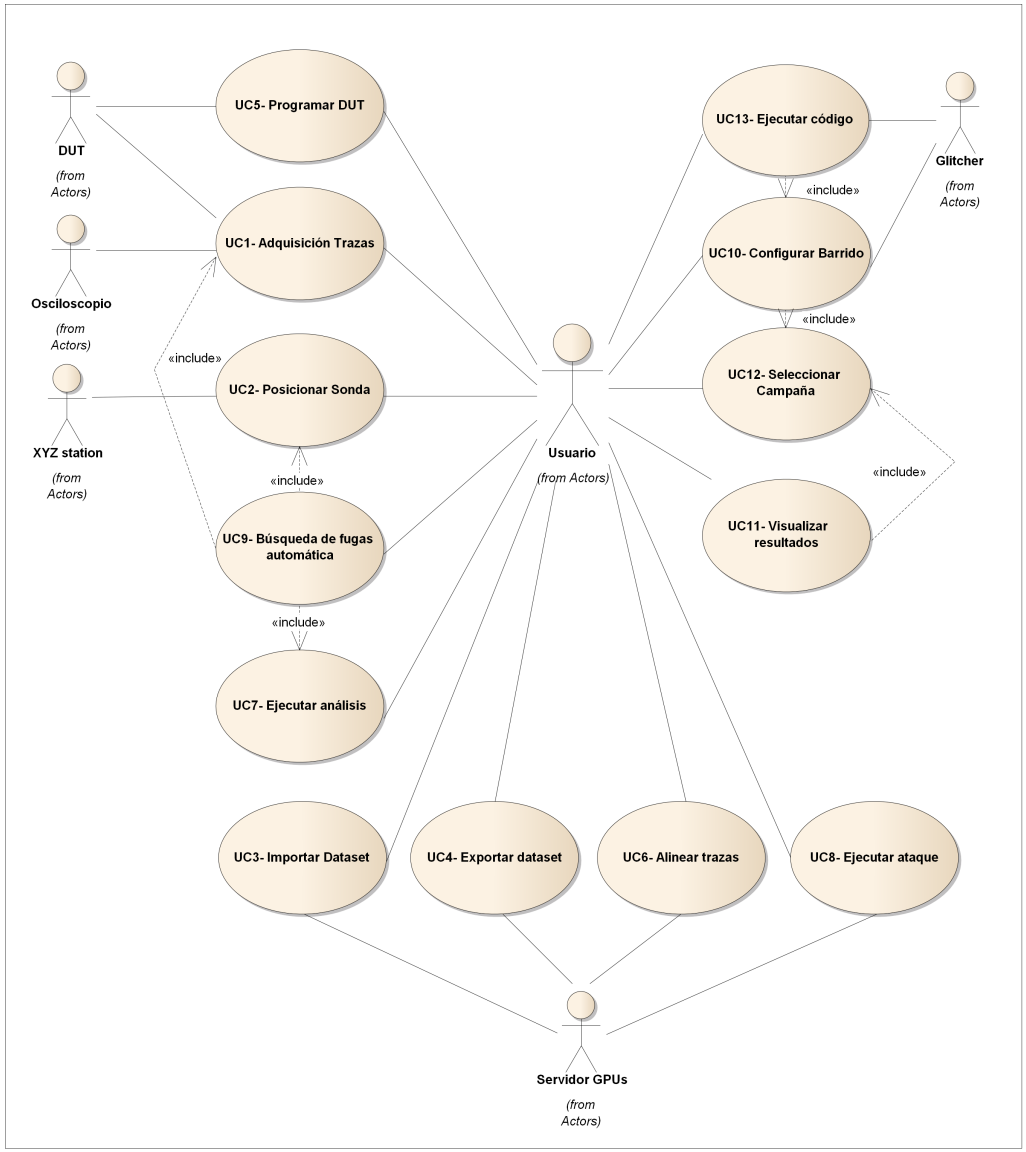
Fase de Análisis: Diagramas



Fase de Análisis: Diagramas

- Modelo Arquitectónico.
- Requerimientos.
- Diagrama de contexto.
- Casos de uso.

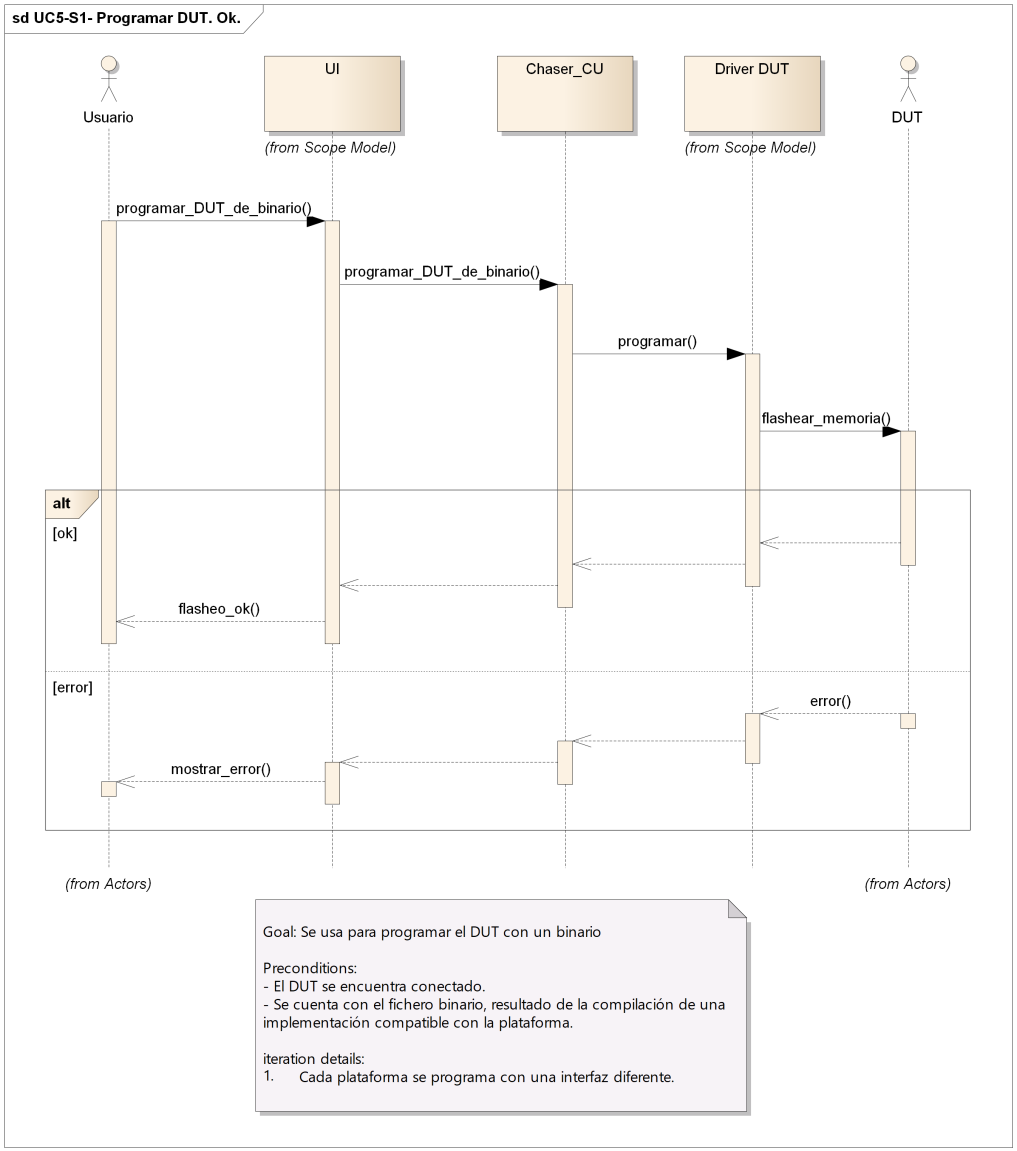
Fase de Análisis: Diagramas



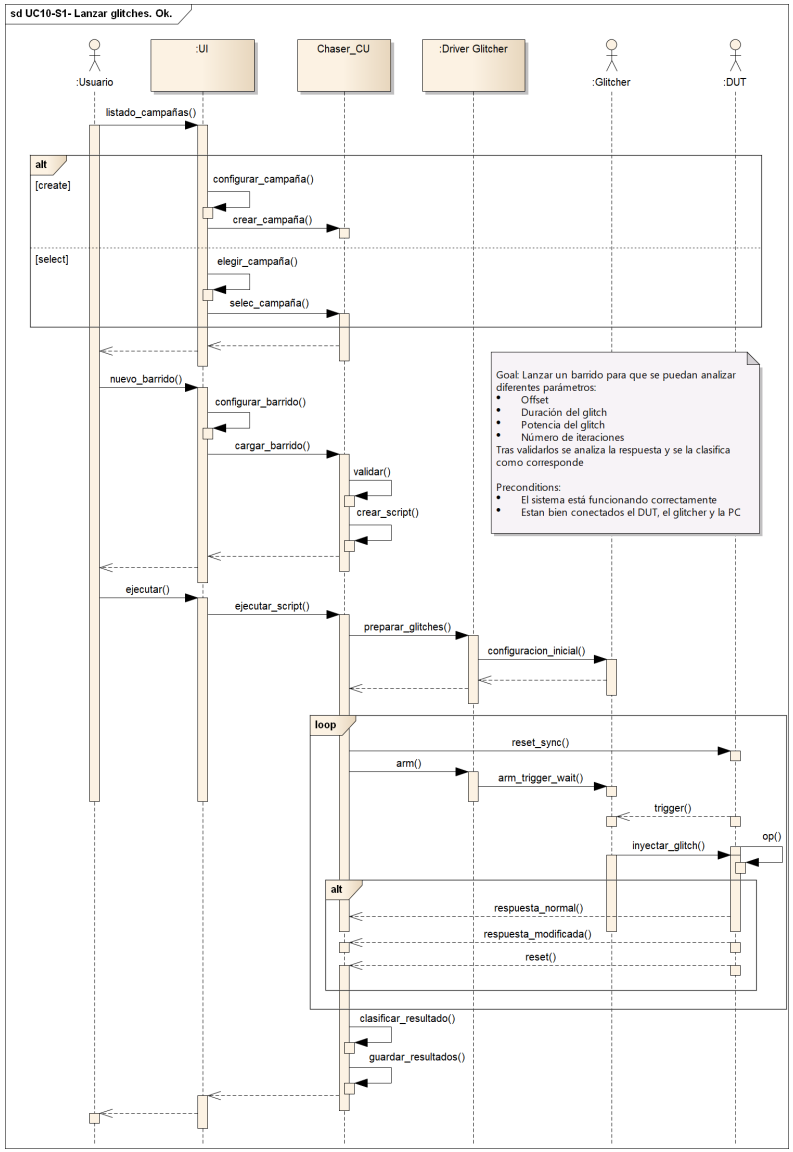
Fase de Análisis: Diagramas

- Modelo Arquitectónico.
- Requerimientos.
- Diagrama de contexto.
- Casos de uso.
- Modelos de interacción.

Fase de Análisis: Diagramas



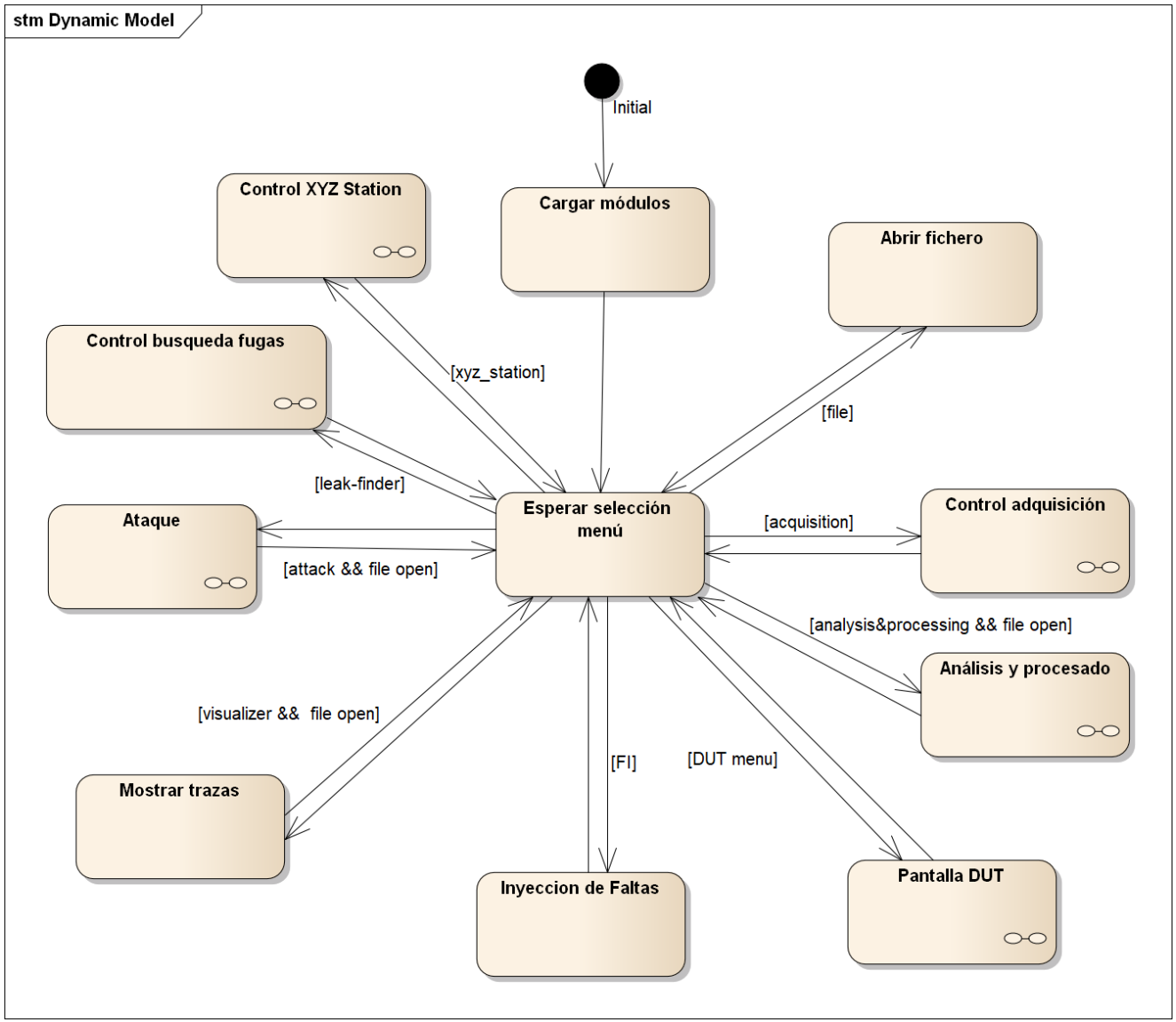
Fase de Análisis: Diagramas



Fase de Análisis: Diagramas

- Modelo Arquitectónico.
- Requerimientos.
- Diagrama de contexto.
- Casos de uso.
- Modelos de interacción.
- Modelos dinámicos.

Fase de Análisis: Diagramas



Fase de Análisis: Diagramas

- Modelo Arquitectónico.
- Requerimientos.
- Diagrama de contexto.
- Casos de uso.
- Modelos de interacción.
- Modelos dinámicos.

Fase de Análisis: Requerimientos

Fase de Análisis: Requerimientos

Funcionales:

Fase de Análisis: Requerimientos

Funcionales:

- Manejo de glitchers y generación de código.

Fase de Análisis: Requerimientos

Funcionales:

- Manejo de glitchers y generación de código.
- Preparación del Dispositivo a Testear (*Device Under Test*, DUT).

Fase de Análisis: Requerimientos

Funcionales:

- Manejo de glitchers y generación de código.
- Preparación del Dispositivo a Testear (*Device Under Test*, DUT).
- Creación de campañas, barridos y visualización.

Fase de Análisis: Requerimientos

Funcionales:

- Manejo de glitchers y generación de código.
- Preparación del Dispositivo a Testear (*Device Under Test*, DUT).
- Creación de campañas, barridos y visualización.
- Estructura modular e interfaz gráfica (individual y general).

Fase de Análisis: Requerimientos

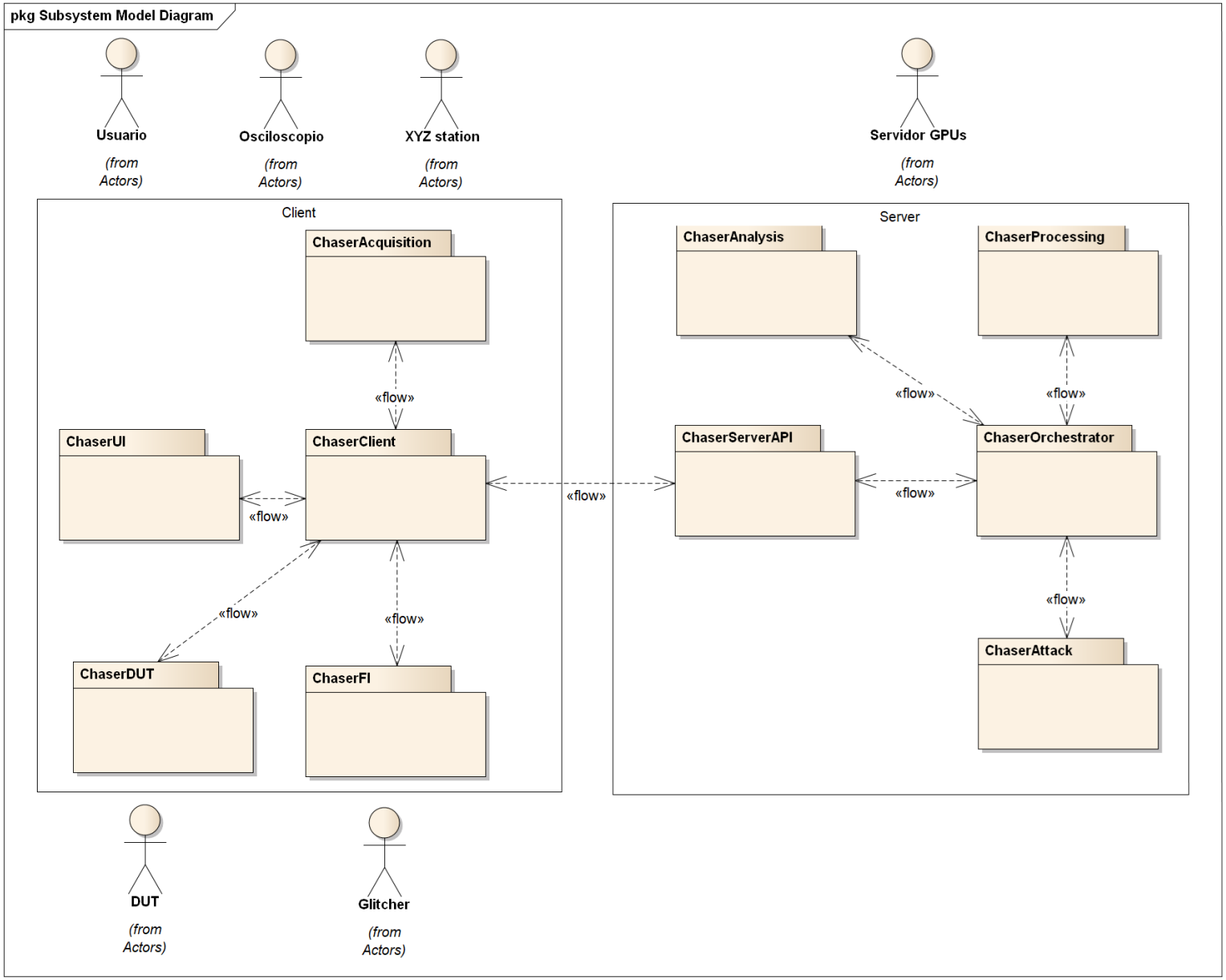
Funcionales:

- Manejo de glitchers y generación de código.
- Preparación del Dispositivo a Testear (*Device Under Test*, DUT).
- Creación de campañas, barridos y visualización.
- Estructura modular e interfaz gráfica (individual y general).
- Comunicación con servidor de GPUs e interoperabilidad.

Agenda

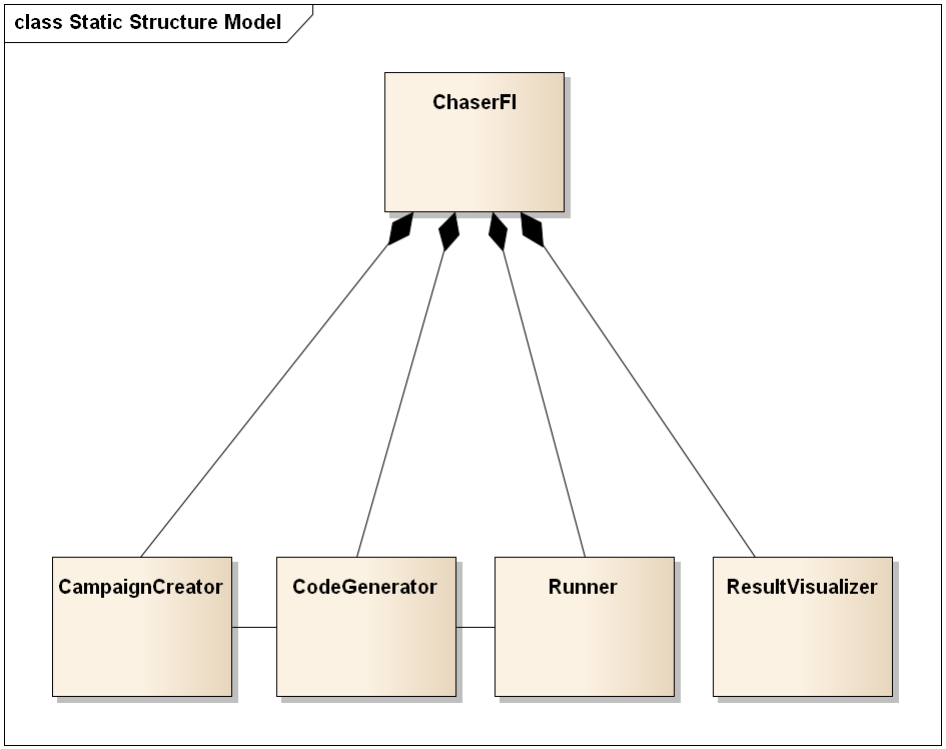
1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Diseño de Subsistemas



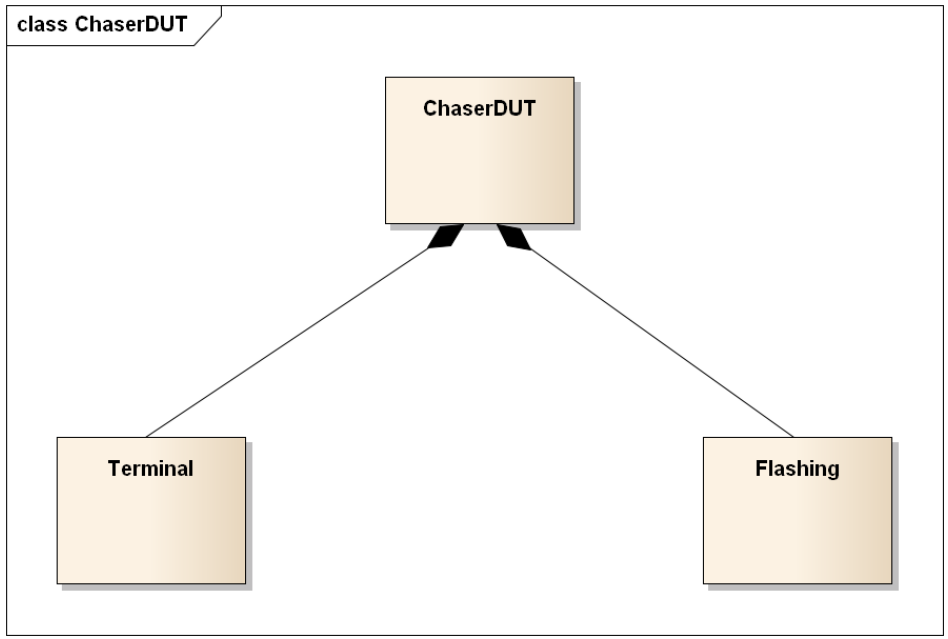
Modelo Estático

Módulo FI



Campañas, generación de código, ejecución y visualización.

Módulo DUT



Terminal interactiva y mecanismo de programación.

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Ítems de Cambio en la Generación de Código

Ítems de Cambio en la Generación de Código

Se analizaron los ítems de cambio y se aplicaron patrones.

Ítems de Cambio en la Generación de Código

Se analizaron los ítems de cambio y se aplicaron patrones.

- Hardware de inyección (Spider, ChipWhisperer).

Ítems de Cambio en la Generación de Código

Se analizaron los ítems de cambio y se aplicaron patrones.

- Hardware de inyección (Spider, ChipWhisperer).
- Tipos de glitches y parámetros de barrido.

Ítems de Cambio en la Generación de Código

Se analizaron los ítems de cambio y se aplicaron patrones.

- Hardware de inyección (Spider, ChipWhisperer).
- Tipos de glitches y parámetros de barrido.
- Comunicación (UART, GPIO) y sincronización (triggers).

Ítems de Cambio en la Generación de Código

Se analizaron los ítems de cambio y se aplicaron patrones.

- Hardware de inyección (Spider, ChipWhisperer).
- Tipos de glitches y parámetros de barrido.
- Comunicación (UART, GPIO) y sincronización (triggers).
- Clasificación de resultados (coincide, contiene, regex,. . .).

Ítems de Cambio en la Generación de Código

Se analizaron los ítems de cambio y se aplicaron patrones.

- Hardware de inyección (Spider, ChipWhisperer).
- Tipos de glitches y parámetros de barrido.
- Comunicación (UART, GPIO) y sincronización (triggers).
- Clasificación de resultados (coincide, contiene, regex,. . .).
- Persistencia (JSON, CSV) y política de recuperación de errores.

Ítems de Cambio en la Generación de Código

Se analizaron los ítems de cambio y se aplicaron patrones.

- Hardware de inyección (Spider, ChipWhisperer).
- Tipos de glitches y parámetros de barrido.
- Comunicación (UART, GPIO) y sincronización (triggers).
- Clasificación de resultados (coincide, contiene, regex,. . .).
- Persistencia (JSON, CSV) y política de recuperación de errores.
- Resumen de la ejecución en tiempo real.

Ítems de Cambio en la Generación de Código

Se analizaron los ítems de cambio y se aplicaron patrones.

- Hardware de inyección (Spider, ChipWhisperer).
- Tipos de glitches y parámetros de barrido.
- Comunicación (UART, GPIO) y sincronización (triggers).
- Clasificación de resultados (coincide, contiene, regex,. . .).
- Persistencia (JSON, CSV) y política de recuperación de errores.
- Resumen de la ejecución en tiempo real.

Cada una de estas partes es un bloque de código.

Patrones Aplicados

Patrones Aplicados

- **CodeBuilder** (*Builder*): conoce el esqueleto y ensambla los bloques, resolviendo el formato (tabulación de Python).

Patrones Aplicados

- **CodeBuilder** (*Builder*): conoce el esqueleto y ensambla los bloques, resolviendo el formato (tabulación de Python).
- **GlitcherFactory** (*Abstract Factory*): agrupa los bloques en familias coherentes por inyector.

Patrones Aplicados

- **CodeBuilder** (*Builder*): conoce el esqueleto y ensambla los bloques, resolviendo el formato (tabulación de Python).
- **GlitcherFactory** (*Abstract Factory*): agrupa los bloques en familias coherentes por inyector.
- **BlockGenerator** (*Strategy*): cada bloque expone `generate`.

Patrones Aplicados

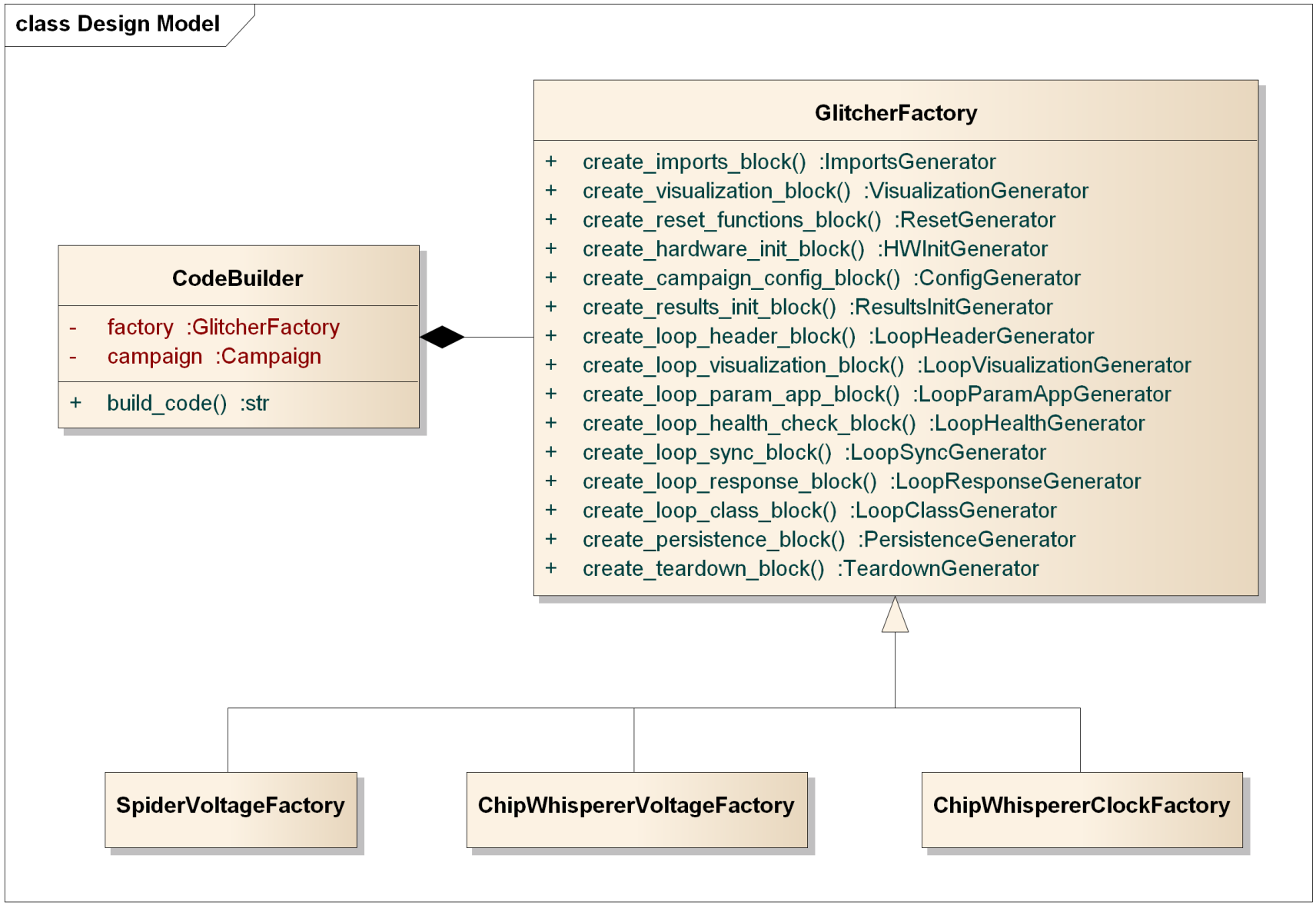
- **CodeBuilder** (*Builder*): conoce el esqueleto y ensambla los bloques, resolviendo el formato (tabulación de Python).
- **GlitcherFactory** (*Abstract Factory*): agrupa los bloques en familias coherentes por inyector.
- **BlockGenerator** (*Strategy*): cada bloque expone `generate`.
- **Command**: encapsula las acciones de la interfaz gráfica.

Patrones Aplicados

- **CodeBuilder** (*Builder*): conoce el esqueleto y ensambla los bloques, resolviendo el formato (tabulación de Python).
- **GlitcherFactory** (*Abstract Factory*): agrupa los bloques en familias coherentes por inyector.
- **BlockGenerator** (*Strategy*): cada bloque expone `generate`.
- **Command**: encapsula las acciones de la interfaz gráfica.

Añadir un inyector se reduce a implementar una nueva fábrica.

Generación de Código



Patrón Command: Ejemplo

```
1: procedure GenerateCodeCommand.execute
2:     factory ← create_factory(campaign)
3:     builder ← create_template(factory)
4:     return builder.build_code()
5: end procedure
6: procedure GUI.on_generate_code
7:     cmd ← GenerateCodeCommand(campaign)
8:     return cmd.execute()
9: end procedure
```

Patrón Command: Ejemplo

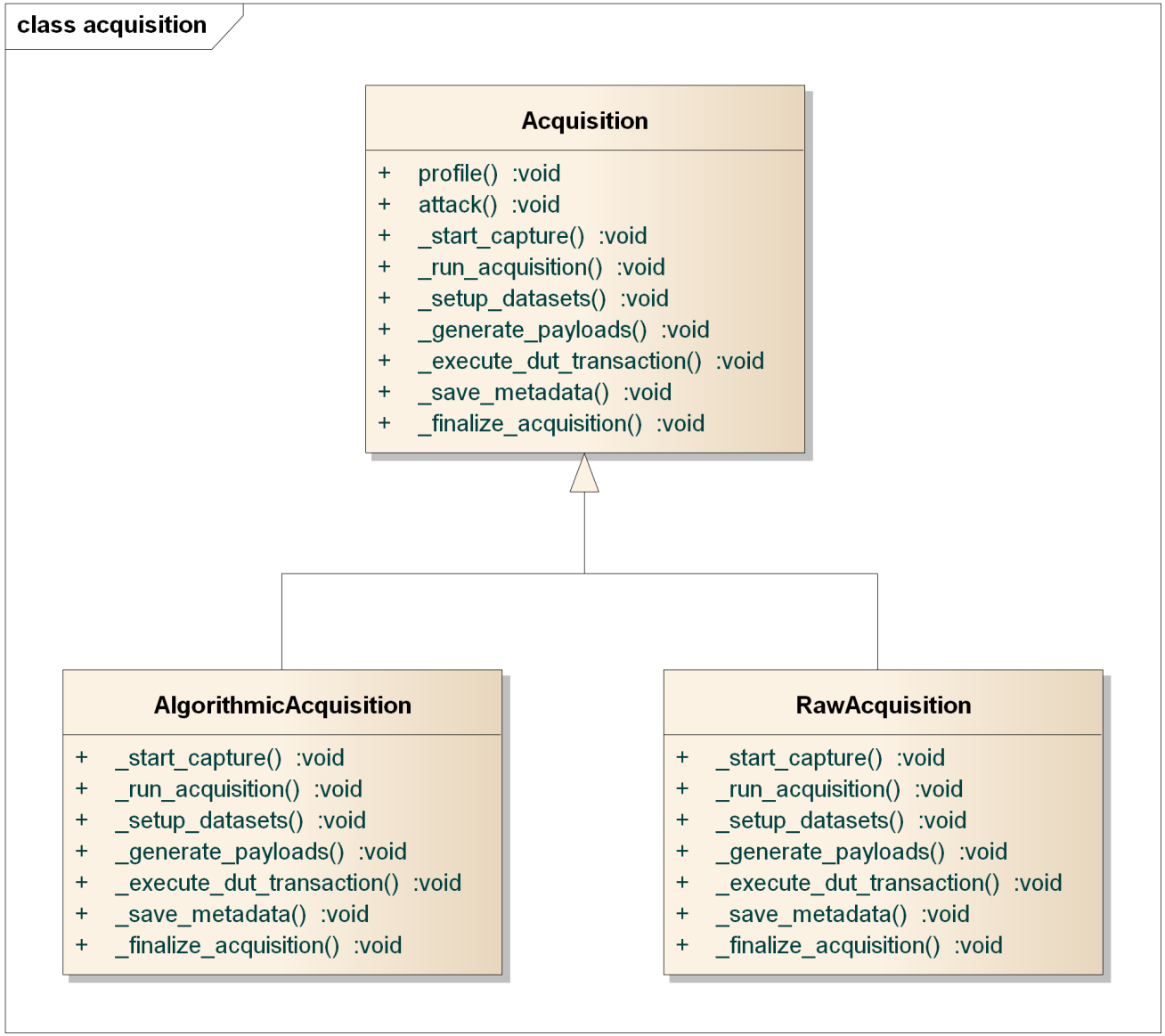
```
1: procedure GenerateCodeCommand.execute
2:     factory ← create_factory(campaign)
3:     builder ← create_template(factory)
4:     return builder.build_code()
5: end procedure
6: procedure GUI.on_generate_code
7:     cmd ← GenerateCodeCommand(campaign)
8:     return cmd.execute()
9: end procedure
```

Cualquier extensión de la interfaz no requiere modificar la lógica.

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Acquisition: Patrón Template



Acquisition: Patrón Template

Pattern	Acquisition
based on	Template
because	**Define la estructura de operaciones**: Existen dos procesos que comparten una secuencia de pasos casi idéntica. Las diferencias se delegan a las subclases.
where	AbstractClass **is** Acquisition TemplateMethod1() **is** profile() TemplateMethod2() **is** attack() ConcreteClass1 **is** AlgorithmicAcquisition ConcreteClass2 **is** RawAcquisition PrimitiveOperation1() **is** _start_capture() PrimitiveOperation2() **is** _run_acquisition() PrimitiveOperation3() **is** _setup_datasets() PrimitiveOperation4() **is** _generate_payloads() PrimitiveOperation5() **is** _execute_dut_transaction() PrimitiveOperation6() **is** _save_metadata() PrimitiveOperation7() **is** _finalize_acquisition()

TVLA: Patrón Facade

TVLA: Patrón Facade

Antes: solo primer orden.

TVLA: Patrón Facade

Antes: solo primer orden.

- Expone una interfaz simple como acceso único (`run`, `tvla_order_n`).

TVLA: Patrón Facade

Antes: solo primer orden.

- Expone una interfaz simple como acceso único (`run`, `tvla_order_n`).
- Oculta la complejidad del sistema (e.g lotes y rutinas aceleradas).

TVLA: Patrón Facade

Antes: solo primer orden.

- Expone una interfaz simple como acceso único (`run`, `tvla_order_n`).
- Oculta la complejidad del sistema (e.g lotes y rutinas aceleradas).
- Reduce el acoplamiento: las optimizaciones internas pueden cambiar sin afectar a los clientes.

TVLA: Patrón Facade

Antes: solo primer orden.

- Expone una interfaz simple como acceso único (`run`, `tvla_order_n`).
- Oculta la complejidad del sistema (e.g lotes y rutinas aceleradas).
- Reduce el acoplamiento: las optimizaciones internas pueden cambiar sin afectar a los clientes.
- Se añadió el cálculo de orden n.

Resumen del Diseño

Resumen del Diseño

- Modelado completo con más de 40 diagramas UML.

Resumen del Diseño

- Modelado completo con más de 40 diagramas UML.
- Patrones aplicados: Builder, Abstract Factory, Strategy, Command, Template, Facade, entre otros.

Resumen del Diseño

- Modelado completo con más de 40 diagramas UML.
- Patrones aplicados: Builder, Abstract Factory, Strategy, Command, Template, Facade, entre otros.
- Diseño guiado por ítems de cambio para maximizar la extensibilidad.

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Contexto

Contexto

La criptografía asimétrica, como RSA o DH, depende de la dificultad de resolver el logaritmo discreto.

Contexto

La criptografía asimétrica, como RSA o DH, depende de la dificultad de resolver el logaritmo discreto.

Una computadora cuántica podría resolver estos problemas.

Contexto

La criptografía asimétrica, como RSA o DH, depende de la dificultad de resolver el logaritmo discreto.

Una computadora cuántica podría resolver estos problemas.

El NIST inició en 2016 un proceso de estandarización de algoritmos criptográficos postcuánticos.

Contexto

Contexto

CRYSTALS-Kyber fue seleccionado como base.

Contexto

CRYSTALS-Kyber fue seleccionado como base.

Posteriormente fue estandarizado como ML-KEM en FIPS 203.

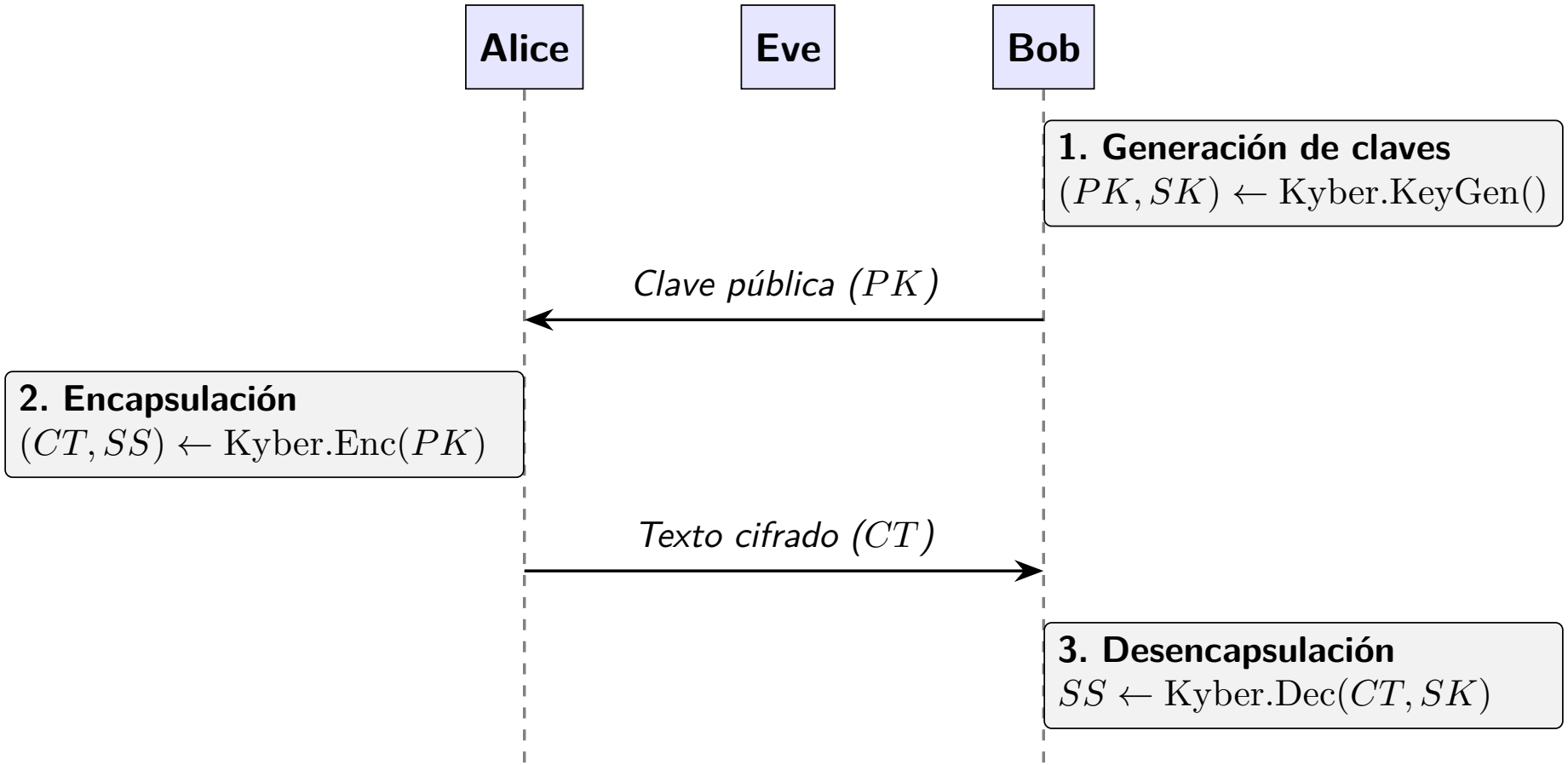
Contexto

CRYSTALS-Kyber fue seleccionado como base.

Posteriormente fue estandarizado como ML-KEM en FIPS 203.

En los últimos años fue objeto de estudio.

Fases de Kyber



$PK \leftarrow$ Public Key (clave pública), $SK \leftarrow$ Secret Key (clave secreta),
$CT \leftarrow$ Ciphertext (texto cifrado), $SS \leftarrow$ Shared Secret (secreto compartido).

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Motivación

Motivación

Ravi et al. (2022), Fiddling the Twiddle Constants.

Motivación

Ravi et al. (2022), Fiddling the Twiddle Constants.

Usan EMFI: bit-sets y bit-resets con $100\,\%$ de reproducibilidad.

Motivación

Ravi et al. (2022), Fiddling the Twiddle Constants.

Usan EMFI: bit-sets y bit-resets con $100\,\%$ de reproducibilidad.

Usamos la idea, pero con ataques de voltaje y reloj.

Carga de constantes en la NTT

Carga de constantes en la NTT

Las operaciones más costosas en Kyber son las multiplicaciones de polinomios, que implementadas trivialmente son $\mathcal{O}(n^2)$.

Carga de constantes en la NTT

Las operaciones más costosas en Kyber son las multiplicaciones de polinomios, que implementadas trivialmente son $\mathcal{O}(n^2)$.

Se aceleran con la NTT: la transformación es $\mathcal{O}(n \log n)$ y la multiplicación es $\mathcal{O}(n)$.

Carga de constantes en la NTT

Las operaciones más costosas en Kyber son las multiplicaciones de polinomios, que implementadas trivialmente son $\mathcal{O}(n^2)$.

Se aceleran con la NTT: la transformación es $\mathcal{O}(n \log n)$ y la multiplicación es $\mathcal{O}(n)$.

Utiliza un arreglo precomputado de constantes ζ (zetas, *twiddle factors*).

Carga de constantes en la NTT

Las operaciones más costosas en Kyber son las multiplicaciones de polinomios, que implementadas trivialmente son $\mathcal{O}(n^2)$.

Se aceleran con la NTT: la transformación es $\mathcal{O}(n \log n)$ y la multiplicación es $\mathcal{O}(n)$.

Utiliza un arreglo precomputado de constantes ζ (zetas, *twiddle factors*).

Una alteración en las constantes corrompe la generación de la clave.

Carga de constantes en la NTT

Carga de constantes en la NTT

El secreto consta, entre otras cosas, de:

Carga de constantes en la NTT

El secreto consta, entre otras cosas, de:

- 3 polinomios.

Carga de constantes en la NTT

El secreto consta, entre otras cosas, de:

- 3 polinomios.
- De 256 coeficientes enteros cada uno.

Carga de constantes en la NTT

El secreto consta, entre otras cosas, de:

- 3 polinomios.
- De 256 coeficientes enteros cada uno.
- Muestreados en el intervalo $[-2, 2]$.

Carga de constantes en la NTT

Carga de constantes en la NTT

Se implementa con una serie de operaciones llamadas mariposas de Cooley-Tukey:

Carga de constantes en la NTT

Se implementa con una serie de operaciones llamadas mariposas de Cooley-Tukey:

$$c = a + \zeta b \pmod{q},$$
$$d = a - \zeta b \pmod{q},$$

Carga de constantes en la NTT

Se implementa con una serie de operaciones llamadas mariposas de Cooley-Tukey:

$$c = a + \zeta b \pmod{q},$$
$$d = a - \zeta b \pmod{q},$$

Si todas valen cero:

Carga de constantes en la NTT

Se implementa con una serie de operaciones llamadas mariposas de Cooley-Tukey:

$$c = a + \zeta b \pmod{q},$$
$$d = a - \zeta b \pmod{q},$$

Si todas valen cero:

$$c = a + 0\ b = a \pmod{q},$$
$$d = a - 0\ b = a \pmod{q}.$$

Carga de constantes en la NTT

La mitad de los coeficientes se sobrescriben con copias de la otra mitad en cada iteración.

Carga de constantes en la NTT

La mitad de los coeficientes se sobrescriben con copias de la otra mitad en cada iteración.

Etapa	len	Coeficientes distintos que sobreviven
0	256	$f_0, \ldots, f_{255}$

Carga de constantes en la NTT

La mitad de los coeficientes se sobrescriben con copias de la otra mitad en cada iteración.

Etapa	len	Coeficientes distintos que sobreviven
0	256	$f_0, \ldots, f_{255}$
1	128	$f_0, \ldots, f_{127}$

Carga de constantes en la NTT

La mitad de los coeficientes se sobrescriben con copias de la otra mitad en cada iteración.

Etapa	len	Coeficientes distintos que sobreviven
0	256	$f_0, \ldots, f_{255}$
1	128	$f_0, \ldots, f_{127}$
2	64	$f_0, \ldots, f_{63}$
3	32	$f_0, \ldots, f_{31}$
4	16	$f_0, \ldots, f_{15}$
5	8	$f_0, \ldots, f_7$
6	4	$f_0, \ldots, f_3$

Carga de constantes en la NTT

La mitad de los coeficientes se sobrescriben con copias de la otra mitad en cada iteración.

Etapa	len	Coeficientes distintos que sobreviven
0	256	$f_0, \ldots, f_{255}$
1	128	$f_0, \ldots, f_{127}$
2	64	$f_0, \ldots, f_{63}$
3	32	$f_0, \ldots, f_{31}$
4	16	$f_0, \ldots, f_{15}$
5	8	$f_0, \ldots, f_7$
6	4	$f_0, \ldots, f_3$
7	2	$f_0,\ f_1$

Carga de constantes en la NTT

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

$$\mathrm{NTT}_0(f) = (f_0,\ f_1\,,\ f_0\,,\ f_1\,,\ \ldots,\ f_0\,,\ f_1)\,,$$

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

$$\mathrm{NTT}_0(f) = (f_0,\ f_1\,,\ f_0\,,\ f_1\,,\ \ldots,\ f_0\,,\ f_1)\,,$$
$$\mathrm{NTT}_1(f) = (f'_0,\ f'_1\,,\ f'_0\,,\ f'_1\,,\ \ldots,\ f'_0\,,\ f'_1)\,,$$

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

$$\begin{aligned}
\mathrm{NTT}_0(f) &= (f_0,\ f_1\,,\ f_0\,,\ f_1\,,\ \ldots,\ f_0\,,\ f_1)\,,\\
\mathrm{NTT}_1(f) &= (f'_0,\ f'_1\,,\ f'_0\,,\ f'_1\,,\ \ldots,\ f'_0\,,\ f'_1)\,,\\
\mathrm{NTT}_2(f) &= (f''_0,\ f''_1,\ f''_0,\ f''_1,\ \ldots,\ f''_0,\ f''_1),
\end{aligned}$$

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

$$\begin{aligned}
\mathrm{NTT}_0(f) &= (f_0,\ f_1\,,\ f_0\,,\ f_1\,,\ \ldots,\ f_0\,,\ f_1)\,,\\
\mathrm{NTT}_1(f) &= (f'_0,\ f'_1\,,\ f'_0\,,\ f'_1\,,\ \ldots,\ f'_0\,,\ f'_1)\,,\\
\mathrm{NTT}_2(f) &= (f''_0,\ f''_1,\ f''_0,\ f''_1,\ \ldots,\ f''_0,\ f''_1),
\end{aligned}$$

Por lo tanto, el número total de secretos posibles es

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

$$\begin{aligned}
\mathrm{NTT}_0(f) &= (f_0,\ f_1\,,\ f_0\,,\ f_1\,,\ \ldots,\ f_0\,,\ f_1)\,,\\
\mathrm{NTT}_1(f) &= (f'_0,\ f'_1\,,\ f'_0\,,\ f'_1\,,\ \ldots,\ f'_0\,,\ f'_1)\,,\\
\mathrm{NTT}_2(f) &= (f''_0,\ f''_1,\ f''_0,\ f''_1,\ \ldots,\ f''_0,\ f''_1),
\end{aligned}$$

Por lo tanto, el número total de secretos posibles es

$$5$$

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

$$\begin{aligned}
\mathrm{NTT}_0(f) &= (f_0,\ f_1\,,\ f_0\,,\ f_1\,,\ \ldots,\ f_0\,,\ f_1)\,,\\
\mathrm{NTT}_1(f) &= (f'_0,\ f'_1\,,\ f'_0\,,\ f'_1\,,\ \ldots,\ f'_0\,,\ f'_1)\,,\\
\mathrm{NTT}_2(f) &= (f''_0,\ f''_1,\ f''_0,\ f''_1,\ \ldots,\ f''_0,\ f''_1),
\end{aligned}$$

Por lo tanto, el número total de secretos posibles es

$$5^2$$

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

$$\begin{aligned}
\mathrm{NTT}_0(f) &= (f_0,\ f_1\,,\ f_0\,,\ f_1\,,\ \ldots,\ f_0\,,\ f_1)\,,\\
\mathrm{NTT}_1(f) &= (f'_0,\ f'_1\,,\ f'_0\,,\ f'_1\,,\ \ldots,\ f'_0\,,\ f'_1)\,,\\
\mathrm{NTT}_2(f) &= (f''_0,\ f''_1,\ f''_0,\ f''_1,\ \ldots,\ f''_0,\ f''_1),
\end{aligned}$$

Por lo tanto, el número total de secretos posibles es

$$5^{2\cdot 3}$$

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

$$\begin{aligned}
\mathrm{NTT}_0(f) &= (f_0,\ f_1\,,\ f_0\,,\ f_1\,,\ \ldots,\ f_0\,,\ f_1)\,,\\
\mathrm{NTT}_1(f) &= (f'_0,\ f'_1\,,\ f'_0\,,\ f'_1\,,\ \ldots,\ f'_0\,,\ f'_1)\,,\\
\mathrm{NTT}_2(f) &= (f''_0,\ f''_1,\ f''_0,\ f''_1,\ \ldots,\ f''_0,\ f''_1),
\end{aligned}$$

Por lo tanto, el número total de secretos posibles es

$$5^{2\cdot 3} = 5^6 =$$

Carga de constantes en la NTT

Cada polinomio pasa a estar definido por dos coeficientes

$$\begin{aligned}
\mathrm{NTT}_0(f) &= (f_0,\ f_1\,,\ f_0\,,\ f_1\,,\ \ldots,\ f_0\,,\ f_1)\,,\\
\mathrm{NTT}_1(f) &= (f'_0,\ f'_1\,,\ f'_0\,,\ f'_1\,,\ \ldots,\ f'_0\,,\ f'_1)\,,\\
\mathrm{NTT}_2(f) &= (f''_0,\ f''_1,\ f''_0,\ f''_1,\ \ldots,\ f''_0,\ f''_1),
\end{aligned}$$

Por lo tanto, el número total de secretos posibles es

$$5^{2\cdot 3} = 5^6 = 15\,625.$$

Pseudocódigo de la generación de la clave

Pseudocódigo de la generación de la clave

1: **procedure** KeyGen
2: $seed_A \in \mathcal{B} \leftarrow \text{Sample}_U()$ ▷ Generamos semilla para A
3: $seed_B \in \mathcal{B} \leftarrow \text{Sample}_U()$ ▷ Generamos semilla para s y e
4: $\hat{\mathbf{A}} = \text{NTT}(\mathbf{A}) \in R_q^{k \times k} \leftarrow \text{Expand}(seed_A)$ ▷ Creamos A y en NTT
5: $\mathbf{s} \in R_q^k \leftarrow \text{Sample}_B(seed_B, coins_s)$ ▷ Se muestrea el secreto
6: $\mathbf{e} \in R_q^k \leftarrow \text{Sample}_B(seed_B, coins_e)$ ▷ Se muestrea el error
7: $\hat{\mathbf{s}} \in R_q^k \leftarrow \text{NTT}(\mathbf{s})$ ▷ Convierte el secreto al dominio NTT
8: $\hat{\mathbf{e}} \in R_q^k \leftarrow \text{NTT}(\mathbf{e})$ ▷ Convierte el error al dominio NTT
9: $\hat{\mathbf{t}} = \hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \hat{\mathbf{e}}$ ▷ Se calcula la clave pública en NTT
10: **return** $(pk = (seed_A, \hat{\mathbf{t}}),\ sk = (\hat{\mathbf{s}}))$ ▷ Se devuelven las claves
11: **end procedure**

Código en C de la generación de la clave

Código en C de la generación de la clave

```
void indcpa_keypair_derand(unsigned char *pk, unsigned char *sk,
                           const unsigned char *coins) {
    [SNIP]
    trigger_high();
    polyvec_ntt(&skpv); // Instrucción objetivo
    trigger_low();
    [SNIP]
}

void polyvec_ntt(polyvec *r) {
    for(int i=0; i<KYBER_K; i++) // #define KYBER_K 3
        poly_ntt(&r->vec[i]);
}

void poly_ntt(poly *r) {
    ntt(r->coeffs);
}

const int32_t zetas_asm[128] = { [SNIP] }

void ntt(int16_t *poly) {
    ntt_fast(poly, zetas_asm); // Usa el array zetas_asm!
}

extern void ntt_fast(int16_t *, const int32_t *);
```

Código en C de la generación de la clave

Usamos la implementación de `pqm4`.

Código en C de la generación de la clave

Usamos la implementación de `pqm4`. Todo tiene mucha importancia:

Código en C de la generación de la clave

Usamos la implementación de `pqm4`. Todo tiene mucha importancia:

- Versión del compilador de GCC.

Código en C de la generación de la clave

Usamos la implementación de `pqm4`. Todo tiene mucha importancia:

- Versión del compilador de GCC.
- Banderas del compilador:

Código en C de la generación de la clave

Usamos la implementación de `pqm4`. Todo tiene mucha importancia:

- Versión del compilador de GCC.
- Banderas del compilador:
  - `-O3`: Optimizar agresivamente.

Código en C de la generación de la clave

Usamos la implementación de `pqm4`. Todo tiene mucha importancia:

- Versión del compilador de GCC.
- Banderas del compilador:
  - `-O3`: Optimizar agresivamente.
  - `-lto`: Optimizar todo como un único archivo.

Código en ensamblador de la generación de la clave

Código en ensamblador de la generación de la clave

```
8005000:  4d70        ldr r5, [pc, #448]  @ (80051c4 <main+0x768>)
[SNIP]
8005050:  4629        mov r1, r5 @ Segundo parámetro: puntero zetas_asm
8005052:  4630        mov r0, r6 @ Primer parámetro: puntero poly
8005054:  f7fd ff7a   bl  8002f4c <ntt_fast>
[SNIP]
8005088:  42b3        cmp r3, r6
800508a:  d1e1        bne.n   8005050 <main+0x5f4> @ Siguiente iteración
[SNIP]
80051c4:  080074f8    .word   0x080074f8
```

Código en ensamblador de la generación de la clave

```
8005000:  4d70        ldr r5, [pc, #448]  @ (80051c4 <main+0x768>)
[SNIP]
8005050:  4629        mov r1, r5 @ Segundo parámetro: puntero zetas_asm
8005052:  4630        mov r0, r6 @ Primer parámetro: puntero poly
8005054:  f7fd ff7a   bl  8002f4c <ntt_fast>
[SNIP]
8005088:  42b3        cmp r3, r6
800508a:  d1e1        bne.n   8005050 <main+0x5f4> @ Siguiente iteración
[SNIP]
80051c4:  080074f8    .word   0x080074f8
```

Necesitamos que `r5` apunte a la memoria RAM.

Código en ensamblador de la generación de la clave

```
8005000:  4d70        ldr r5, [pc, #448]  @ (80051c4 <main+0x768>)
[SNIP]
8005050:  4629        mov r1, r5 @ Segundo parámetro: puntero zetas_asm
8005052:  4630        mov r0, r6 @ Primer parámetro: puntero poly
8005054:  f7fd ff7a   bl  8002f4c <ntt_fast>
[SNIP]
8005088:  42b3        cmp r3, r6
800508a:  d1e1        bne.n   8005050 <main+0x5f4> @ Siguiente iteración
[SNIP]
80051c4:  080074f8    .word   0x080074f8
```

Necesitamos que `r5` apunte a la memoria RAM.
En estas placas, la memoria RAM comienza con `0x2`.

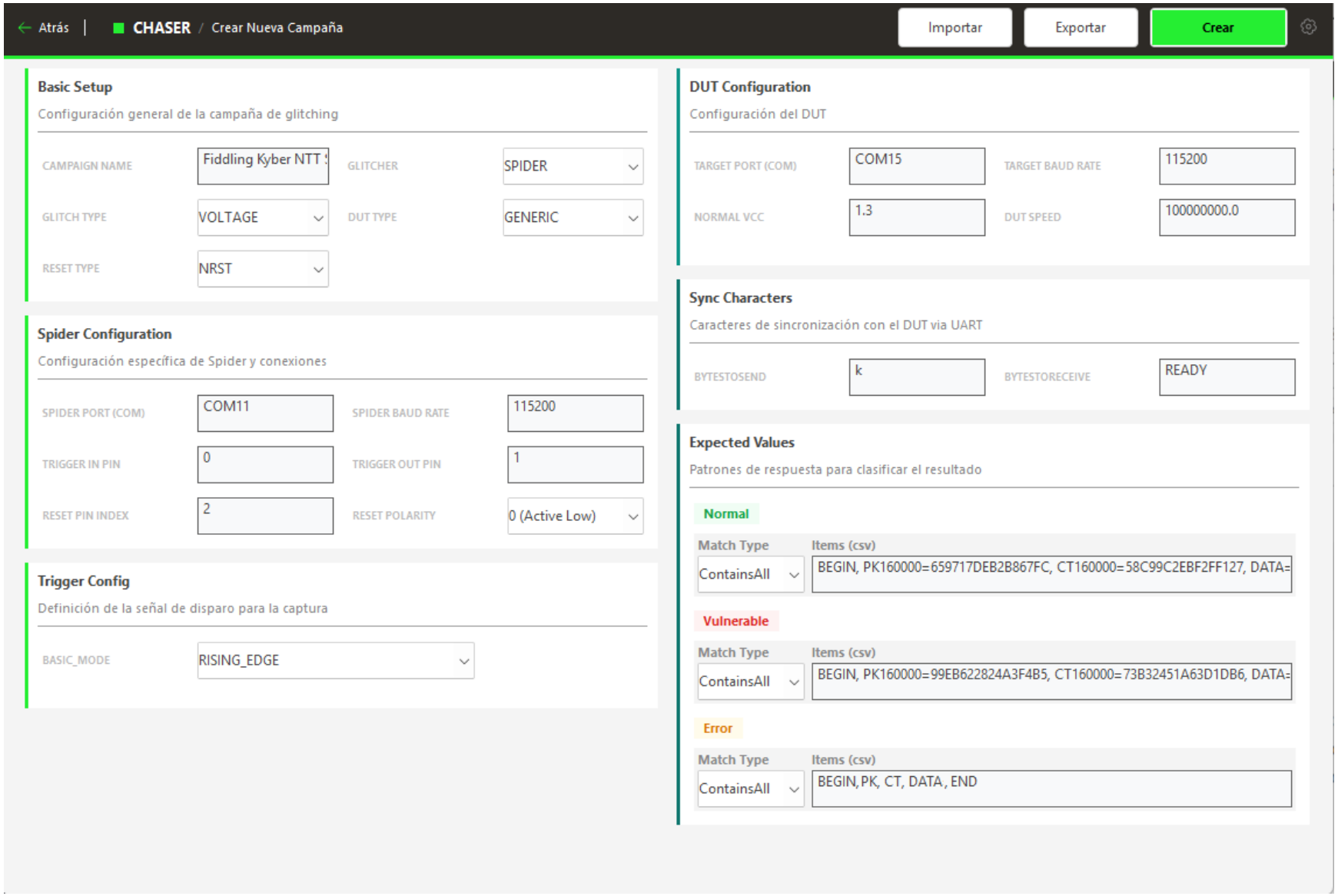
Código en ensamblador de la generación de la clave

```
8005000:  4d70        ldr r5, [pc, #448]  @ (80051c4 <main+0x768>)
[SNIP]
8005050:  4629        mov r1, r5 @ Segundo parámetro: puntero zetas_asm
8005052:  4630        mov r0, r6 @ Primer parámetro: puntero poly
8005054:  f7fd ff7a   bl  8002f4c <ntt_fast>
[SNIP]
8005088:  42b3        cmp r3, r6
800508a:  d1e1        bne.n   8005050 <main+0x5f4> @ Siguiente iteración
[SNIP]
80051c4:  080074f8    .word   0x080074f8
```

Necesitamos que `r5` apunte a la memoria RAM.
En estas placas, la memoria RAM comienza con `0x2`.
Al encenderse, un gran bloque se inicializa en cero.

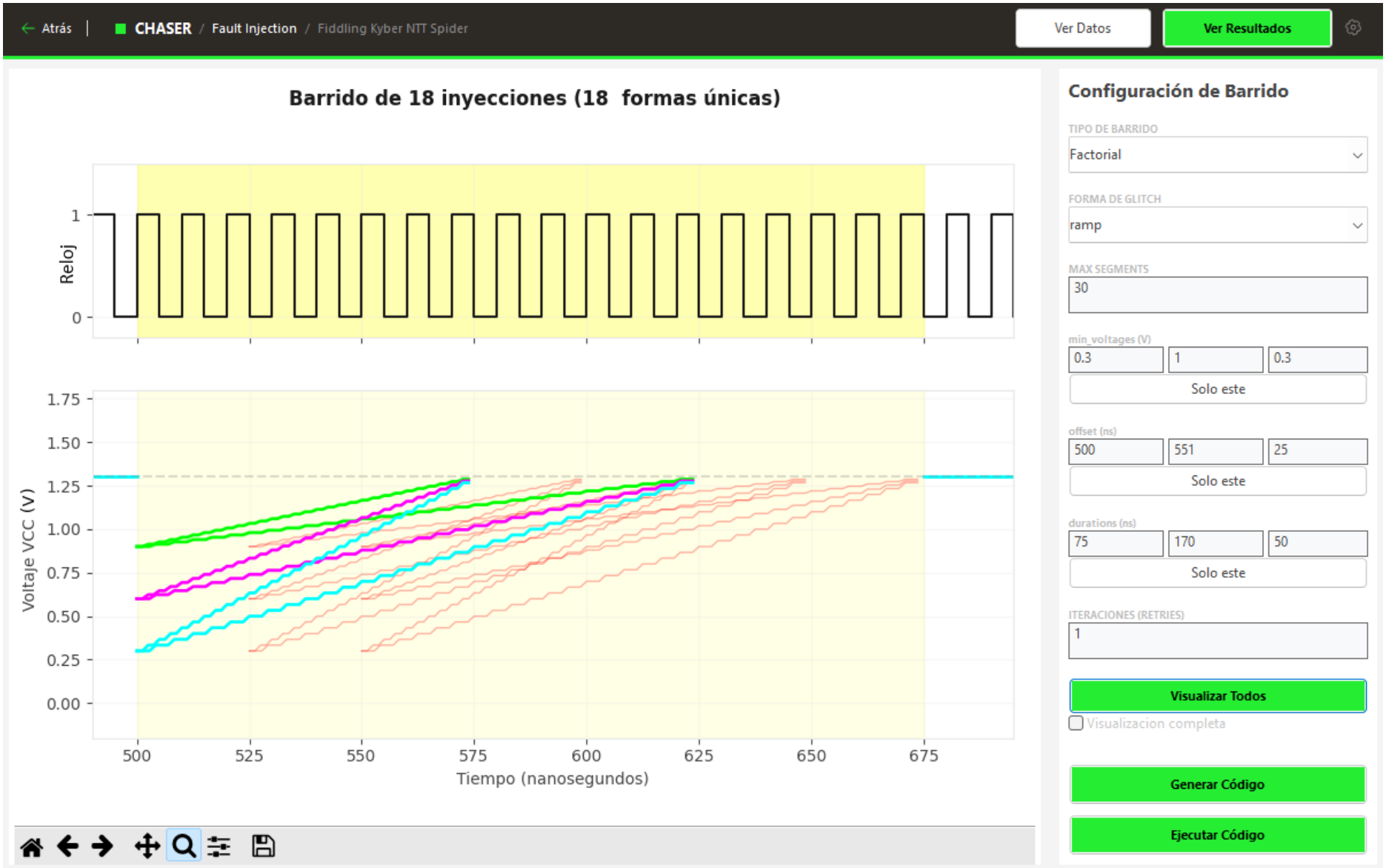
Campaña con Spider (Voltaje): Campaña

Campaña con Spider (Voltaje): Campaña



Campaña con Spider (Voltaje): Barrido de ejemplo

Campaña con Spider (Voltaje): Barrido de ejemplo



Campaña con Spider (Voltaje): Barrido de ejemplo

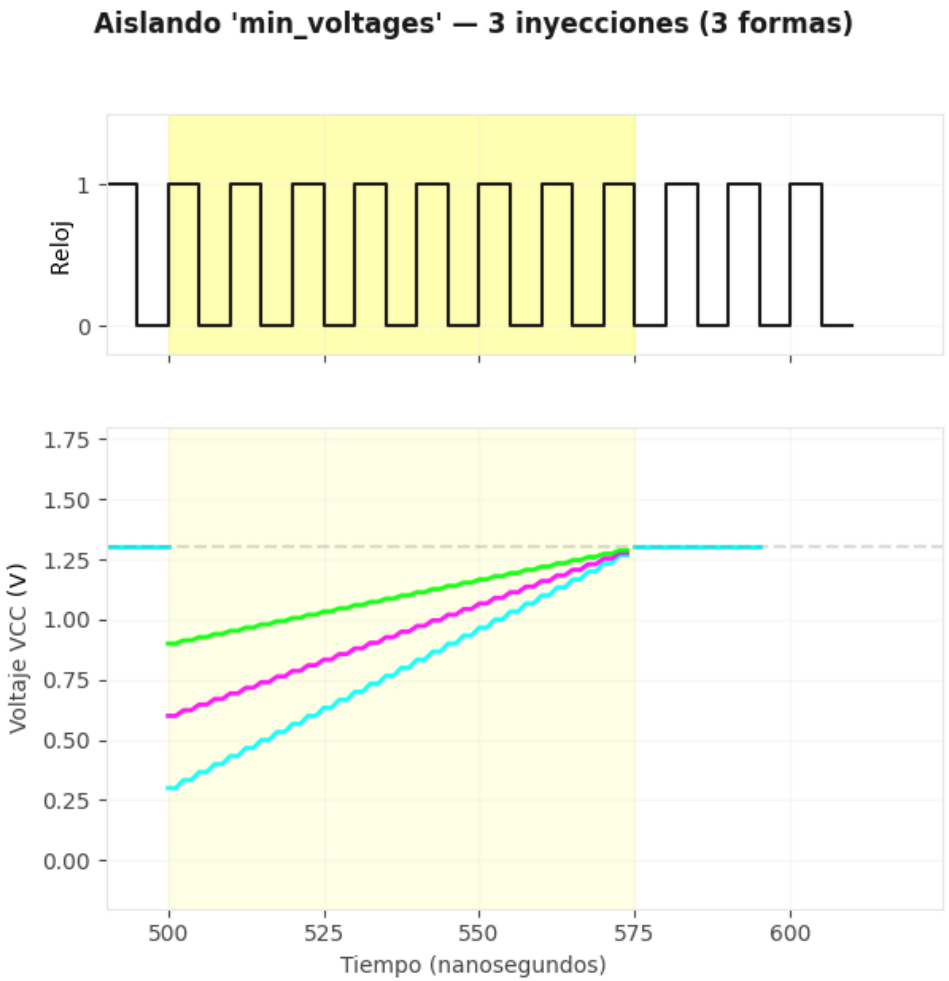


Figura: Aislando `voltage` en $[0.3, 1, 0.3]$ V

Campaña con Spider (Voltaje): Barrido de ejemplo

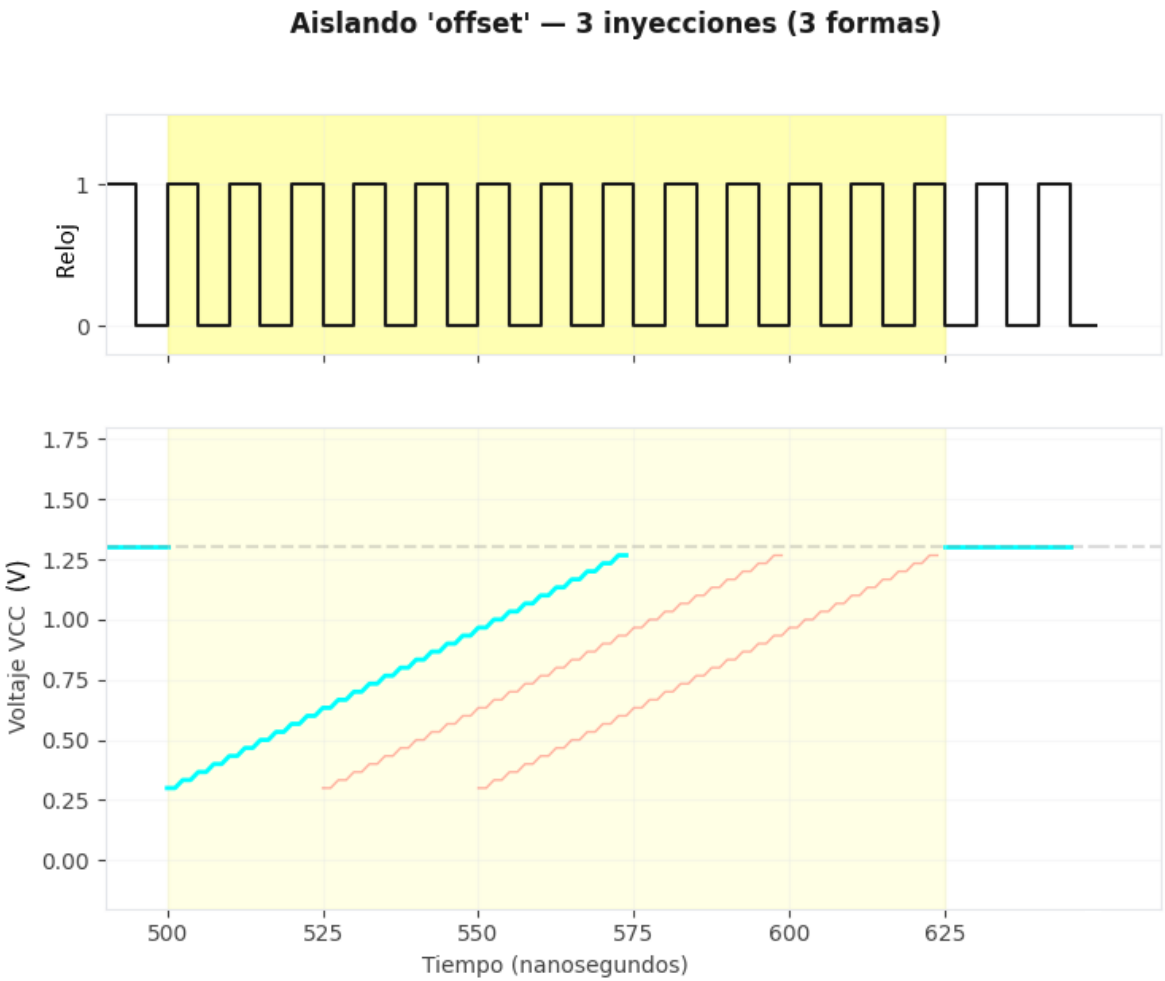


Figura: Aislando `offset` en $[500, 551, 25]$ ns

Campaña con Spider (Voltaje): Barrido de ejemplo

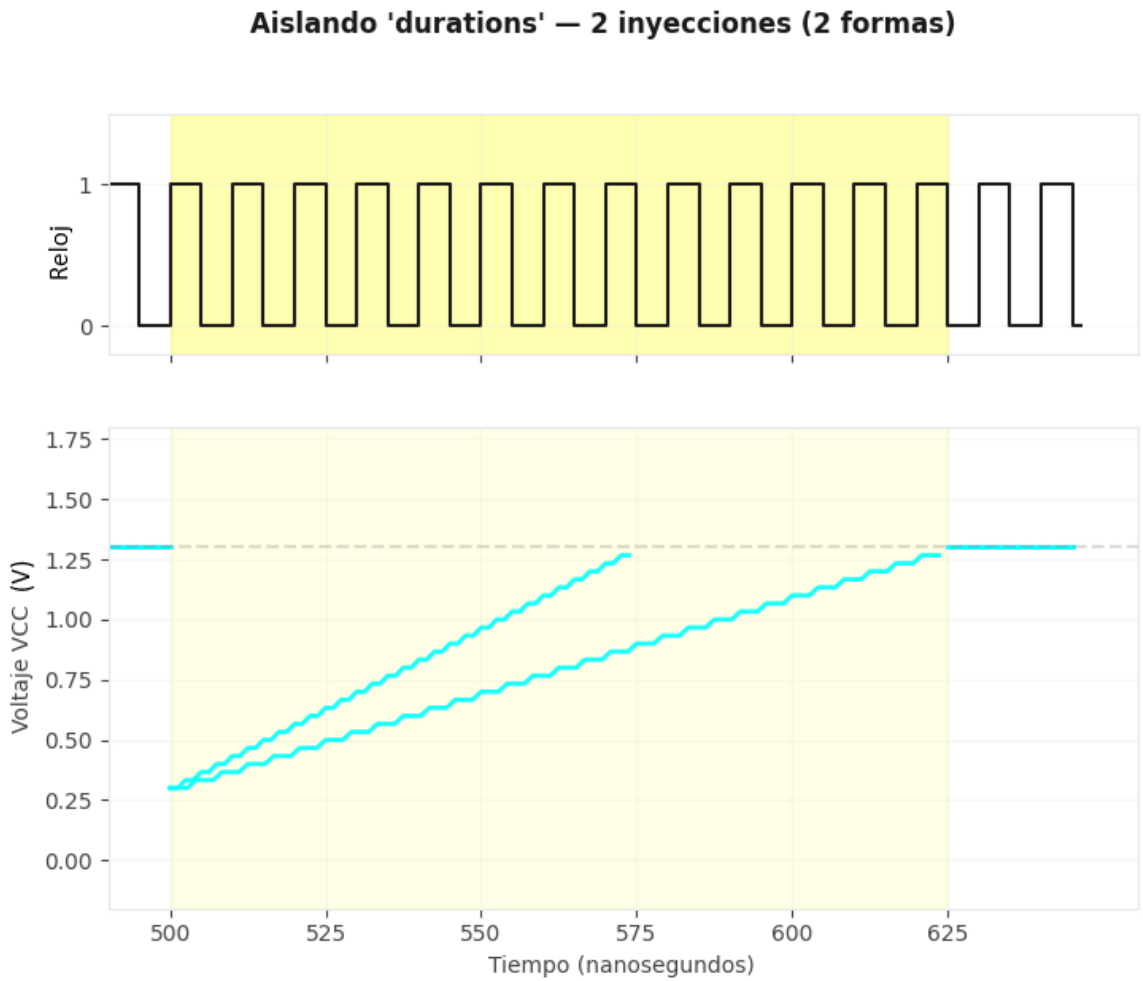
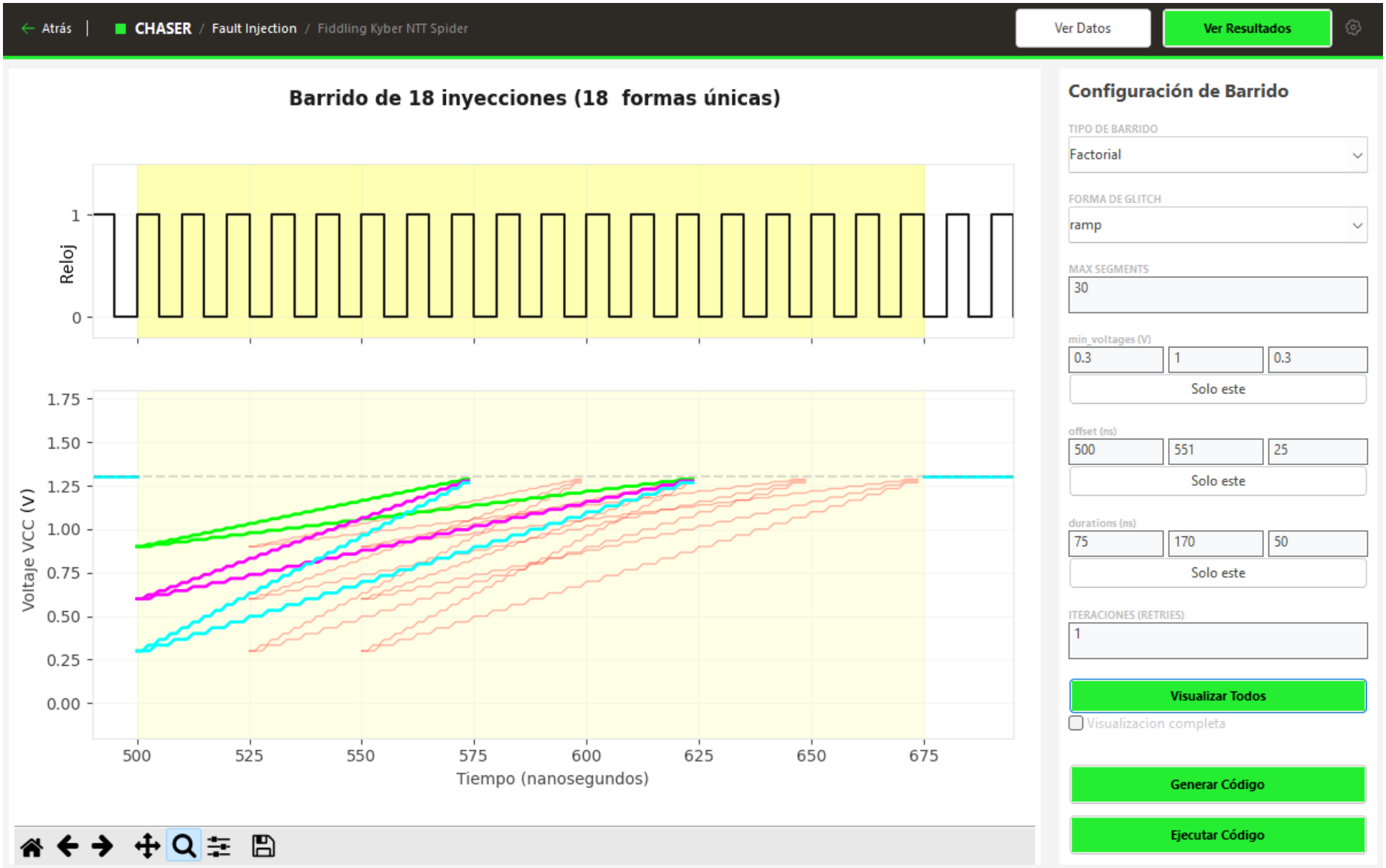
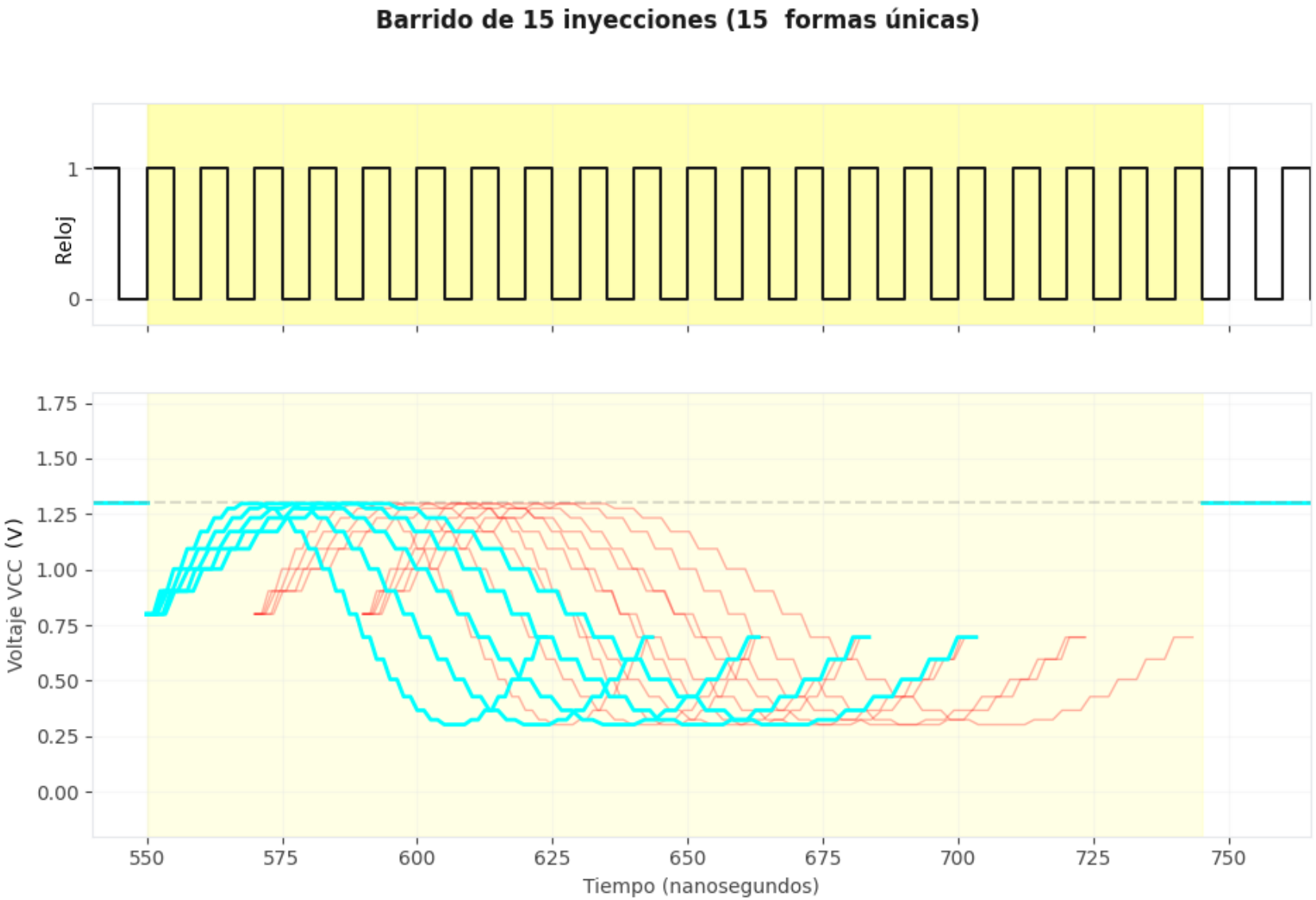


Figura: Aislando `duration` en $[75, 170, 50]$ ns

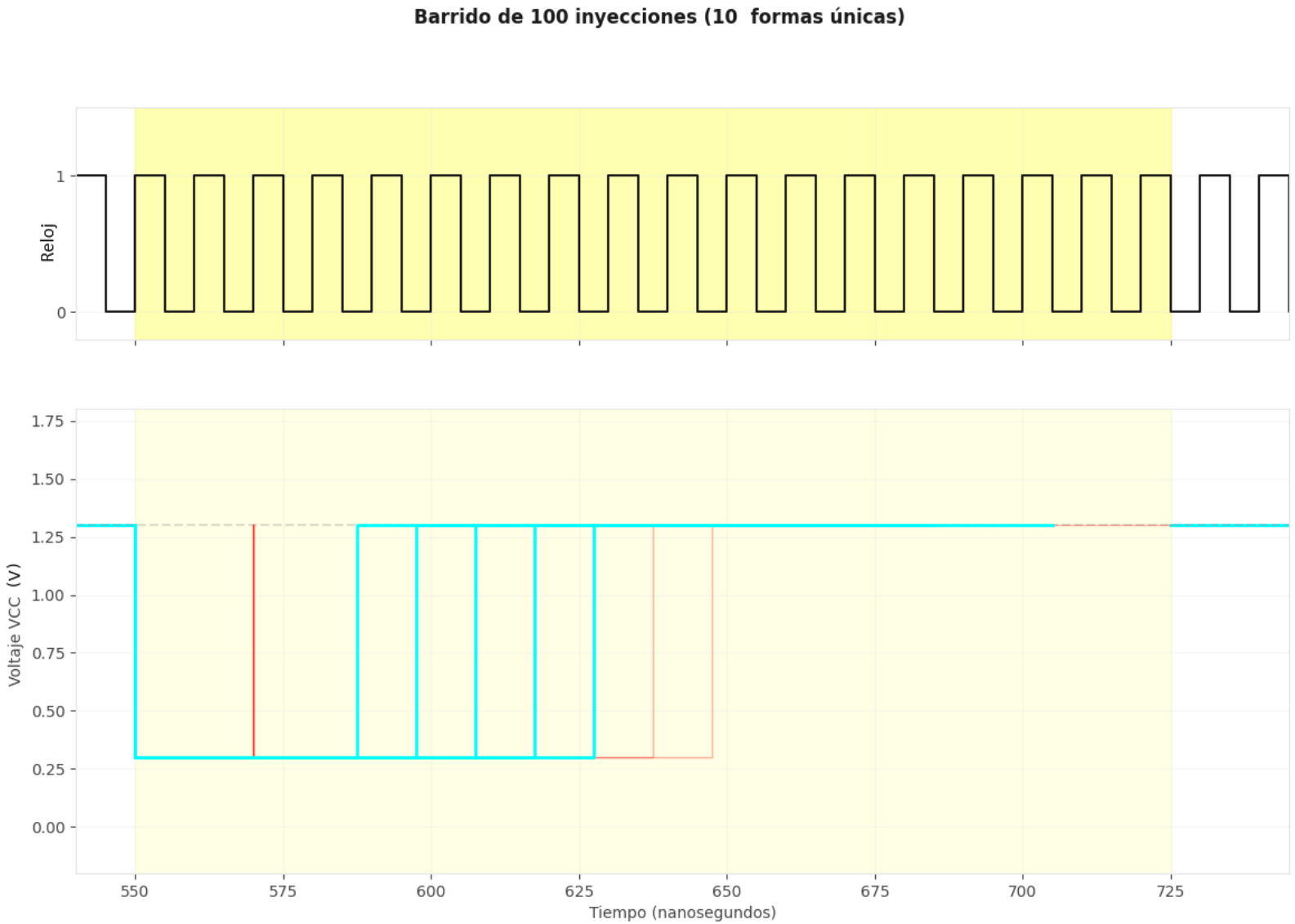
Campaña con Spider (Voltaje): Barrido de ejemplo



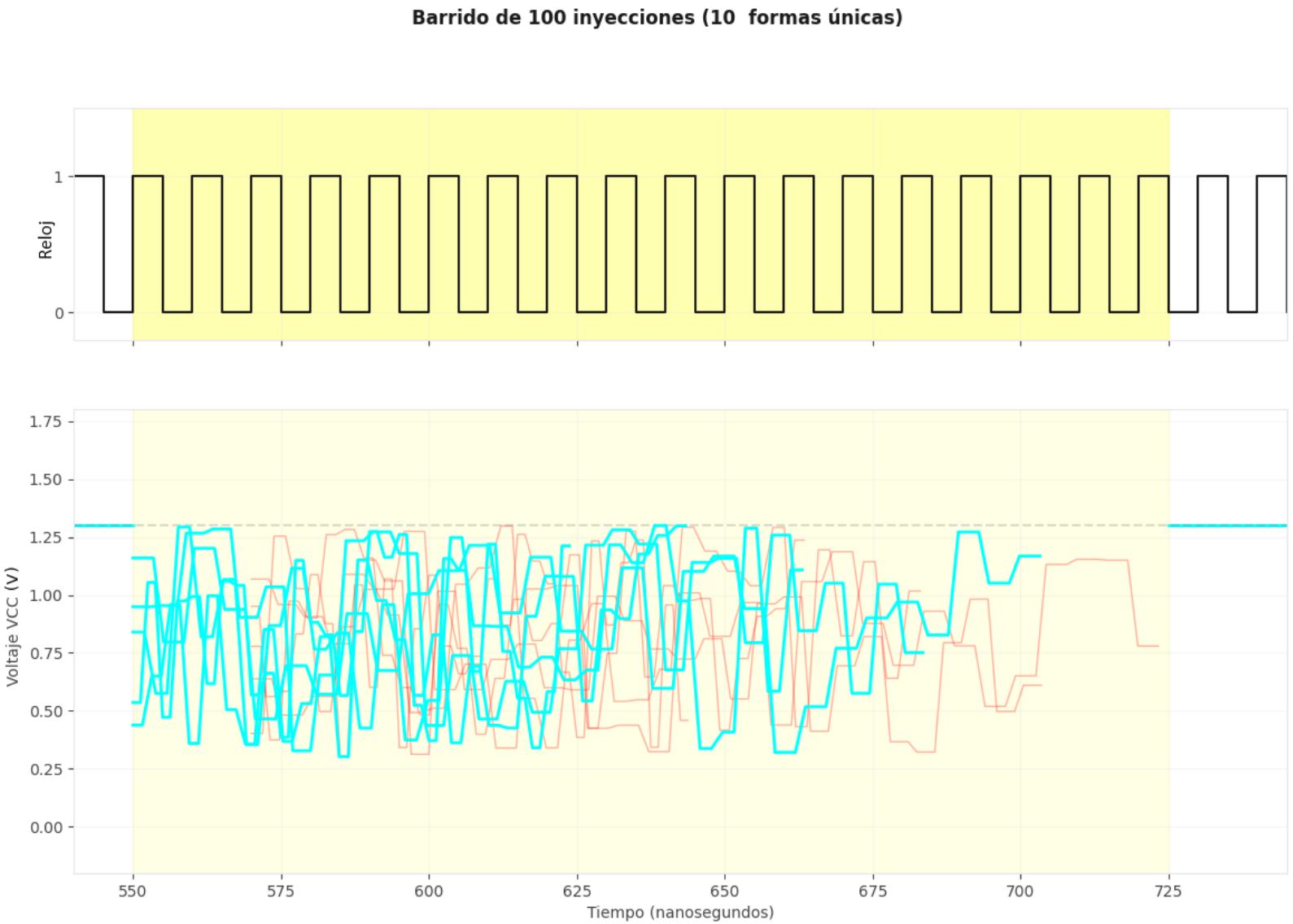
Campaña con Spider (Voltaje): Ejemplo con sine



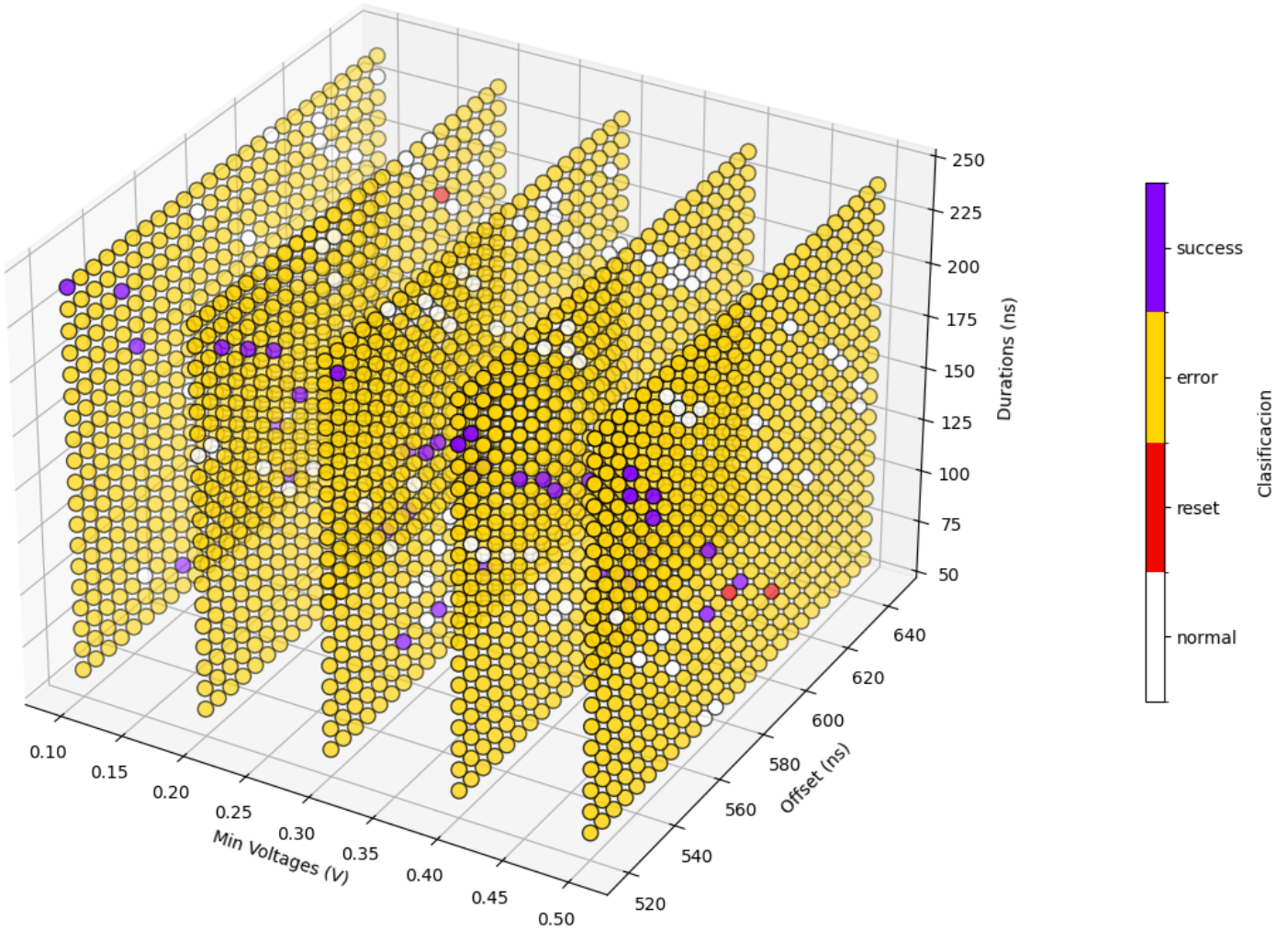
Campaña con Spider (Voltaje): Ejemplo con drop



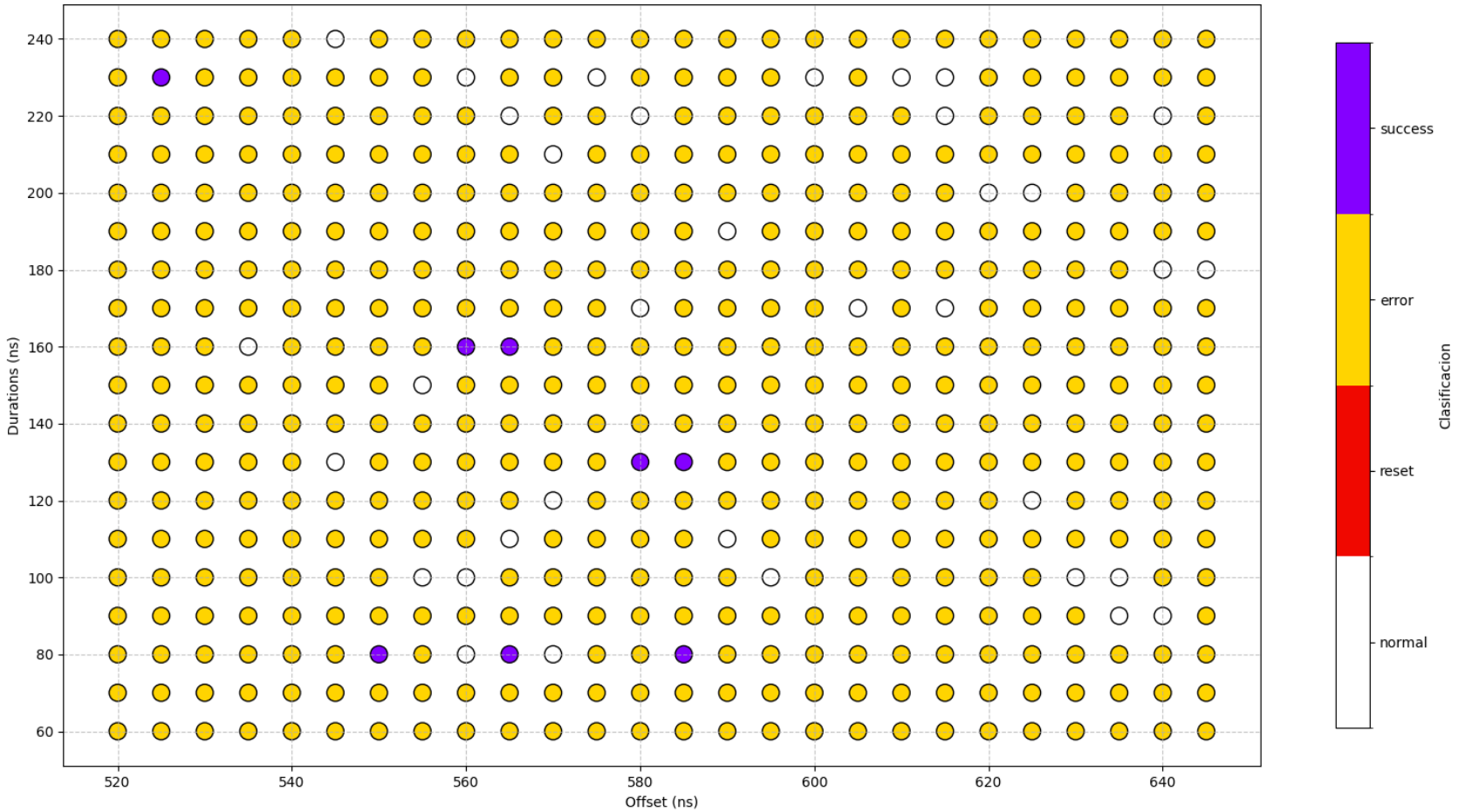
Campaña con Spider (Voltaje): Ejemplo con rand



Campaña con Spider (Voltaje): Primer barrido

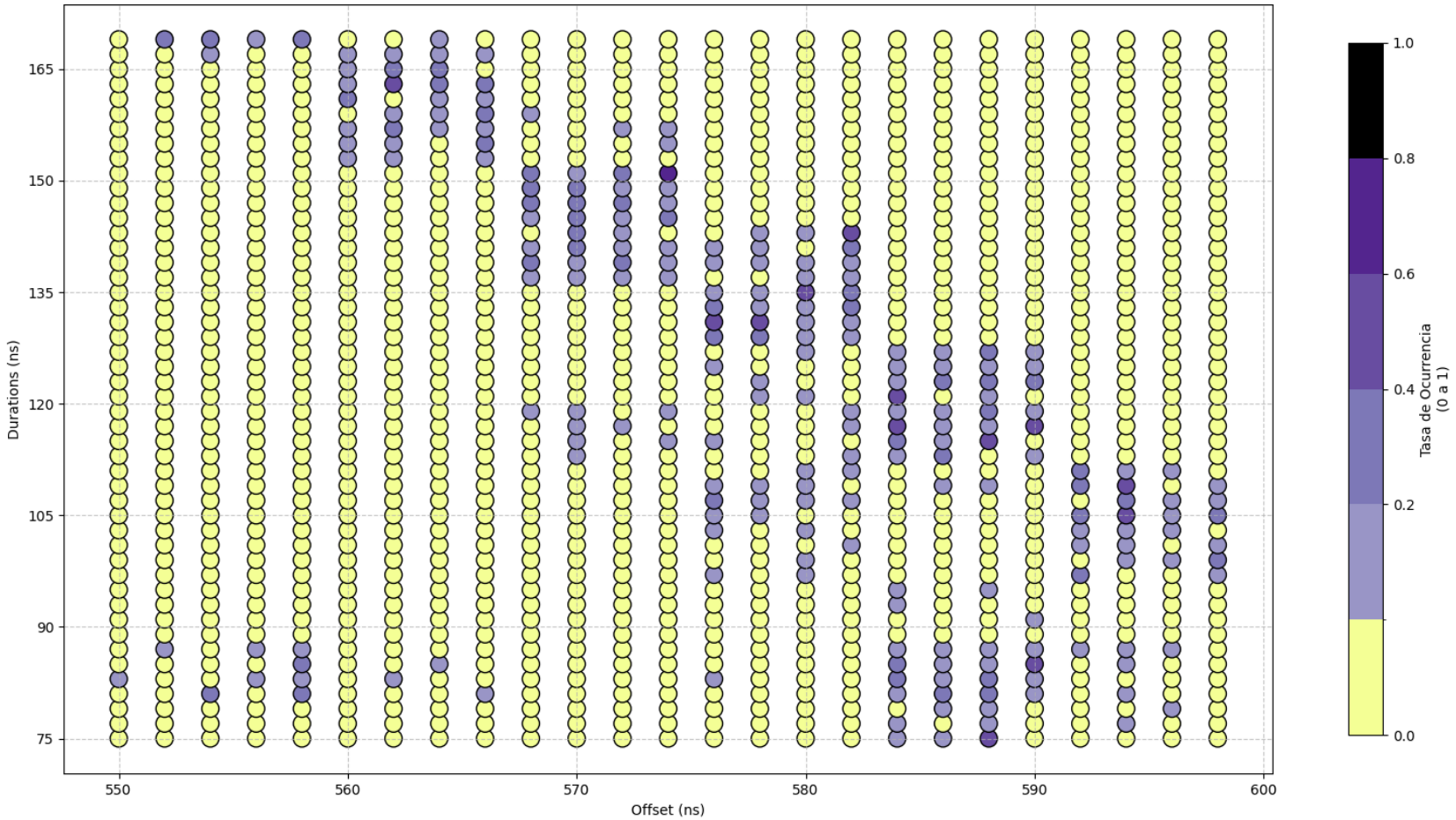


Campaña con Spider (Voltaje): Primer barrido

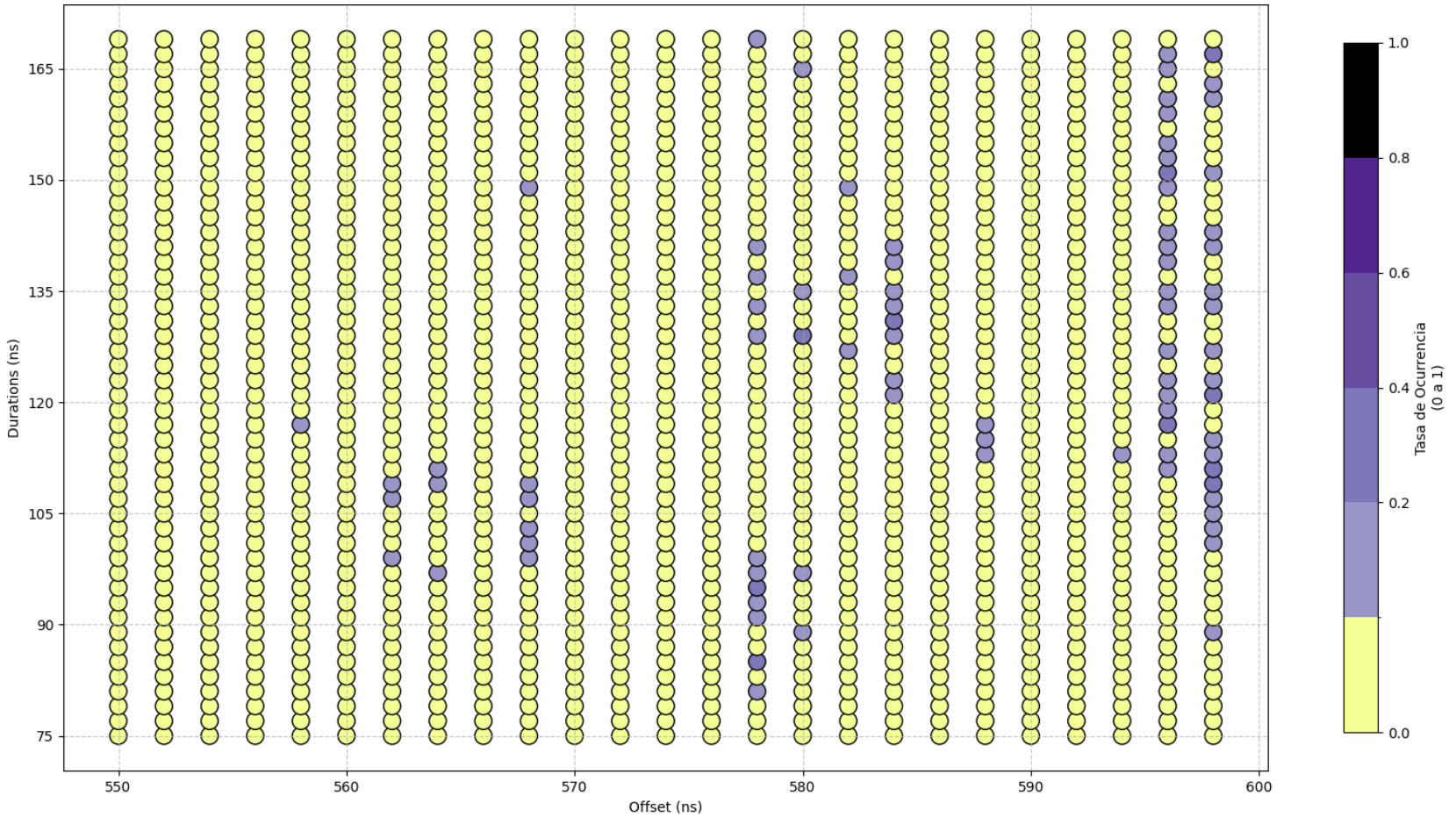


Campaña con Spider (Voltaje): Barrido: sine

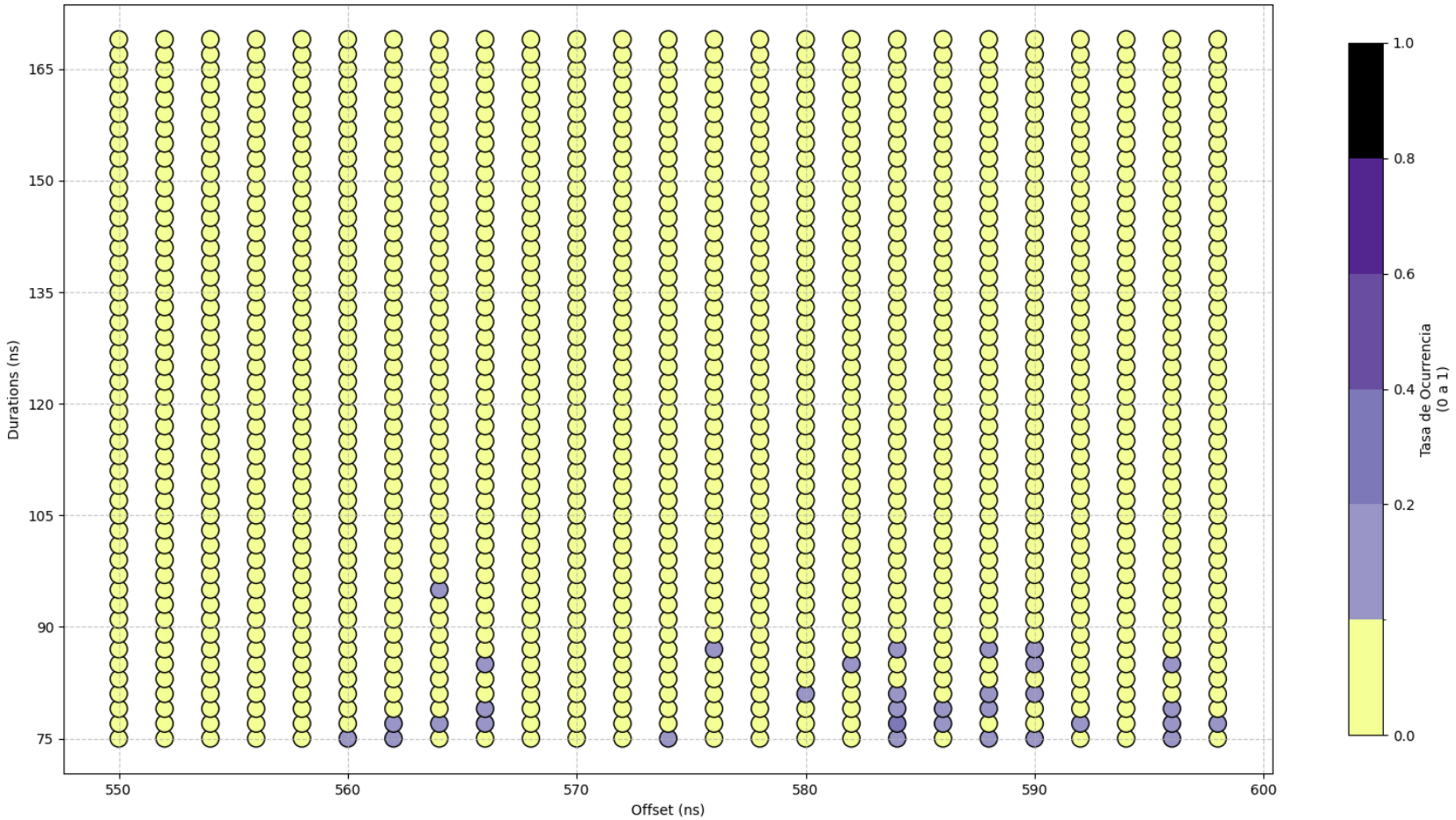
Campaña con Spider (Voltaje): Barrido: sine



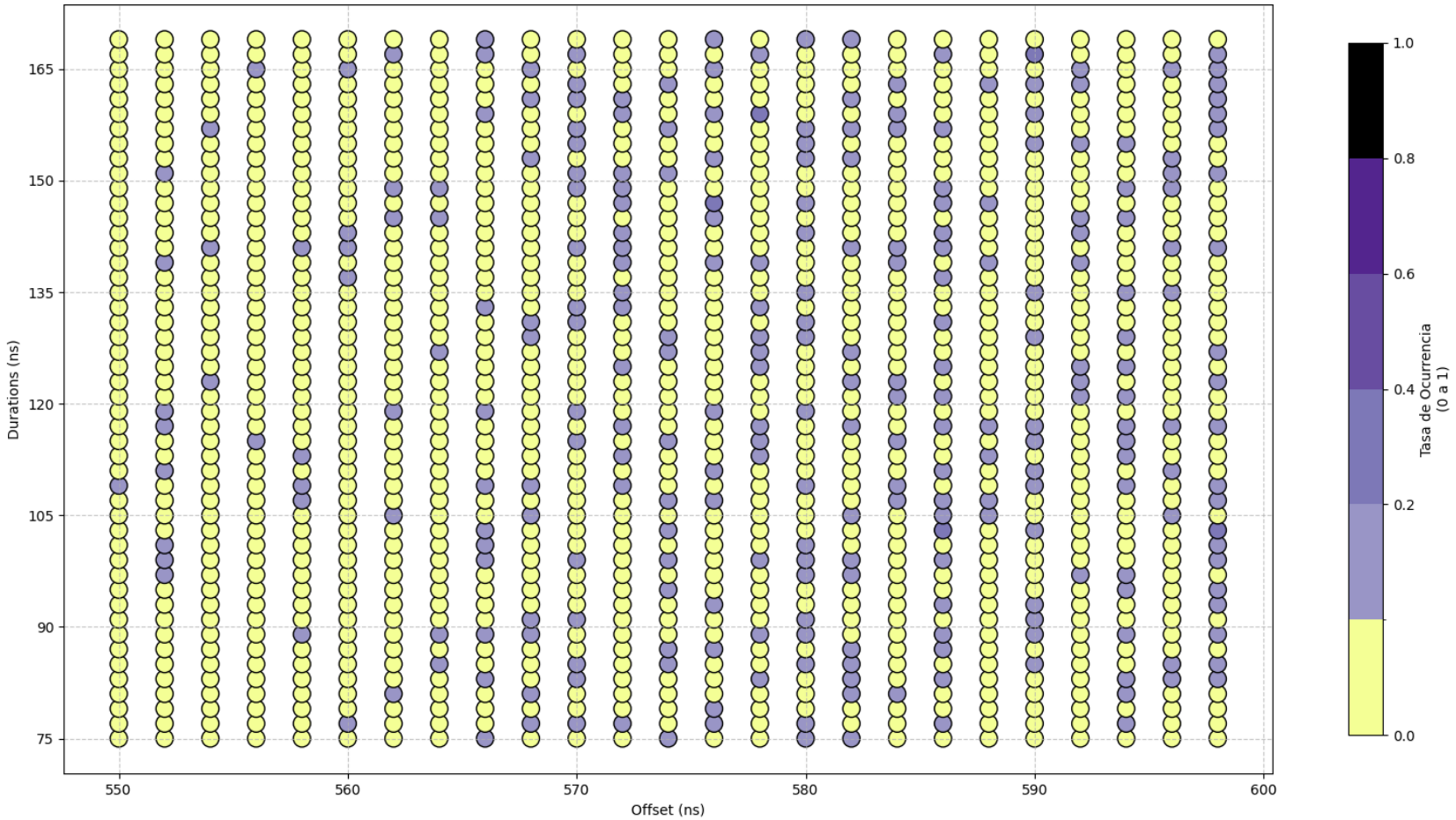
Campaña con Spider (Voltaje): Barrido: drop



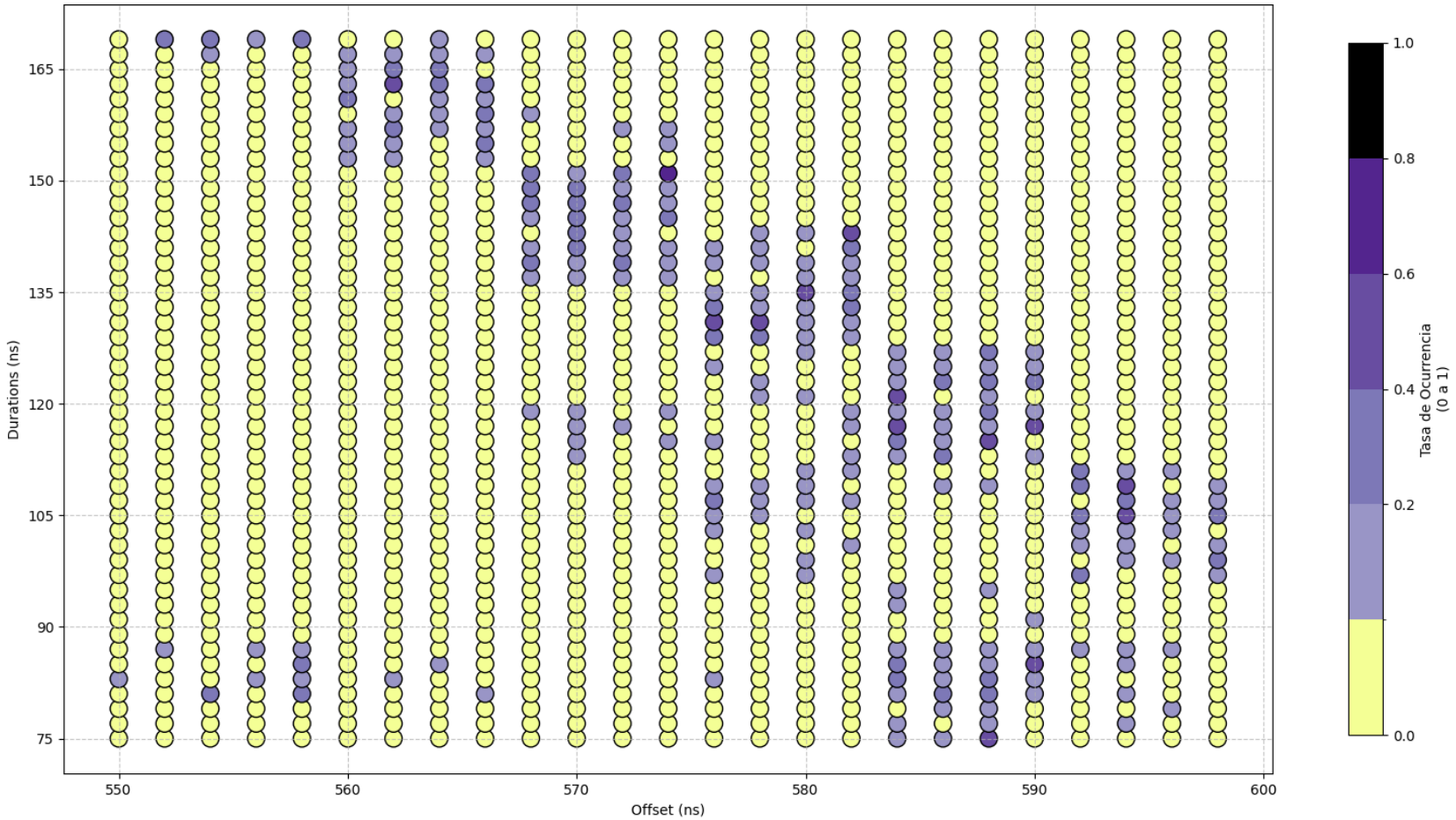
Campaña con Spider (Voltaje): Barrido: ramp



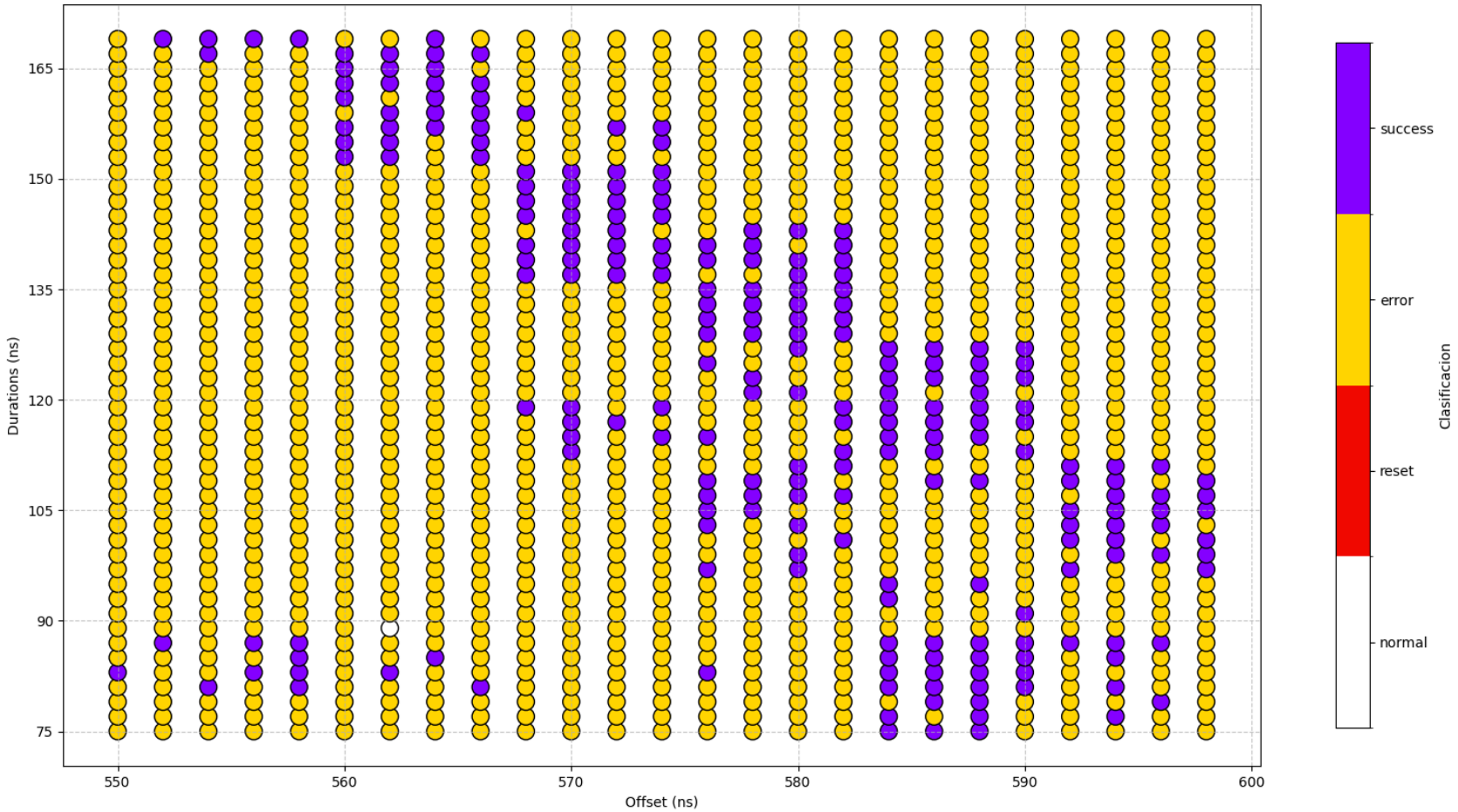
Campaña con Spider (Voltaje): Barrido: rand



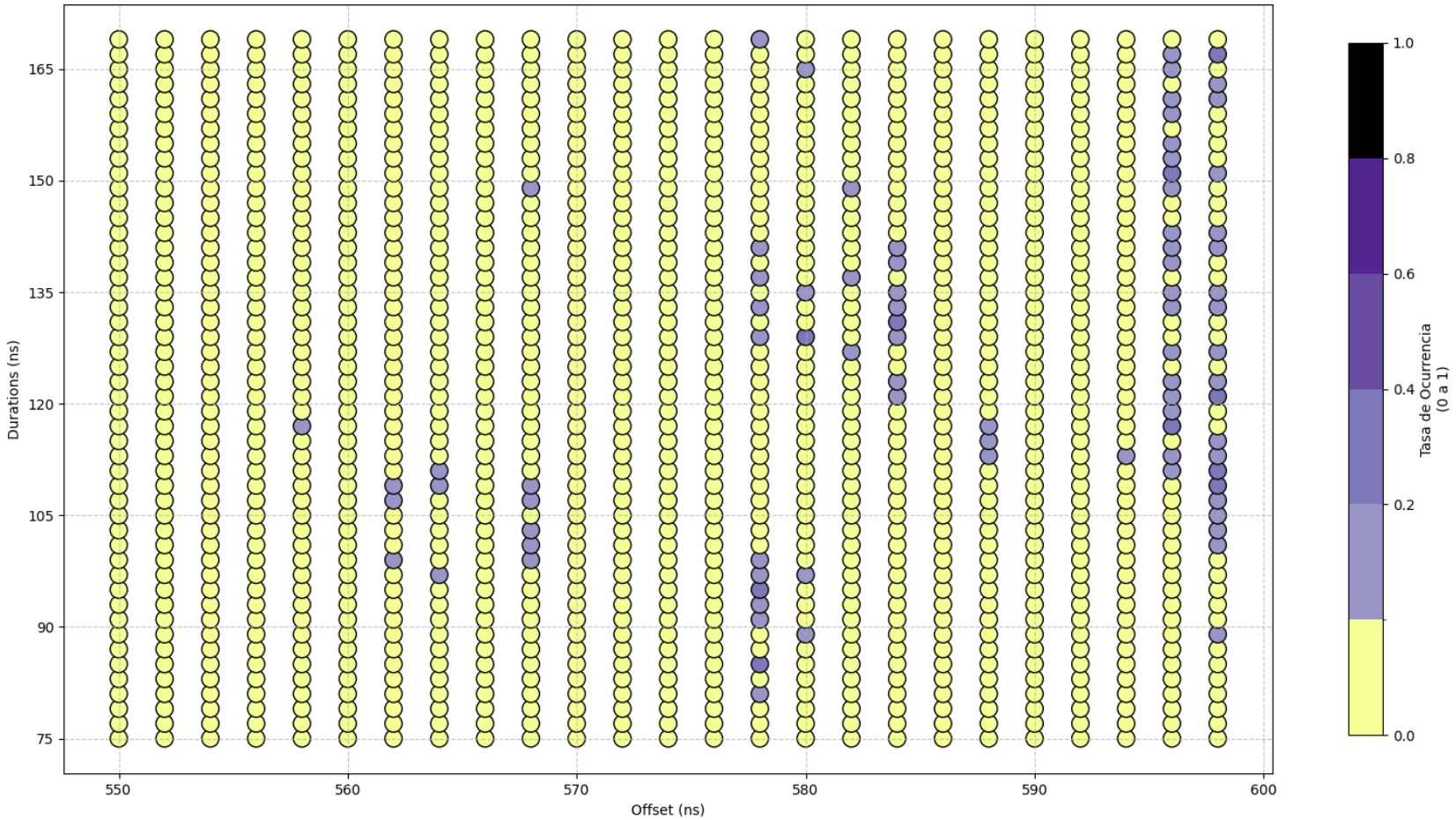
Campaña con Spider (Voltaje): Barrido: sine



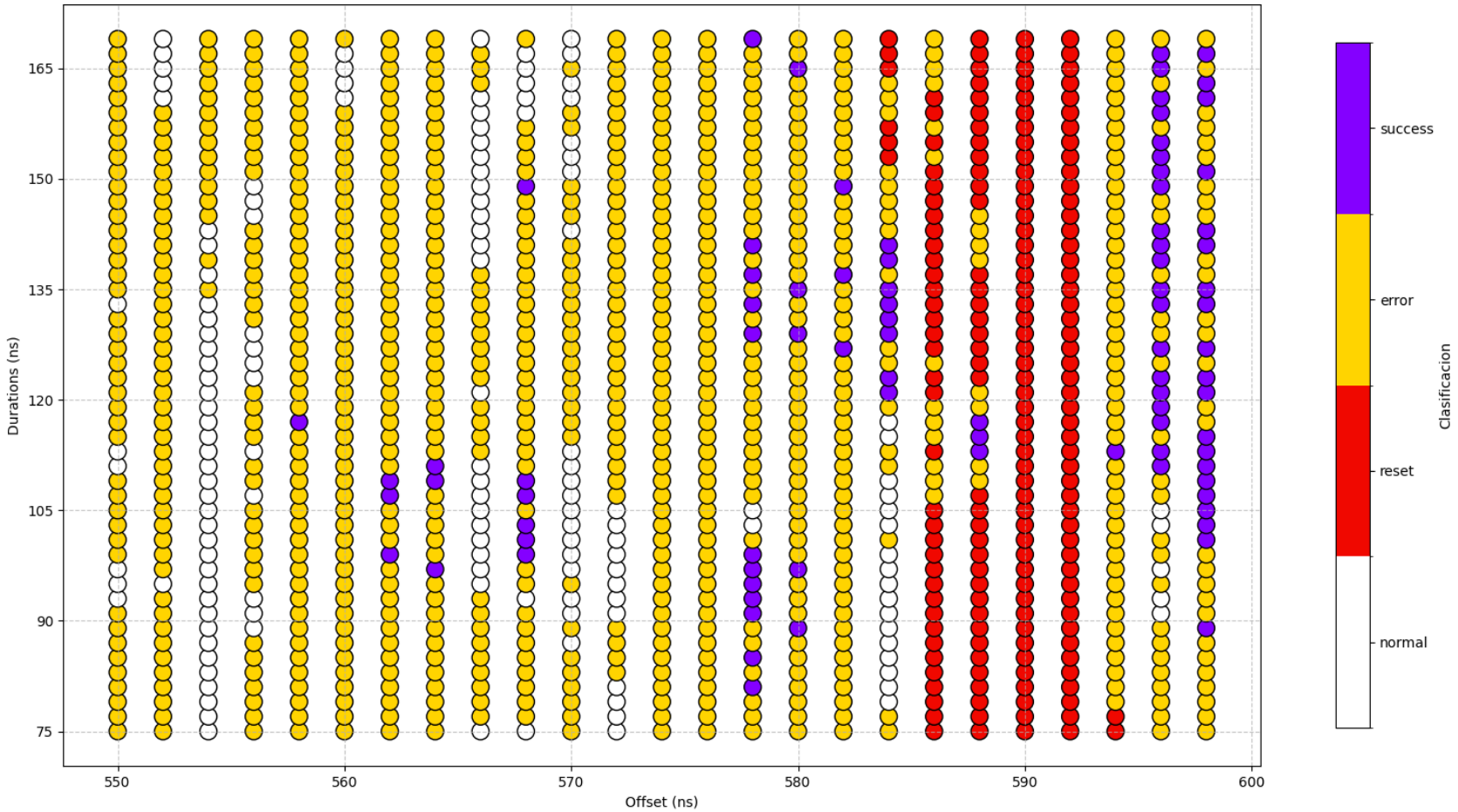
Campaña con Spider (Voltaje): Barrido: sine



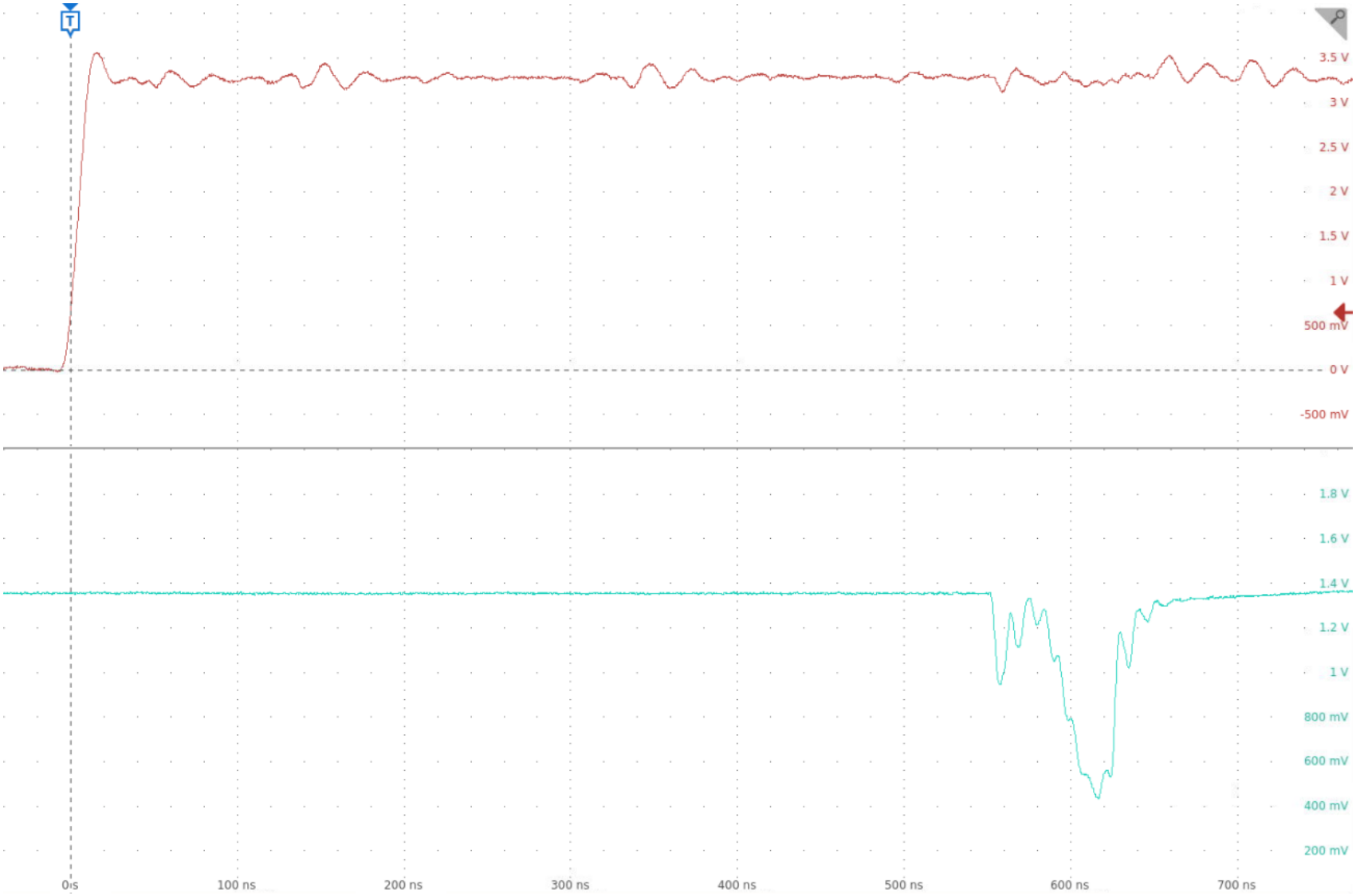
Campaña con Spider (Voltaje): Barrido: drop



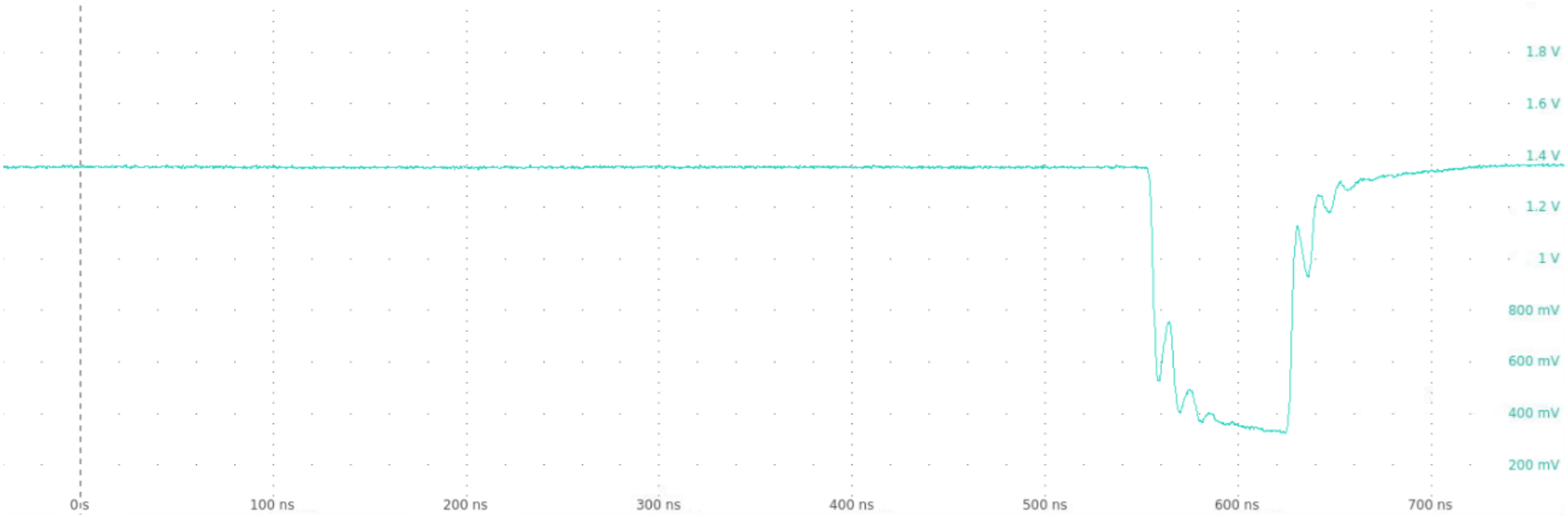
Campaña con Spider (Voltaje): Barrido: drop



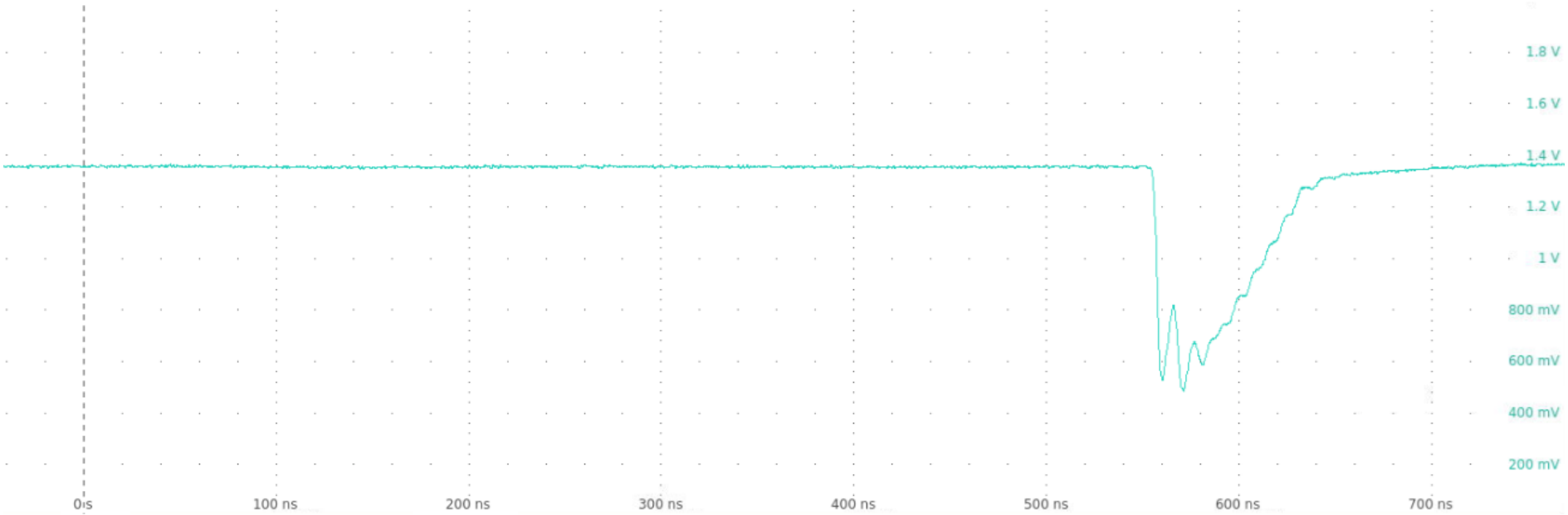
Campaña con Spider (Voltaje): Osciloscopio: sine



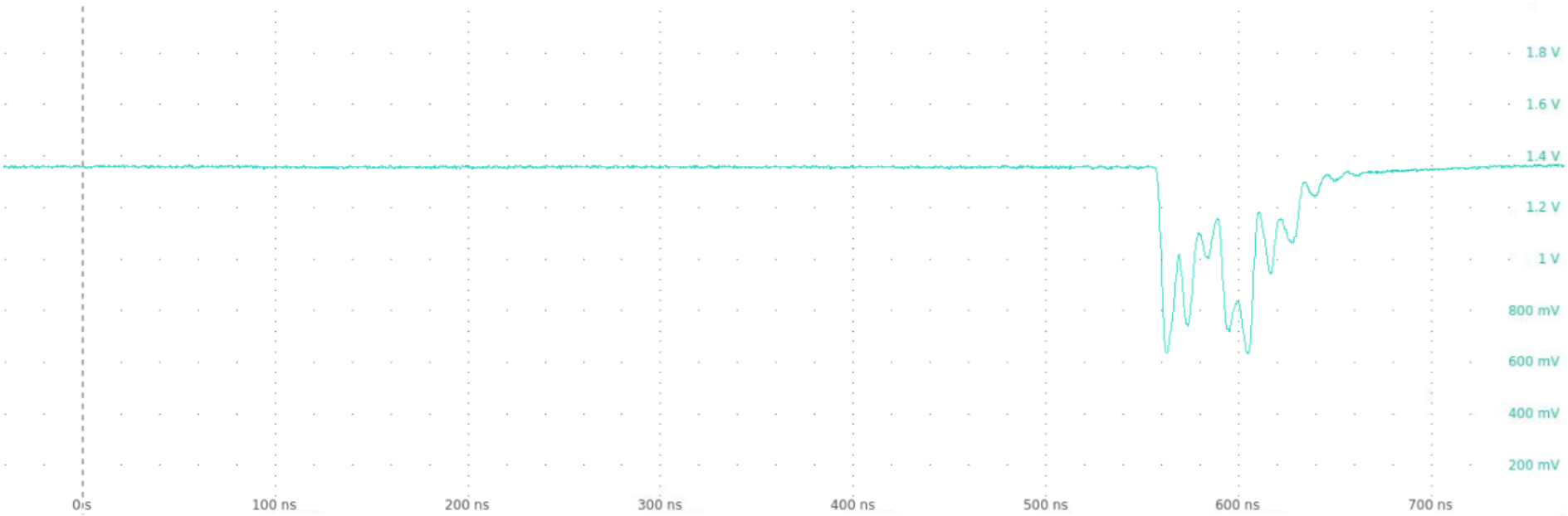
Campaña con Spider (Voltaje): Osciloscopio: drop



Campaña con Spider (Voltaje): Osciloscopio: ramp



Campaña con Spider (Voltaje): Osciloscopio: rand

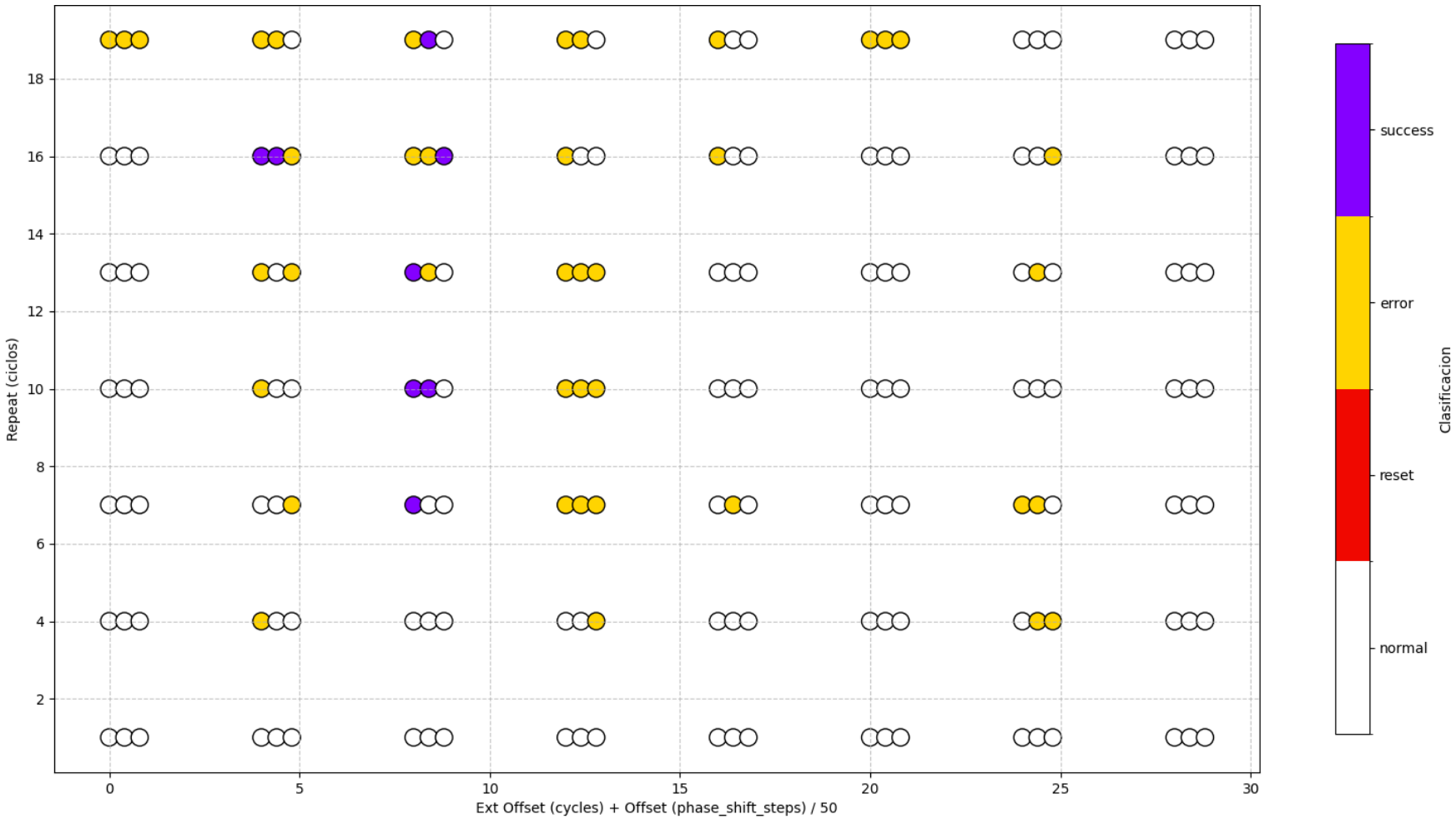


Resumen de Ataques

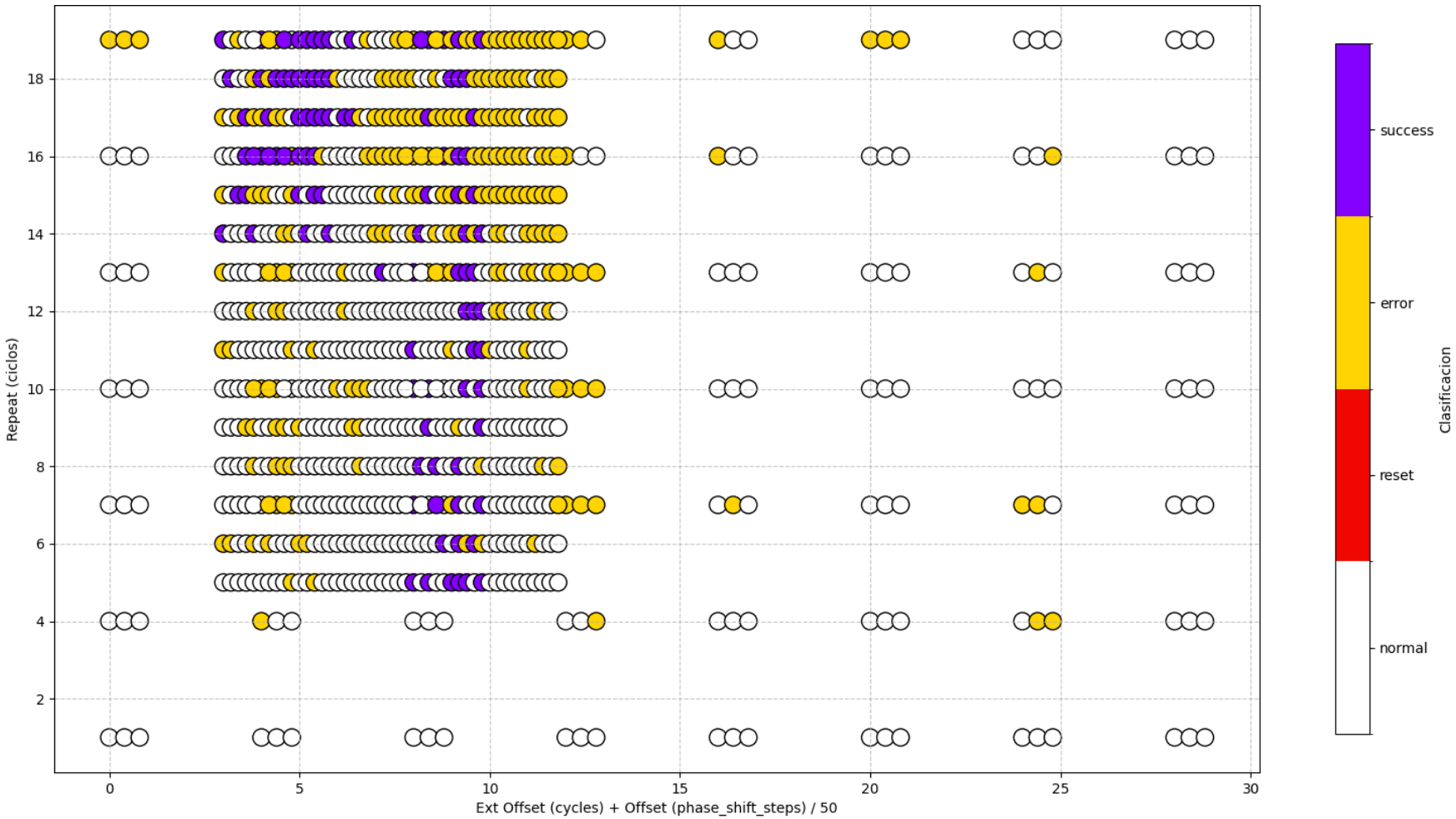
	Generación de Claves	Desencapsulación
Spider (Voltaje)	✓ (66 %)	
ChipWhisperer (Voltaje)		
ChipWhisperer (Reloj)		

Campaña con ChipWhisperer (Voltaje): Primer barrido

Campaña con ChipWhisperer (Voltaje): Primer barrido



Campaña con ChipWhisperer (Voltaje): Segundo barrido



Campaña con ChipWhisperer (Voltaje): Iteraciones

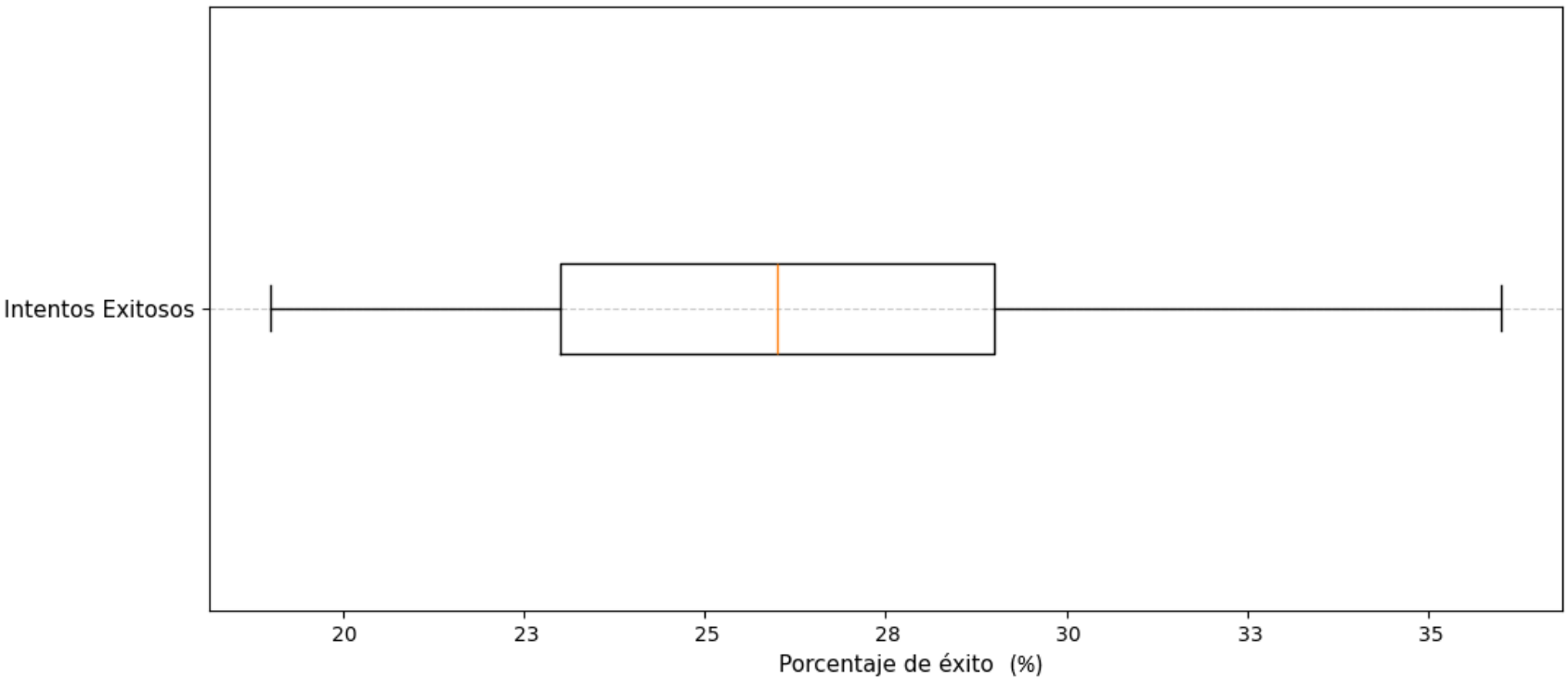
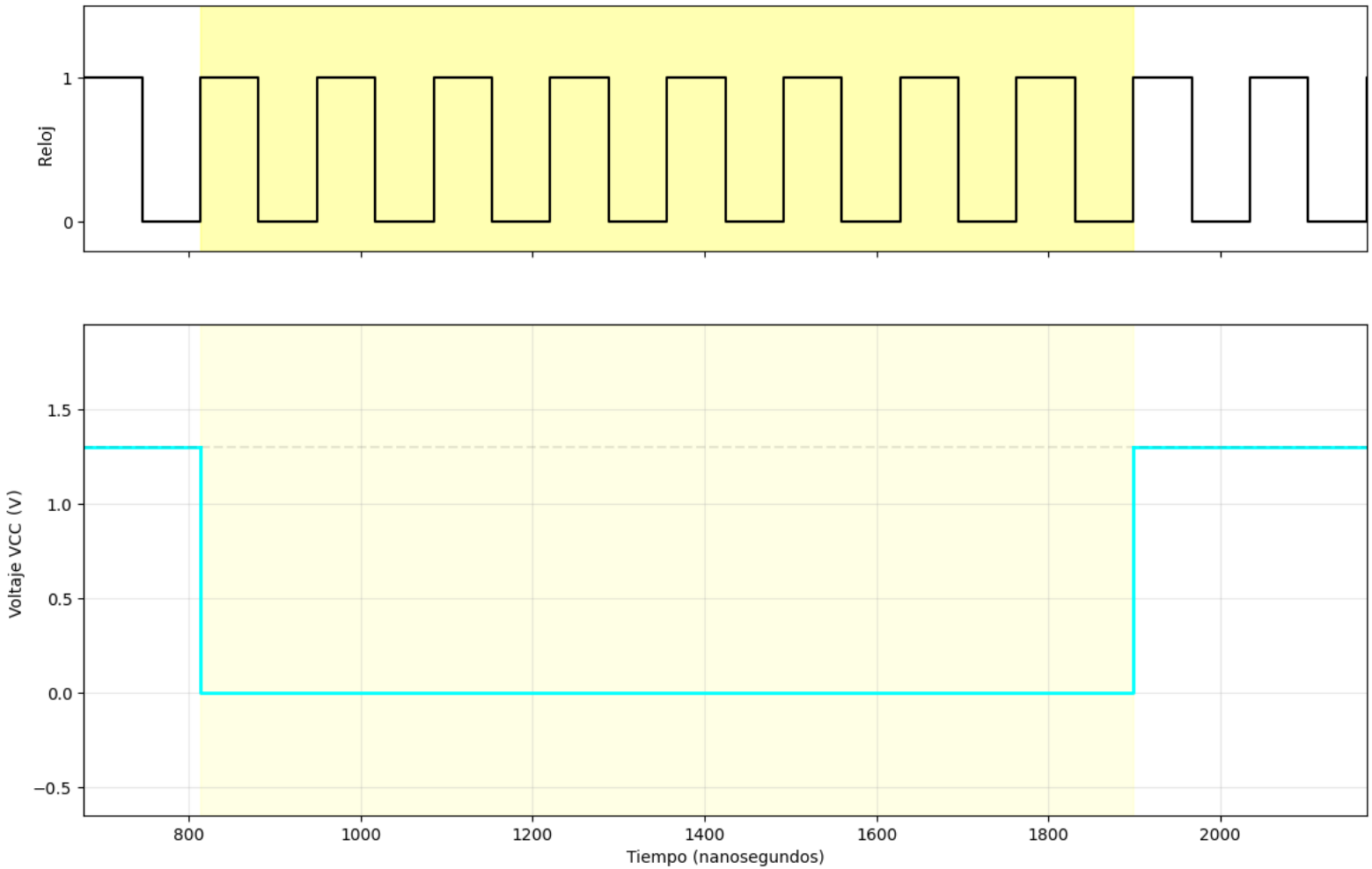


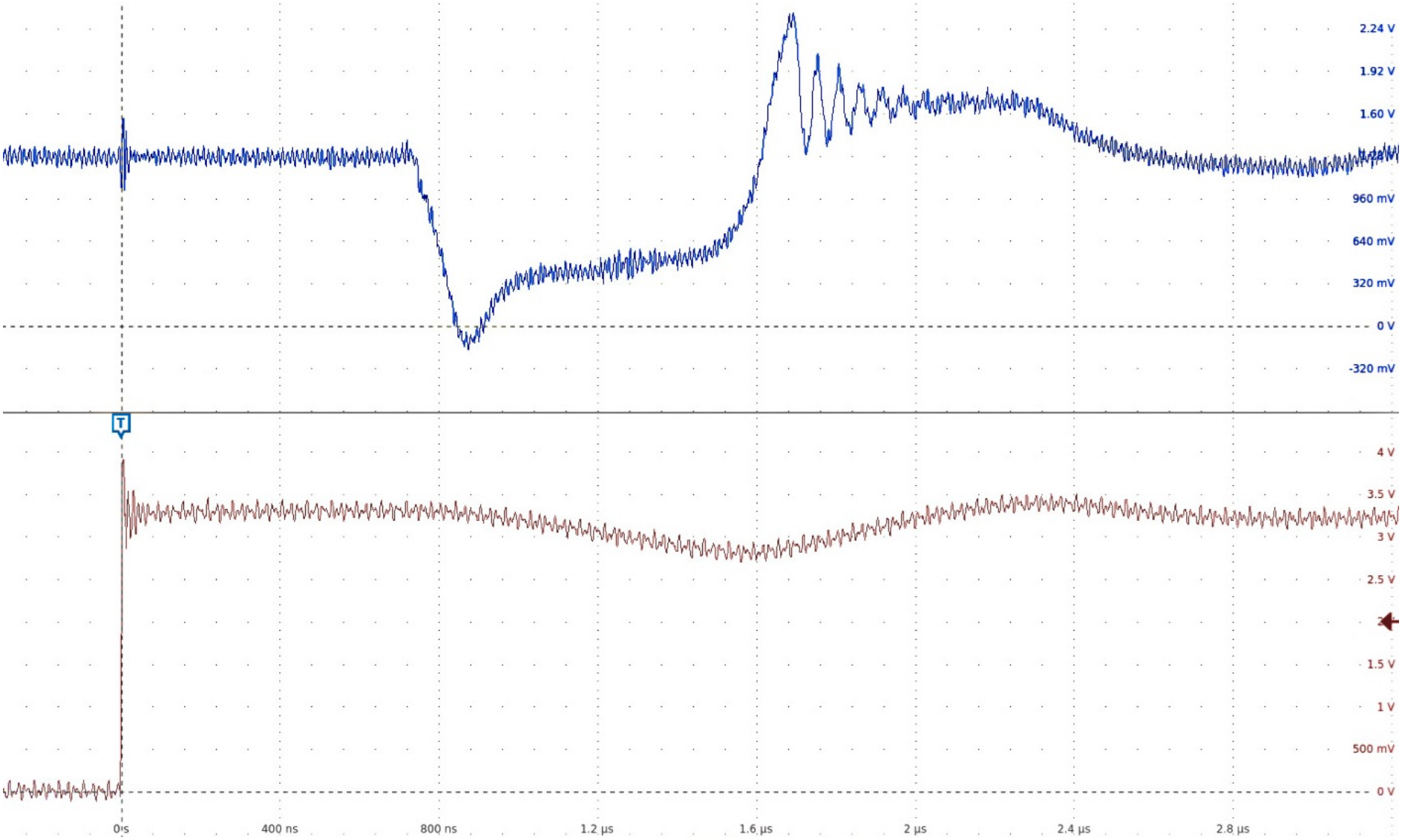
Figura: Diagrama de caja de glitches exitosos sobre 100 iteraciones

Campaña con ChipWhisperer (Voltaje): Iteración particular

Campaña con ChipWhisperer (Voltaje): Iteración particular



Campaña con ChipWhisperer (Voltaje): Iteración particular



Resumen de Ataques

	Generación de Claves	Desencapsulación
Spider (Voltaje)	✓ $(66\,\%)$	
ChipWhisperer (Voltaje)	✓ $(36\,\%)$	
ChipWhisperer (Reloj)		

Campaña con ChipWhisperer (Reloj)

Campaña con ChipWhisperer (Reloj)

Aunque se realizaron más de 80 barridos...

Campaña con ChipWhisperer (Reloj)

Aunque se realizaron más de 80 barridos...

Contabilizando un total de $95\,000$ intentos...

Campaña con ChipWhisperer (Reloj)

Aunque se realizaron más de 80 barridos...

Contabilizando un total de $95\,000$ intentos...

Tras 100 h de ejecución...

Campaña con ChipWhisperer (Reloj)

Aunque se realizaron más de 80 barridos...

Contabilizando un total de $95\,000$ intentos...

Tras $100\,\mathrm{h}$ de ejecución...

No se encontraron glitches exitosos.

Campaña con ChipWhisperer (Reloj)

Campaña con ChipWhisperer (Reloj)

Si bien puede producir un error que no sea un salto de instrucción,

Campaña con ChipWhisperer (Reloj)

Si bien puede producir un error que no sea un salto de instrucción, se decidió modificar el archivo `.hex` para forzar uno:

Campaña con ChipWhisperer (Reloj)

Si bien puede producir un error que no sea un salto de instrucción,

se decidió modificar el archivo `.hex` para forzar uno:

Analizar a qué dirección apunta y observar cuál es la salida.

Campaña con ChipWhisperer (Reloj)

Campaña con ChipWhisperer (Reloj)

```
[SNIP]
8004fa0:    4d70        ldr r5, [pc, #448]    @ skip
8004fa2:    ae36        add r6,  sp, #216
8004fa4:    2703        movs     r7, #3
[SNIP]
```

Instrucción objetivo en ensamblador para el ataque de reloj.

Campaña con ChipWhisperer (Reloj)

```
[SNIP]
8004fa0:    4d70        ldr r5, [pc, #448]    @ skip
8004fa2:    ae36        add r6,  sp, #216
8004fa4:    2703        movs     r7, #3
[SNIP]
```

Instrucción objetivo en ensamblador para el ataque de reloj.

```
:104F900000BF00BF00BF00BF00BF00BFBFF34F8F07
:104FA000704D36AE03270FCD0FC695E80F0086E88B
:104FB0000F0036A9084621228DF8F870FCF7B2F9E7
```

Contenido original del archivo `.hex` antes de sustituir la instrucción.

Campaña con ChipWhisperer (Reloj)

```
[SNIP]
8004fa0:    4d70        ldr r5, [pc, #448]    @ skip
8004fa2:    ae36        add r6,  sp, #216
8004fa4:    2703        movs     r7, #3
[SNIP]
```

Instrucción objetivo en ensamblador para el ataque de reloj.

```
:104F900000BF00BF00BF00BF00BF00BFBFF34F8F07
:104FA000704D36AE03270FCD0FC695E80F0086E88B
:104FB0000F0036A9084621228DF8F870FCF7B2F9E7
```

Contenido original del archivo `.hex` antes de sustituir la instrucción.

```
:104F900000BF00BF00BF00BF00BF00BFBFF34F8F07
:104FA00000BF36AE03270FCD0FC695E80F0086E889
:104FB0000F0036A9084621228DF8F870FCF7B2F9E7
```

Contenido modificado del archivo `.hex` tras insertar la instrucción `NOP`.

Campaña con ChipWhisperer (Reloj)

Campaña con ChipWhisperer (Reloj)

Resultado: El registro `r5` tenía el valor `0xc60`.

Campaña con ChipWhisperer (Reloj)

Resultado: El registro `r5` tenía el valor `0xc60`.

Al revisar las respuestas obtenidas en los experimentos anteriores, se verificó que esa misma era la respuesta recibida.

Campaña con ChipWhisperer (Reloj)

Resultado: El registro `r5` tenía el valor `0xc60`.

Al revisar las respuestas obtenidas en los experimentos anteriores, se verificó que esa misma era la respuesta recibida.

Aunque el ataque no resultó exitoso, puede asegurarse que la herramienta produce saltos de instrucciones.

Resumen de Ataques

	Generación de Claves	Desencapsulación
Spider (Voltaje)	✓ $(66\,\%)$	
ChipWhisperer (Voltaje)	✓ $(36\,\%)$	
ChipWhisperer (Reloj)	✗	

Resumen de Ataques

	Generación de Claves	Desencapsulación
Spider (Voltaje)	✓ (66 %)	
ChipWhisperer (Voltaje)	✓ (36 %)	
ChipWhisperer (Reloj)	✗	

Recuperación de Claves

Recuperación de Claves

Búsqueda por fuerza bruta sobre el espacio reducido de coeficientes:

Recuperación de Claves

Búsqueda por fuerza bruta sobre el espacio reducido de coeficientes:

Se recrea el secreto, se junta con el `ct` en `NTT`, se aplica la `INVNTT`, se calcula el ruido del mensaje, y nos quedamos con el mínimo.

Recuperación de Claves

Algorithm 4 Fuerza bruta para recuperar el secreto de Kyber

procedure BRUTEFORCE(pk, ct, $data$)
  $\mathcal{V} \leftarrow \{-\eta, \ldots, \eta\}$ ▷ Valores posibles para cada coeficiente, $[-2..2]$
  $(\mathbf{u}, v) \leftarrow$ UNPACKCT(ct) ▷ Se recuperan las dos componentes del texto cifrado
  $\hat{\mathbf{u}} \leftarrow [\text{NTT}(u_i) \mid u_i \in \mathbf{u}]$ ▷ Transforma $\mathbf{u}$ al dominio NTT
  best_noise $\leftarrow +\infty$
  best_coeffs $\leftarrow \perp$
  best_m $\leftarrow \perp$
  best_ss $\leftarrow \perp$
  for all coeffs $\in \mathcal{V}^{2K}$ **do**
    $\hat{\mathbf{s}}_{faulted} \leftarrow$ MAKEFAULTEDSKPVFROMCOEFFS(coeffs) ▷ Secreto candidato
    $m_p \leftarrow$ POLYVECBASEMULACC($\hat{\mathbf{s}}_{faulted}, \hat{\mathbf{u}}$) ▷ Se calcula $\mathbf{u} \cdot \mathbf{s}$ en NTT
    $m_p \leftarrow \text{InvNTT}(m_p)$ ▷ Volvemos al dominio normal
    $m_{poly} \leftarrow v - m_p$ ▷ Se aproxima el polinomio del mensaje
    total_noise $\leftarrow 0$
    for all $x \in m_{poly}$ **do**
      $x_{mod} \leftarrow x$ mód q
      $dist_0 \leftarrow \text{mín}(x_{mod},\ q - x_{mod})$ ▷ Distancia a 0
      $dist_{q/2} \leftarrow |x_{mod} - \lfloor q/2 \rfloor|$ ▷ Distancia a $q/2$
      total_noise $\leftarrow$ total_noise $+ \text{mín}(dist_0,\ dist_{q/2})$ ▷ Sumamos el mínimo
    end for
    if total_noise $<$ best_noise **then** ▷ Actualizamos el mejor candidato
      $m \leftarrow$ POLYTOMSG(m_{poly})
      $ss \leftarrow$ DERIVESUCCESSSS(pk, ct, m)
      best_noise $\leftarrow$ total_noise
      best_coeffs $\leftarrow$ coeffs
      best_m $\leftarrow$ m
      best_ss $\leftarrow$ ss
    end if
  end for
  plaintext $\leftarrow data \oplus$ REPEATKEY(best_ss, $|data|$) ▷ Se recupera el plaintext
  return (plaintext, best_coeffs, best_m, best_ss)
end procedure

Recuperación de Claves

```
PS C:\Users\icanut\OneDrive - IKERLAN S. COOP\Escritorio> python.exe .\recover.py
[*] brute force 5^6...
[*] tested=2000 (best noise so far: 92696)
[*] tested=4000 (best noise so far: 92696)
[*] tested=6000 (best noise so far: 92696)
[*] tested=8000 (best noise so far: 92000)
[*] tested=10000 (best noise so far: 91743)
[*] tested=12000 (best noise so far: 14708)
[*] tested=14000 (best noise so far: 14708)

[+] BRUTE FORCE FINISHED
Best candidate coeffs: (1, 0, -1, -1, 0, -1) (noise=14708)
Inferred Shared Secret: CAD9771F8DF716F852D909722323464A6524DB40C42BDEC49E799848D8F8E040
Recovered Data: b'Hello Kyber on STM32!'
```

Recuperación de Claves

```
PS C:\Users\icanut\OneDrive - IKERLAN S. COOP\Escritorio> .\recover.exe
[*] brute force 5^6...
[*] tested=2000 (best noise so far: 91743)
[*] tested=4000 (best noise so far: 91743)
[*] tested=6000 (best noise so far: 14708)
[*] tested=8000 (best noise so far: 14708)
[*] tested=10000 (best noise so far: 14708)
[*] tested=12000 (best noise so far: 14708)
[*] tested=14000 (best noise so far: 14708)

[+] BRUTE FORCE FINISHED
Best candidate coeffs: (1, 0, -1, -1, 0, -1) (noise=14708)
Inferred Shared Secret: CAD9771F8DF716F852D909722323464A6524DB40C42BDEC49E799848D8F8E040
Recovered Data (hex): 48656C6C6F204B79626572206F6E2053544D333221
Recovered Data (ascii): Hello Kyber on STM32!
PS C:\Users\icanut\OneDrive - IKERLAN S. COOP\Escritorio> Measure-Command {start-process .\recover.exe -Wait}


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 1
Milliseconds      : 75
Ticks             : 10758869
TotalDays         : 1,24523946759259E-05
TotalHours        : 0,000298857472222222
TotalMinutes      : 0,0179314483333333
TotalSeconds      : 1,0758869
TotalMilliseconds : 1075,8869
```

Conclusiones del ataque a la NTT

Conclusiones del ataque a la NTT

- El ataque por voltaje es viable.

Conclusiones del ataque a la NTT

- El ataque por voltaje es viable.
- La forma del glitch tiene mucha influencia.

Conclusiones del ataque a la NTT

- El ataque por voltaje es viable.
- La forma del glitch tiene mucha influencia.
- En esta implementación y con esta configuración, el ataque por reloj no fue suficiente.

Conclusiones del ataque a la NTT

- El ataque por voltaje es viable.
- La forma del glitch tiene mucha influencia.
- En esta implementación y con esta configuración, el ataque por reloj no fue suficiente.
- Se puede realizar el ataque completo en C y recuperar la clave secreta en poco más de un segundo.

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Motivación

Motivación

Xagawa et al. (2021), Fault-Injection Attacks against NIST's Post-Quantum Cryptography Round 3 KEM Candidates.

Motivación

Xagawa et al. (2021), Fault-Injection Attacks against NIST's Post-Quantum Cryptography Round 3 KEM Candidates.

Usan una ChipWhisperer y ataques de reloj.

Motivación

Xagawa et al. (2021), Fault-Injection Attacks against NIST's Post-Quantum Cryptography Round 3 KEM Candidates.

Usan una ChipWhisperer y ataques de reloj.

Buscamos replicar el mismo resultado y probar con voltaje.

Transformación de Fujisaki–Okamoto de Kyber

Transformación de Fujisaki–Okamoto de Kyber

El receptor verifica que el ct recibido sea válido.

Transformación de Fujisaki–Okamoto de Kyber

El receptor verifica que el ct recibido sea válido.

Primero descifra el ct para obtener m'.

Transformación de Fujisaki–Okamoto de Kyber

El receptor verifica que el ct recibido sea válido.

Primero descifra el ct para obtener m'.

Con el mensaje m' y su clave pública pk, recalcula ct'.

Transformación de Fujisaki–Okamoto de Kyber

El receptor verifica que el ct recibido sea válido.

Primero descifra el ct para obtener m'.

Con el mensaje m' y su clave pública pk, recalcula ct'.

Entonces compara ct con ct'.

Transformación de Fujisaki–Okamoto de Kyber

El receptor verifica que el ct recibido sea válido.

Primero descifra el ct para obtener m'.

Con el mensaje m' y su clave pública pk, recalcula ct'.

Entonces compara ct con ct'.

Si no coinciden, el ct fue modificado y se usa de secreto una clave aleatoria para no filtrar información.

Transformación de Fujisaki–Okamoto de Kyber

Esto es para aumentar la seguridad.

Transformación de Fujisaki–Okamoto de Kyber

Esto es para aumentar la seguridad.

Xagawa et al. demostraron que, si se elude esta verificación, es posible recuperar la clave secreta después de $3\,000$ consultas con textos cifrados elegidos.

Pseudocódigo de la transformación de FO

Pseudocódigo de la transformación de FO

1: **procedure** $\mathrm{CCA.DECAPS}(sk, ct)$
2: $(pk, \mathcal{H}(pk), z) \leftarrow \mathrm{UnpackSK}(sk)$
3: $m' = \mathrm{CPA.Decrypt}(sk, ct)$ ▷ Descifrado del texto cifrado a mensaje
4: $(\bar{K}', r') = \mathcal{G}(m' \parallel \mathcal{H}(pk))$ ▷ Generación de la preclave $\bar{K}$
5: $T = \bar{K}'$
6: $ct_R = \mathrm{CPA.Encrypt}(pk, m', r')$ ▷ Recifrado del mensaje descifrado
7: **if** $\mathrm{CompareCT}(ct_R, ct) == 0$ **then** ▷ Comparación del texto cifrado
8: $T = z$
9: **end if**
10: **return** $K = \mathrm{KDF}(T \parallel \mathcal{H}(ct))$ ▷ Generación de la clave de sesión
11: **end procedure**

Código en C de la transformación de FO

Código en C de la transformación de FO

```c
static void run_kyber_test(void) {
    [SNIP]
    memset(kyber_ct, 0, sizeof(kyber_ct)); // Generamos un ct inválido
    status = crypto_kem_dec(kyber_key_a, kyber_ct, kyber_sk);
    trigger_low();
    if (status) {
        send_line("DEC_FAIL END"); return;
    }
    [SNIP]
}

int crypto_kem_dec(unsigned char *ss, const unsigned char *ct, const unsigned char *sk) {
    [SNIP]
    fail = indcpa_enc_cmp(ct, buf, pk, kr + KYBER_SYMBYTES); // <- Glitch acá
    rkprf(ss, sk + KYBER_SECRETKEYBYTES - KYBER_SYMBYTES, ct);
    cmov(ss, kr, KYBER_SYMBYTES, (uint8_t) (1 - fail));
    return fail; // return 0; <- Modificación
}

unsigned char indcpa_enc_cmp(const unsigned char *c, const unsigned char *m,
                             const unsigned char *pk, const unsigned char *coins) {
    [SNIP]
    trigger_high();
    rc = ~rc + 1; rc >>= 63;
    return (unsigned char)rc;
}
```

Código en ensamblador de la transformación de FO

Código en ensamblador de la transformación de FO

```
@ trigger_high()
8005dd0: 425b       negs    r3, r3
8005dd2: eb63 0303  sbc.w   r3, r3, r3
8005dd6: ea4f 78d3  mov.w   r8, r3, lsr #31
[SNIP]
@ -- cmov(ss, kr, 32, 1 - fail): generación de la máscara
8005eb0: f108 33ff      add.w   r3, r8, #4294967295
8005eb4: b2db           uxtb    r3, r3
8005eb6: f363 0507      bfi     r5, r3, #0, #8
8005eba: f363 250f      bfi     r5, r3, #8, #8
8005ebe: f363 4517      bfi     r5, r3, #16, #8
8005ec2: f363 651f      bfi     r5, r3, #24, #8
@ -- r5 = 0xffffffff si fail = 0
@ -- r5 = 0x00000000 si fail = 1
```

Campaña con Spider (Voltaje)

Se analizaron $16\,000$ puntos, $4\,000$ con cada forma.

Campaña con Spider (Voltaje)

Se analizaron $16\,000$ puntos, $4\,000$ con cada forma.

78 éxitos con `ramp`, con `sine` 53, con `drop` 39 y con `rand` 34.

Campaña con Spider (Voltaje)

Se analizaron $16\,000$ puntos, $4\,000$ con cada forma.

78 éxitos con `ramp`, con `sine` 53, con `drop` 39 y con `rand` 34.

Se consiguió un $78\,\%$ de éxito con `ramp` sobre 50 iteraciones en un punto.

Resumen de Ataques

	Generación de Claves	Desencapsulación
Spider (Voltaje)	✓ (66 %)	✓ (78 %)
ChipWhisperer (Voltaje)	✓ (36 %)	
ChipWhisperer (Reloj)	✗	

Campaña con ChipWhisperer (Voltaje): Primer barrido

Campaña con ChipWhisperer (Voltaje): Primer barrido

`ext_offset` (ciclos)	`offset` (pss)	`repeat` (ciclos)	It.	Conclusiones

Campaña con ChipWhisperer (Voltaje): Primer barrido

`ext_offset` (ciclos)	`offset` (pss)	`repeat` (ciclos)	It.	Conclusiones
$[1, 60, 2]$	$[0, 50, 20]$	$[4, 20, 3]$	1	Éxitos en $(43, 0, 16)$, $(41, 40, 16)$, $(43, 40, 16)$

Campaña con ChipWhisperer (Voltaje): Primer barrido

`ext_offset` (ciclos)	`offset` (pss)	`repeat` (ciclos)	It.	Conclusiones
$[1, 60, 2]$	$[0, 50, 20]$	$[4, 20, 3]$	1	Éxitos en $(43, 0, 16)$, $(41, 40, 16)$, $(43, 40, 16)$
$[40, 47, 1]$	$[0, 100, 1]$	$[13, 19, 1]$	1	Se consiguieron muchos éxitos entre el `ext_offset` 42 y 47 y entre `repeat` 16 y 19

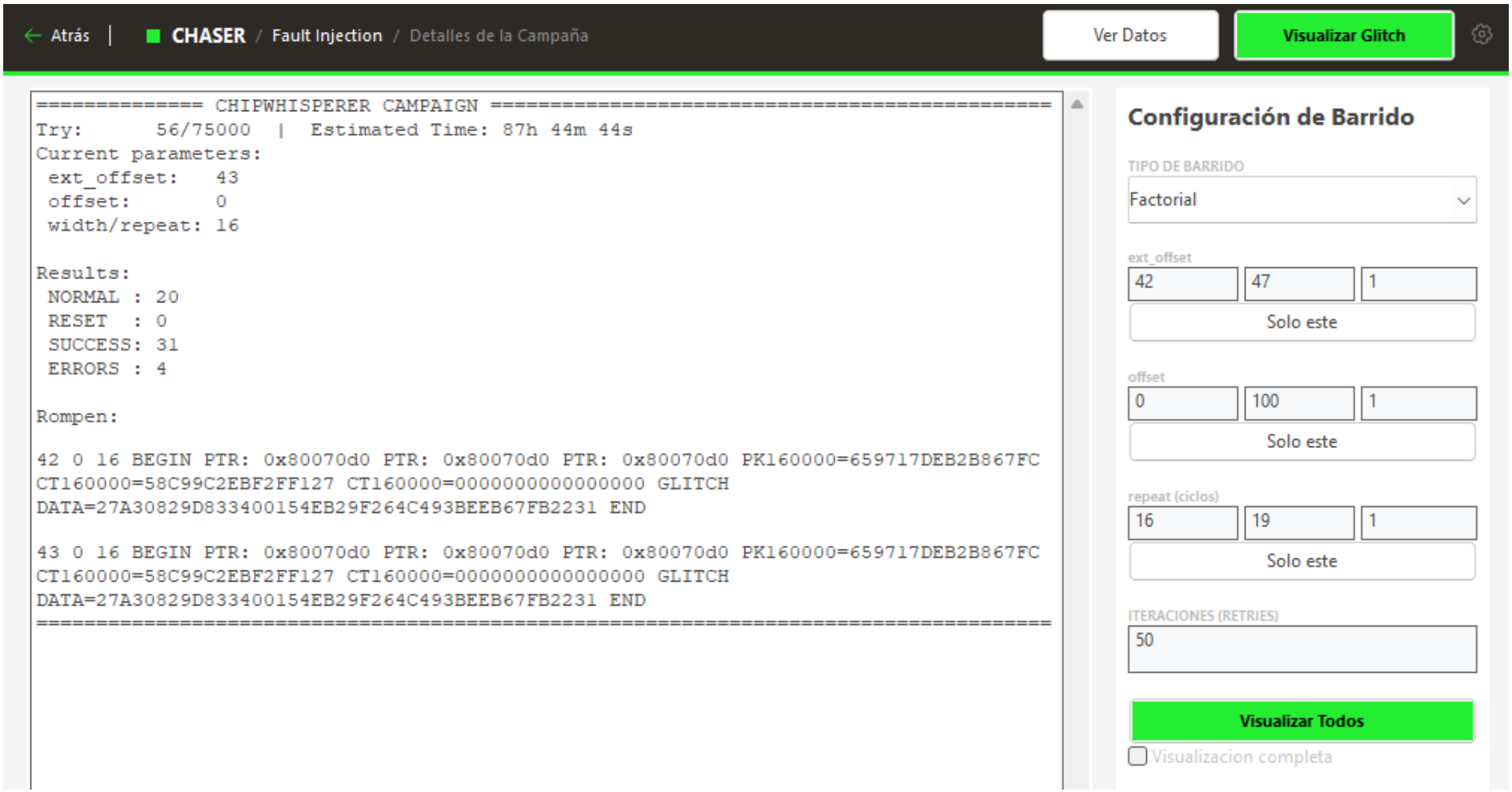
Campaña con ChipWhisperer (Voltaje): Primer barrido

`ext_offset` (ciclos)	`offset` (pss)	`repeat` (ciclos)	It.	Conclusiones
$[1, 60, 2]$	$[0, 50, 20]$	$[4, 20, 3]$	1	Éxitos en $(43, 0, 16)$, $(41, 40, 16)$, $(43, 40, 16)$
$[40, 47, 1]$	$[0, 100, 1]$	$[13, 19, 1]$	1	Se consiguieron muchos éxitos entre el `ext_offset` 42 y 47 y entre `repeat` 16 y 19
$[42, 47, 1]$	$[0, 100, 1]$	$[16, 19, 1]$	50	Muy buena repetibilidad con `repeat` 17 ciclos

Campaña con ChipWhisperer (Voltaje): Resultados

Campaña con ChipWhisperer (Voltaje): Resultados

De los distintos barridos realizados, el más relevante fue el último, compuesto por $75\,000$ puntos, y llevó más de 87 h:



Campaña con ChipWhisperer (Voltaje): Resultados

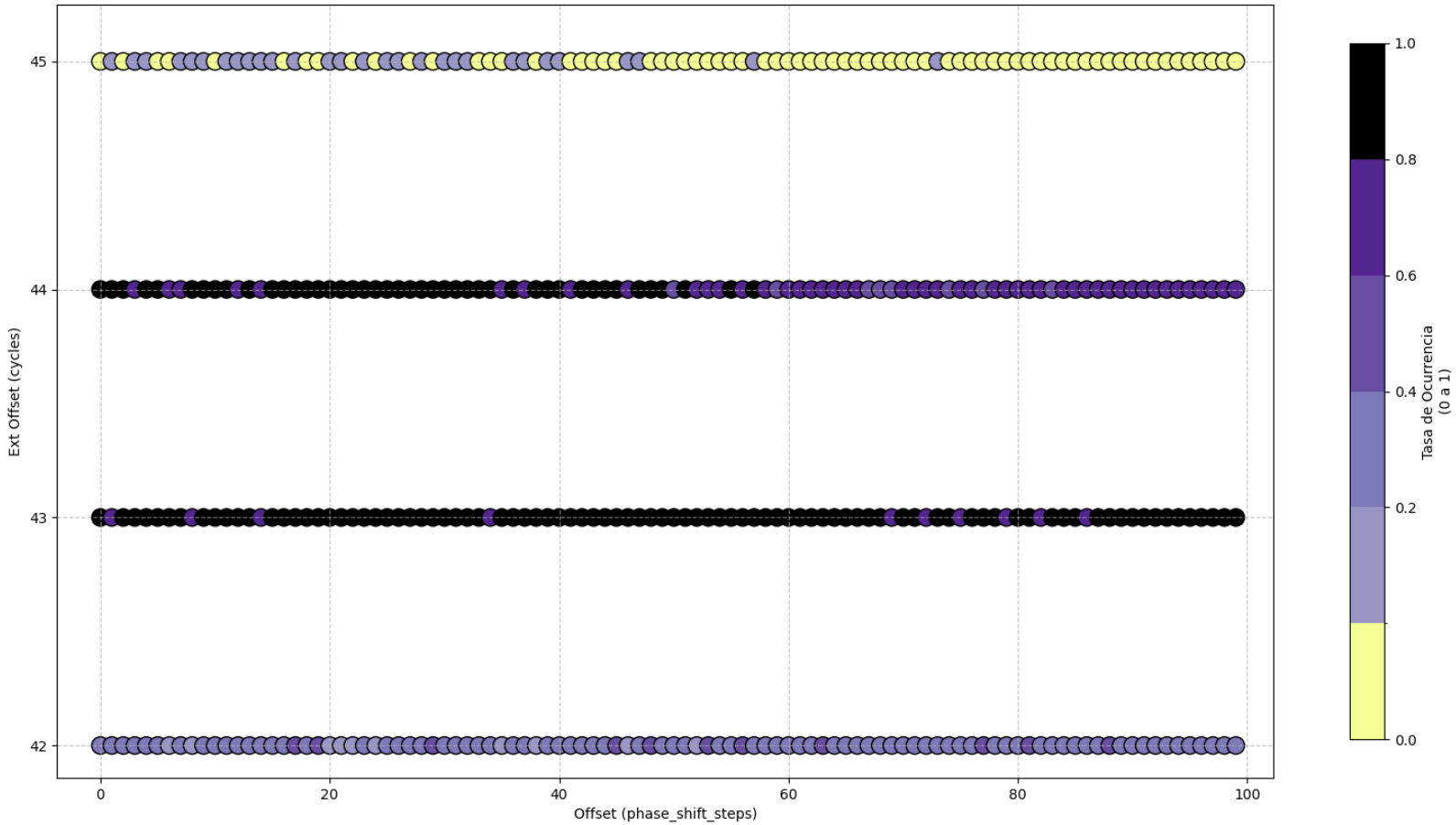


Figura: Visualización de `ext_offset` y `offset` con `repeat` fijo en 17 ciclos

Campaña con ChipWhisperer (Voltaje): Resultados

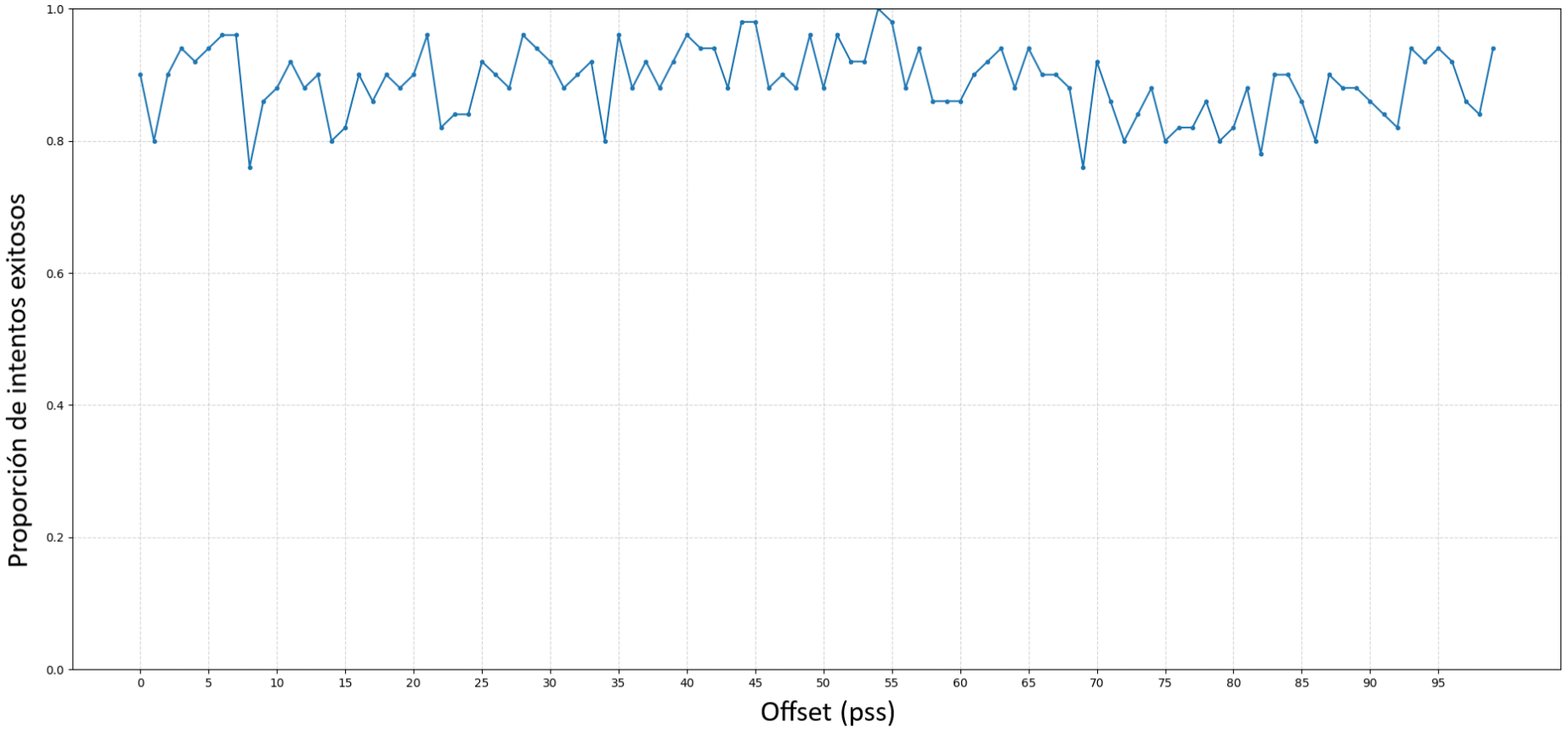


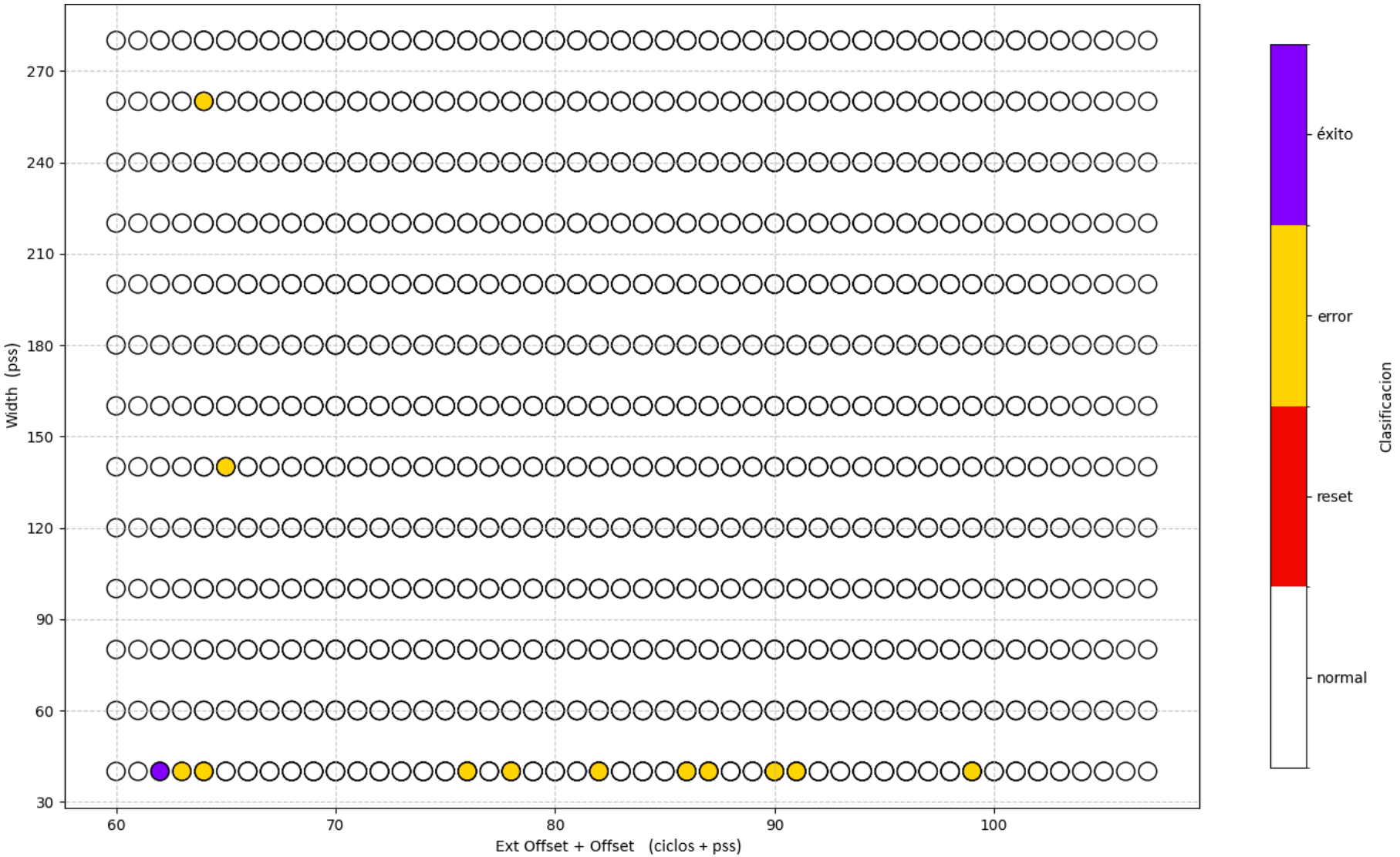
Figura: Tasa de éxito para `ext_offset` de 43 ciclos y `repeat` de 17 ciclos sobre 50 puntos

Resumen de Ataques

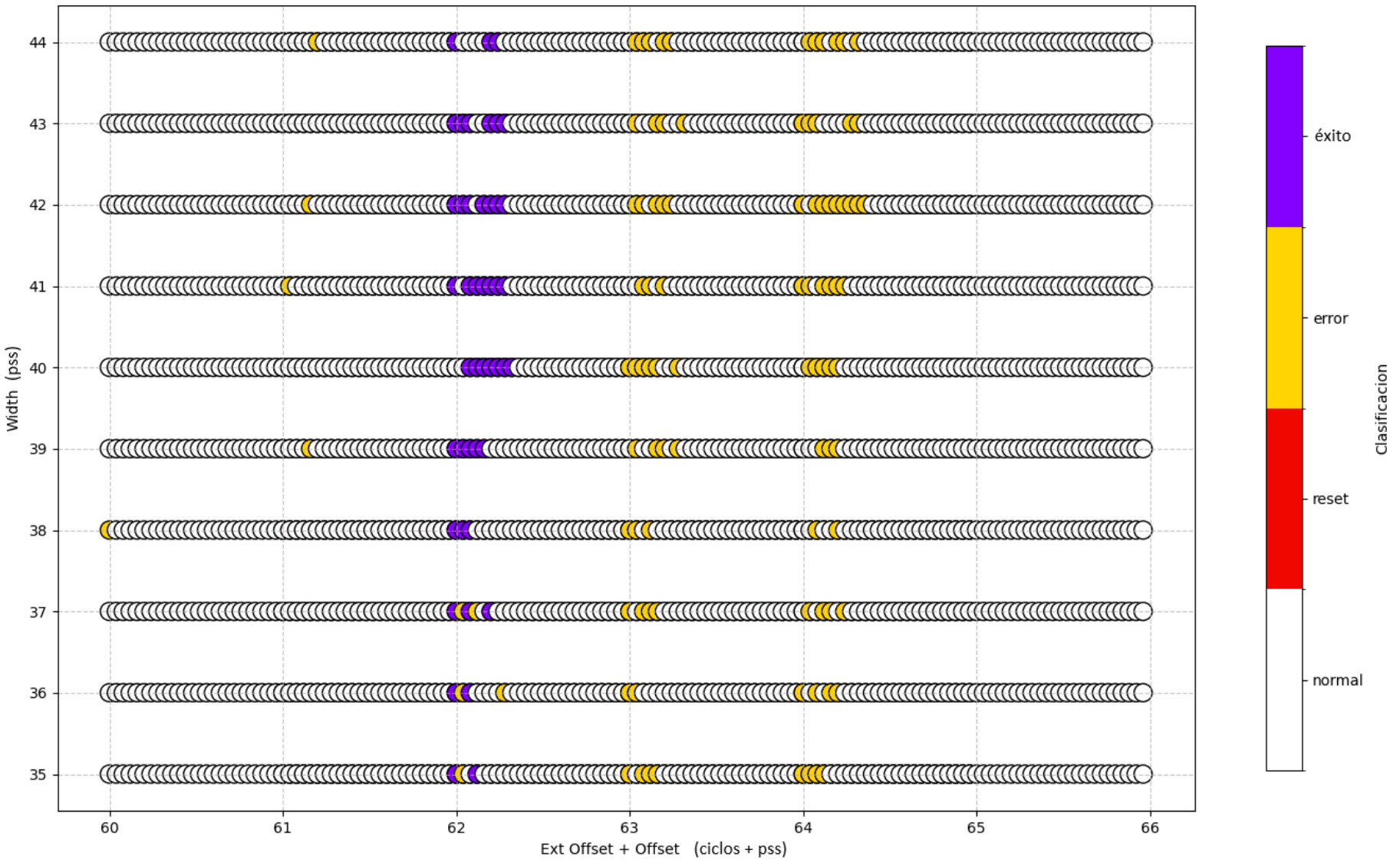
	Generación de Claves	Desencapsulación
Spider (Voltaje)	✓ (66 %)	✓ (78 %)
ChipWhisperer (Voltaje)	✓ (36 %)	✓ (100 %)
ChipWhisperer (Reloj)	✗	

Campaña con ChipWhisperer (Reloj): Primer barrido

Campaña con ChipWhisperer (Reloj): Primer barrido



Campaña con ChipWhisperer (Reloj): Segundo barrido



Campaña con ChipWhisperer (Reloj): Resultados

Configuración (`ext_offset`, `offset`, `width`)	Porcentaje de éxito (sobre 50 puntos)
$(62, 1, 41)$	$88.0\,\%$
$(62, 0, 40)$	$84.0\,\%$
$(62, 2, 40)$	$84.0\,\%$
$(62, 1, 39)$	$82.0\,\%$
$(62, 2, 41)$	$80.0\,\%$
$(62, 3, 38)$	$80.0\,\%$
$(62, 4, 40)$	$80.0\,\%$
$(62, 0, 42)$	$78.0\,\%$
$(62, 2, 39)$	$78.0\,\%$
$(62, 0, 39)$	$76.0\,\%$

Cuadro: Mayores porcentajes de éxitos por configuración

Campaña con ChipWhisperer (Reloj): Osciloscopio

Campaña con ChipWhisperer (Reloj): Osciloscopio

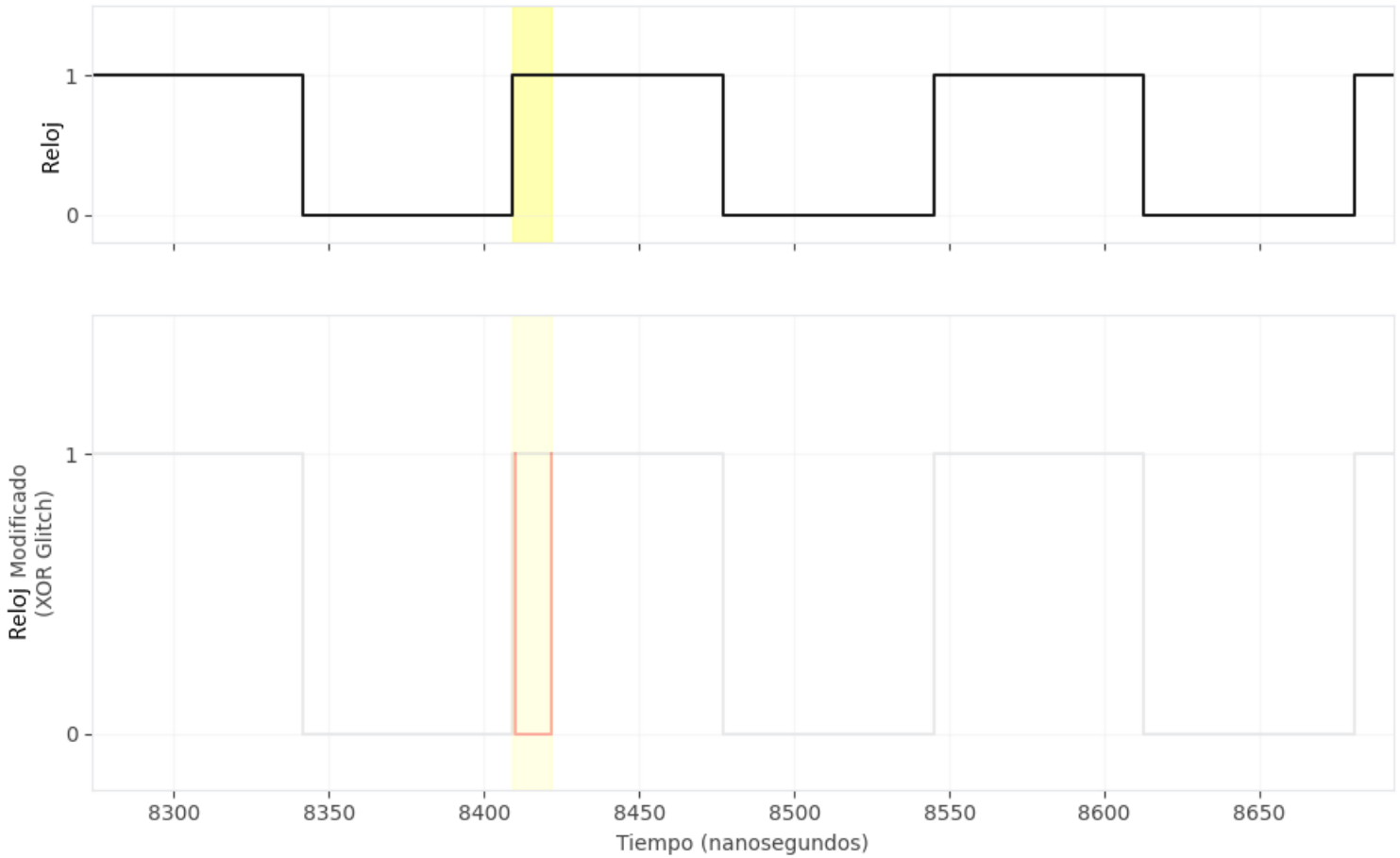


Figura: Iteración particular $(62, 20, 400)$ (`ext_offset`, `offset`, `width`) con CW por reloj desde Chaser

Campaña con ChipWhisperer (Reloj): Osciloscopio



Figura: Iteración particular $(62, 20, 400)$ (`ext_offset`, `offset`, `width`) con CW por reloj desde osciloscopio

Resumen de Ataques

	Generación de Claves	Desencapsulación
Spider (Voltaje)	✓ (66 %)	✓ (78 %)
ChipWhisperer (Voltaje)	✓ (36 %)	✓ (100 %)
ChipWhisperer (Reloj)	✗	✓ (88 %)

Conclusiones del ataque a la Desencapsulación

Conclusiones del ataque a la Desencapsulación

- **Novedad por voltaje**: Se demostró la viabilidad del ataque mediante fallos por voltaje.

Conclusiones del ataque a la Desencapsulación

- **Novedad por voltaje**: Se demostró la viabilidad del ataque mediante fallos por voltaje.
- **Alta repetibilidad** por voltaje y por reloj.

Agenda

1. Introducción: Motivación y Objetivos
2. Conceptos Previos
3. Diseño
   - 3.1 Fase de Análisis
   - 3.2 Fase de Diseño
   - 3.3 Patrones de Diseño
   - 3.4 Reingeniería de SCA
4. Experimentos sobre Kyber
   - 4.1 Atacando la NTT
   - 4.2 Atacando la Desencapsulación
5. Conclusiones

Conclusiones

Conclusiones

- La motivación principal fue el diseño y desarrollo de una herramienta orientada a la ejecución sistemática de experimentos de FI.

Conclusiones

- La motivación principal fue el diseño y desarrollo de una herramienta orientada a la ejecución sistemática de experimentos de FI.
- Se abordaron aspectos de diferentes disciplinas: ingeniería de software, criptografía postcuántica, matemática, programación de bajo nivel y seguridad de hardware.

Conclusiones

Conclusiones

- Con el fin de validar el desarrollo de la herramienta, se eligió ML-KEM por ser un algoritmo que se está implementando progresivamente en protocolos, bibliotecas y dispositivos reales.

Conclusiones

- Con el fin de validar el desarrollo de la herramienta, se eligió ML-KEM por ser un algoritmo que se está implementando progresivamente en protocolos, bibliotecas y dispositivos reales.
- Tomando como base dos publicaciones, se obtuvieron nuevos resultados al emplear nuevas clases de glitches.

Conclusiones

Ataque	Dispositivo	Inyección	Tasa de éxito
NTT (EMFI)	Spider	Voltaje	66 %
	ChipWhisperer	Voltaje	36 %
	ChipWhisperer	Reloj	No explotable
Desencapsulación (Reloj)	Spider	Voltaje	78 %
	ChipWhisperer	Voltaje	100 %
	ChipWhisperer	Reloj	88 %

Cuadro: Mejor tasa de éxito obtenida por ataque y dispositivo

Conclusiones

Conclusiones

La herramienta permitió localizar parámetros efectivos, provocar fallos de manera controlada y recopilar la información necesaria para evaluar sus consecuencias.

Conclusiones

La herramienta permitió localizar parámetros efectivos, provocar fallos de manera controlada y recopilar la información necesaria para evaluar sus consecuencias.

Además, proporciona una base extensible, facilitando nuevos estudios de seguridad física sobre sistemas embebidos.

Conclusiones

La herramienta permitió localizar parámetros efectivos, provocar fallos de manera controlada y recopilar la información necesaria para evaluar sus consecuencias.

Además, proporciona una base extensible, facilitando nuevos estudios de seguridad física sobre sistemas embebidos.

Con esto podemos hacer dispositivos más seguros.

Trabajo futuro

Trabajo futuro

- Al incorporar nuevos inyectores de glitches, hay que implementar las interfaces correspondientes.

Trabajo futuro

- Al incorporar nuevos inyectores de glitches, hay que implementar las interfaces correspondientes.
- Aunque Chaser ya puede generar código para EMFI, falta integrarlo con el control de la estación XYZ, para hacer barridos espaciales completos y visualizar los resultados.

Trabajo futuro

- Al incorporar nuevos inyectores de glitches, hay que implementar las interfaces correspondientes.
- Aunque Chaser ya puede generar código para EMFI, falta integrarlo con el control de la estación XYZ, para hacer barridos espaciales completos y visualizar los resultados.
- Como la herramienta alcanzó una primera versión funcional, conviene llevar a cabo un proceso de validación más amplio, en especial de la parte de SCA.

Muchas gracias

CHASER

¿Preguntas?